

THE INTERVIEW HANDBOOK

面试手册

计算机基础 • 七科完整手册

把基础，读成底气。

7 个科目 / 140 道问题

回答要点 • 常见追问 • 易错点 • 参考来源

整理日期：2026-10-09

本手册总结经典面试知识点，不是全网公司面经的穷举，也不代表真实招聘频次统计。题解为独立中文整理，具体版本与实现请参考题目中的原始资料。

C++ 以 C++17/20 为主；MySQL 以 8.4 LTS InnoDB 为主；Redis 以开源版常用功能为主。目录、PDF 书签及参考链接均可点击。

阅读目录

阅读目录	2
C++	8
01. 指针、引用与对象生命周期有什么区别？	8
02. const、constexpr 和 static 分别解决什么问题？	8
03. new/delete 与 malloc/free 有什么区别？	9
04. 虚函数如何实现多态？基类析构为什么常设为虚函数？	10
05. 构造和析构顺序是什么？构造函数能调用虚函数吗？	11
06. 浅拷贝、深拷贝与 Rule of Zero/Five 怎么理解？	11
07. RAII 为什么能改善异常安全？	12
08. unique_ptr 的所有权怎样转移？release 与 reset 有何不同？	13
09. shared_ptr 的控制块与线程安全边界是什么？	13
10. weak_ptr 怎样打破循环引用？为什么使用 lock？	14
11. std::move、右值引用和完美转发有什么联系？	15
12. 模板、重载、特化与 C++20 concepts 怎么区分？	16
13. lambda 的值捕获、引用捕获与生命周期怎样判断？	16
14. vector 为什么适合默认使用？扩容与失效规则是什么？	17
15. list、deque 与 vector 如何选？	18
16. map 和 unordered_map 的性能与键规则有什么差别？	18
17. 迭代器类别、算法要求与删除遍历怎样关联？	19
18. 异常安全的基本、强与不抛保证怎么区分？	20
19. 未定义行为、越界、对齐与类型别名怎样理解？	20
20. 数据竞争、atomic、mutex 与内存序是什么关系？	21
数据结构	23
01. 时间复杂度、空间复杂度与摊还分析怎么区分？	23
02. 数组与链表怎么选？链表插入真的是 O(1) 吗？	23
03. 栈、队列、循环队列与双端队列有什么区别？	24

04. 哈希表为什么平均 $O(1)$ ，却可能退化？	25
05. 二分查找如何避免边界错误？能对答案二分吗？	26
06. 常见排序算法怎样比较？稳定性有什么意义？	26
07. 快速排序如何分区？为什么会退化，怎样优化？	27
08. 归并排序为何稳定？如何统计逆序对？	28
09. 二叉堆为何能 $O(n)$ 建堆？Top K 怎么做？	29
10. BST、AVL 与红黑树有什么区别？	29
11. 二叉树遍历与重建的条件是什么？	30
12. Trie 如何支持前缀搜索？空间开销怎样优化？	31
13. 图用邻接矩阵还是邻接表？BFS 与 DFS 如何选？	31
14. 拓扑排序怎样检测有向环？顺序是否唯一？	32
15. BFS、Dijkstra、Bellman-Ford 与 Floyd 怎么选择？	33
16. 最小生成树与最短路不同在哪里？Prim 和 Kruskal 如何工作？	33
17. 并查集的路径压缩与按秩合并为什么有效？	34
18. 动态规划的状态、转移、边界与遍历顺序怎么设计？	35
19. 单调栈、双指针与滑动窗口为什么能做到线性？	36
20. 怎样设计平均 $O(1)$ 的 LRU 缓存？	36

操作系统 38

01. 进程和线程有什么区别？	38
02. 进程状态和上下文切换如何理解？	38
03. 常见 CPU 调度算法怎么比较？	39
04. fork、exec、wait 分别做什么？	40
05. 常见 IPC 方式如何选择？	40
06. 虚拟内存解决了什么问题？	41
07. 分页、分段和多级页表有什么区别？	42
08. TLB 是什么？TLB miss 等于缺页吗？	43
09. 缺页异常如何处理？什么是写时复制？	43
10. 页面置换算法和抖动是什么？	44
11. 互斥锁、自旋锁、读写锁怎么选？	45
12. 信号量和互斥锁、条件变量的关系？	45

13. 为什么条件变量要在 while 中等待?	46
14. 死锁的四个必要条件和处理方式?	47
15. 阻塞、非阻塞、同步和异步 I/O 怎么区分?	47
16. epoll 的 LT 与 ET 有什么区别?	48
17. 文件描述符、打开文件描述和 inode 的关系?	49
18. inode、目录和软硬链接怎么理解?	49
19. 文件系统日志为什么能帮助崩溃恢复?	50
20. 用户态、内核态、系统调用和中断的关系?	51

计算机网络 52

01. OSI 七层与 TCP/IP 模型怎么对应?	52
02. TCP 与 UDP 的区别和使用场景?	52
03. TCP 为什么通常需要三次握手?	53
04. TCP 关闭为什么常见四次挥手?	54
05. TIME_WAIT 为什么存在? 大量出现怎么办?	55
06. TCP 如何检测丢包并重传?	55
07. 流量控制、滑动窗口与零窗口探测?	56
08. 拥塞控制与流量控制有什么区别?	57
09. TCP 粘包与拆包如何解决?	57
10. HTTP/1.1 的连接复用和队头阻塞?	58
11. HTTP/2 怎样多路复用? 还会队头阻塞吗?	59
12. HTTP/3、QUIC 与 UDP 是什么关系?	59
13. HTTP 方法、幂等性和状态码怎么回答?	60
14. HTTP 强缓存与协商缓存如何工作?	61
15. HTTPS 与 TLS 1.3 握手如何理解?	61
16. DNS 解析过程和缓存、UDP/TCP 的关系?	62
17. IP 地址、子网掩码与 CIDR 如何计算?	63
18. 路由、最长前缀匹配和 NAT 的区别?	64
19. ARP 如何工作? IPv6 用什么?	64
20. 输入 URL 后发生什么? socket 在哪里?	65

计算机组成原理	67
01. 冯·诺依曼结构与哈佛结构有什么区别？	67
02. 一条指令从取指到完成经历什么？	67
03. ISA、微架构与 RISC/CISC 怎么区分？	68
04. 补码、符号扩展和整数溢出怎么解释？	69
05. 大端、小端和内存对齐是什么？	69
06. IEEE 754 浮点数为什么有误差？	70
07. 存储层次与局部性为什么有效？	71
08. cache 的直接映射、组相联和全相联？	72
09. cache 写直达、写回与写分配怎么区分？	72
10. 如何分析 cache miss 与平均访问时间？	73
11. MMU、页表与内存保护如何配合？	74
12. 流水线提高的是吞吐还是单条延迟？	74
13. 结构、数据和控制冒险如何处理？	75
14. 分支预测、乱序执行与顺序退休？	76
15. 中断、异常与精确异常怎么区分？	76
16. DMA 为什么减少 CPU 开销？IOMMU 做什么？	77
17. 缓存一致性、MESI 与内存一致性模型？	78
18. 什么是伪共享？如何发现与改善？	78
19. 原子指令、内存屏障和 acquire/release？	79
20. CPU 性能、Amdahl 定律、SIMD 与 NUMA 怎么分析？	80
MySQL	82
01. 为什么 InnoDB 使用 B+ 树索引？	82
02. 聚簇索引、二级索引与回表有什么关系？	82
03. 联合索引为什么有最左前缀原则？	83
04. 覆盖索引与索引条件下推有什么区别？	84
05. 哪些 SQL 写法可能降低索引利用率？	84
06. ACID 分别由什么机制保证？	85
07. MVCC 与 Read View 如何实现快照读？	86
08. RC 与 RR 的关键区别是什么？	87

09. RR 是否彻底解决幻读？	87
10. 记录锁、间隙锁与 next-key 锁怎么区分？	88
11. 死锁如何发生、排查与处理？	89
12. redo 与 undo 分别解决什么问题？	89
13. binlog 与 redo 如何通过两阶段提交协调？	90
14. 如何读懂 EXPLAIN 和 EXPLAIN ANALYZE？	91
15. JOIN 的语义与实现有哪些高频问题？	92
16. 慢 SQL 的排查步骤是什么？	92
17. 深分页为什么慢，游标分页怎么做？	93
18. 主从复制与读写分离有哪些一致性问题？	94
19. 范式、反范式与表设计如何取舍？	94
20. 如何防止 SQL 注入并管理数据库连接？	95

Redis 97

01. Redis 常用数据类型分别适合什么场景？	97
02. Redis 的类型与底层编码有什么区别？	97
03. Redis 是单线程吗，为什么能很快？	98
04. 单条命令原子是否意味着业务操作原子？	99
05. MULTI、EXEC、WATCH 能提供什么保证？	100
06. Lua 脚本的原子性与风险是什么？	100
07. RDB 快照如何工作，有什么代价？	101
08. AOF、重写与混合持久化有什么关系？	102
09. 过期 key 如何删除，TTL 有哪些细节？	103
10. 内存淘汰策略如何选择？	103
11. 主从复制、全量与增量同步如何工作？	104
12. Sentinel 解决什么问题，quorum 是什么？	105
13. Cluster 的 16384 个槽与 hash tag 怎么用？	105
14. 缓存穿透、击穿与雪崩如何区分？	106
15. MySQL 与 Redis 缓存如何保持一致？	107
16. Redis 分布式锁怎么写，有哪些不能保证的事？	107
17. Pipeline 为什么提速，和事务有何区别？	108

18. Stream 消费者组如何实现确认与重试?	109
19. 大 key 与热 key 如何发现和治理?	109
20. 如何评估 Redis 的可靠性与尾延迟?	110

C++

对象、所有权与现代语言机制

围绕 C++17/20 常见面试主题组织 20 题，解释语言规则、工程取舍与典型误区。草案链接用于延伸阅读，较新标准新增条款不属于本章默认口径。

01. 指针、引用与对象生命周期有什么区别？

语言基础 / 基础

指针是可以保存地址的对象，引用通常表达已绑定对象的别名。二者都不自动延长任意对象的生命，判断能否访问时要同时检查地址、对象类型和生命周期。

回答要点

1. 指针可以为空并且可以重新指向其他对象；引用初始化后不能改绑。给引用赋值是在修改它所指对象，而不是切换目标。接口中可选参数通常用指针，必需的借用参数可用引用。
2. 对象的存储存在不代表对象已经构造，也不代表它仍然存活。局部对象离开作用域、容器发生重分配或所有者销毁后，原先保存的地址都可能失效，必须从拥有者追踪有效期。
3. 语言没有规定引用必须无额外空间；编译器可以按调用约定实现引用。不能返回局部变量的指针或引用，也不能用空指针人为构造有效引用。临时量生命期延长还有特定规则。

常见追问

追问：返回 `const` 引用能避免拷贝吗？答：可以借用仍然存活的对象，但必须保证调用方使用期间拥有者存在。返回局部变量的 `const` 引用仍然悬空；仅仅加上 `const` 不能修复生命周期。小对象直接按值返回往往更清晰。

易错点

把引用描述成“永远安全的指针”会漏掉悬空引用。检查异常路径、异步回调和容器修改，因为它们都可能在使用前让被借用对象失效。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 引用

ISO C++ 工作草案 • 生命周期

02. `const`、`constexpr` 和 `static` 分别解决什么问题？

语言基础 / 基础

`const` 约束通过某种访问路径进行修改；`constexpr` 为编译期求值提供条件；`static` 的含义依声明位置而变，可能涉及存储期、链接或类成员共享，三者不能互换。

回答要点

1. `const int*` 表示不能通过该指针修改整数，`int* const` 表示指针本身不能改指向。`const` 成员函数约束普通成员的修改，但 `mutable` 成员可变，指针成员所指对象也不是自动只读。
2. `constexpr` 变量要求常量表达式初始化；`constexpr` 函数只在满足常量表达式要求的调用中于编译期求值，其他调用仍可发生在运行期。用它表达可计算的常量，而非性能魔法。
3. 局部 `static` 对象具有静态存储期，首次执行到初始化时完成初始化；类 `static` 成员不属于某个实例。C++11 起局部静态初始化具有并发保障，但随后对对象的读写仍需同步。

常见追问

追问：`const_cast` 去掉 `const` 后能写吗？答：只有原始对象本身并非 `const` 时，通过去掉限定的路径写入才可能合法。若原始对象就是 `const`，强行写入是未定义行为；不能根据一次测试结果宣称可写。

易错点

`const` 不代表数据物理上放在只读区，`constexpr` 函数也不保证每次都在编译期执行。讨论 `static` 时先指出声明位置，才能给出准确含义。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • `cv` 限定

ISO C++ 工作草案 • 存储期

03. `new/delete` 与 `malloc/free` 有什么区别？

语言基础 / 基础

`new` 通常完成存储分配与对象初始化，`delete` 负责析构和释放；`malloc/free` 只管理原始存储。分配函数 `operator new` 与完整的 `new` 表达式也需区分。

回答要点

1. `new T` 先获取适合类型的存储，再执行初始化；`delete p` 对适用对象先调用析构，再释放存储。`malloc` 返回原始内存，不会替你调用类构造函数，`free` 也不会调用类析构函数。
2. 资源必须使用匹配的释放机制：`new` 对应 `delete`，`new[]` 对应 `delete[]`，`malloc` 对应 `free`。混用可能破坏分配器状态或漏掉数组元素析构，其后果不是简单的固定大小内存泄漏。
3. 默认抛异常的 `new` 分配失败通常报告 `bad_alloc`；`nothrow` 形式失败返回空指针。`placement new` 在已有位置构造对象，常用于内存池，生命周期和释放存储仍须由相应协议管理。

常见追问

追问：构造函数抛异常会泄漏吗？答：普通 `new` 表达式在构造失败时会调用匹配的释放函数归还本次存储，已成功构造的成员也会析构。但构造函数内部另行申请且未交给资源管理对象的资源仍可能泄漏。

易错点

不要把 `new` 当作 `malloc` 的语法糖。手动处理原始存储时同时考虑对齐、构造、析构与匹配释放；工程代码通常优先采用容器和智能指针。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • `new` 表达式

ISO C++ 工作草案 • `delete` 表达式

04. 虚函数如何实现多态？基类析构为什么常设为虚函数？

面向对象 / 基础

通过基类指针或引用调用虚函数时，会按实际对象类型选择最终覆盖函数。若要通过基类指针删除派生对象，基类通常必须有虚析构函数，才能按多态方式正确销毁。

回答要点

1. 动态分派依赖实际对象类型和虚函数覆盖关系；`override` 让编译器检查覆盖是否成立，避免参数或 `const` 不一致形成隐藏函数。显式限定 `Base::f` 的调用可以绕过普通虚函数分派。
2. 虚函数表和虚表指针是主流编译器的常见实现策略，但不是语言强制的内存布局。讨论对象大小、多继承或性能时需要说明编译器、平台 ABI，以及可能进行的去虚拟化优化。
3. 基类若允许外部多态删除，采用公开虚析构通常最直接；若不允许这样删除，也可采用受保护的虚析构限制接口。按值传递派生对象给基类参数会切片，无法保留完整派生状态。

常见追问

追问：纯虚函数能有实现吗？答：可以在类外提供定义，但类仍可能是抽象类，派生类需完成要求的覆盖。纯虚析构函数通常必须有定义，因为销毁派生对象时仍会执行基类析构链。

易错点

把“有虚函数就一定拥有固定一个虚表指针”当标准结论不准确。经基类指针删除时不能仅根据派生类是否有虚函数判断安全，应检查基类析构。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 虚函数

ISO C++ 工作草案 • 析构函数

05. 构造和析构顺序是什么？构造函数能调用虚函数吗？

面向对象 / 基础

构造依次完成虚基类、直接基类、成员和构造函数体；成员按声明顺序初始化，析构大体反向进行。构造析构期间的虚调用受当前阶段限制，不能期待派生类行为已经存在。

回答要点

1. 最派生类负责虚基类初始化，直接基类按基类列表顺序构造，数据成员按类中声明顺序构造。初始化列表的书写次序不会改变成员顺序，因此依赖关系需要体现在成员声明顺序中。
2. 构造函数体执行前，相关基类和成员已经完成初始化。引用成员、const 成员和没有默认构造函数的成员通常需要在初始化列表中初始化，先默认构造再赋值会带来多余工作。
3. 在基类构造或析构过程中调用虚函数，分派到当前构造析构阶段对应的类，而不会调用尚未构造或已经销毁的派生部分。把多态工作放到对象完整构造后的明确操作中更易推理。

常见追问

追问：构造失败会调用完整对象析构吗？答：不会，因为完整对象尚未成功构造。但已构造完成的基类和成员会被销毁；所以将资源放进具有可靠析构函数的成员，能让构造失败路径自动回收。

易错点

容易将初始化列表顺序误认为真实执行顺序。还要警惕构造期间发布 this 给其他线程或回调，使外部观察到对象不完整的状态。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 基类与成员初始化

ISO C++ 工作草案 • 构造析构期间的对象

06. 浅拷贝、深拷贝与 Rule of Zero/Five 怎么理解？

所有权与资源 / 基础

浅拷贝复制地址或句柄，深拷贝建立独立资源。管理资源的类必须明确拷贝和移动语义；优先让标准资源成员代管，遵循 Rule of Zero，而非为每个类手写五个特殊成员。

回答要点

1. 编译器生成的拷贝通常按成员进行，裸指针成员只复制指针值，不会复制所指资源。两个对象若都把同一指针当独占所有权，就可能重复释放；若都不释放，则可能泄漏。
2. 需要手动管理资源时，析构、拷贝构造、拷贝赋值、移动构造和移动赋值要作为整体审视。可以选择深拷贝，也可以删除拷贝并支持移动；五法则并不要求所有类都允许五种操作。

3. 采用 `string`、`vector`、`unique_ptr` 等成员，往往可以由它们实现销毁和转移。拷贝赋值还要处理自赋值和异常安全：先完成新资源，再提交替换，比先释放旧资源更容易保留原状态。

常见追问

追问：写了析构函数后还会自动生成移动吗？答：用户声明的析构函数会影响隐式移动特殊成员生成。应检查实际类型的生成规则，必要时显式 `default` 或 `delete`。仅看到 `std::move` 调用不能证明一定走移动构造。

易错点

不要对包含 `string` 或虚函数等复杂对象直接用 `memcpy` 完成语义拷贝。对象字节相同不等于拥有关系正确，特殊成员是否可用还取决于成员类型。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 拷贝与移动构造

ISO C++ 工作草案 • 拷贝与移动赋值

07. RAII 为什么能改善异常安全？

所有权与资源 / 基础

RAII 把资源所有权绑定到对象生命周期：成功构造取得资源，析构释放资源。离开作用域或异常展开时，已经构造的局部管理对象会自动析构，减少遗漏释放的路径。

回答要点

1. 资源不仅是内存，也包括文件、锁、套接字和事务句柄。先设计一个明确拥有资源的对象，再通过作用域控制释放时机；让裸指针承担借用角色，可避免谁负责关闭资源的歧义。
2. 取得资源后应尽快交给管理对象；随后任一步骤抛异常，已建立的资源成员仍能析构。多个资源各自由成员管理，构造失败时语言会销毁已经完成构造的成员，降低清理分支数量。
3. 析构中应尽量使用不抛异常的释放操作，尤其不能在栈展开时继续让异常逃逸。RAII 保证资源清理，但自动保证业务操作原子性，强异常保证仍需要先准备、后提交的状态设计。

```
#include <mutex>
void update(std::mutex& mutex, int& value) {
    std::lock_guard<std::mutex> lock(mutex);
    ++value;
}
```

常见追问

追问：`lock_guard` 与 `unique_lock` 怎么选？答：只需要固定作用域加锁时使用 `lock_guard` 很直接；需要延迟加锁、提前解锁或与条件变量配合时可用 `unique_lock`。两者都表达锁的所有权，但接口能力与成本不同。

易错点

RAII 不保证进程被强制终止时一定执行析构，也不能代替事务回滚策略。它解决常规 C++ 控制流中的资源生命周期，仍需要正确表达资源拥有关系。

延伸阅读 / 原始参考资料

C++ Core Guidelines • R.1 RAII

ISO C++ 工作草案 • 异常中的构造与析构

08. `unique_ptr` 的所有权怎样转移？`release` 与 `reset` 有何不同？

所有权与资源 / 基础

`unique_ptr` 表达独占所有权，不能复制但可移动。`reset` 处理原资源并接管新资源，`release` 交出指针并放弃管理；调用 `release` 后必须落实新拥有者。

回答要点

1. 优先使用 `make_unique` 构造独占对象，把所有权转移写成 `std::move`。转移后的源 `unique_ptr` 为空；仅借用对象时传指针或引用即可，不必把所有调用都设计成智能指针参数。
2. `reset` 会用删除器销毁原来管理的对象，然后接管给定指针；`release` 返回原指针而不销毁对象。调用 `release` 后如果无人负责释放，就会泄漏，因此它主要用于接入明确的所有权接口。
3. `unique_ptr` 可以定制删除器以管理特定资源，但删除器类型也会影响对象大小和可移动性。
`unique_ptr<T[]>` 管理数组；转为 `unique_ptr<Base>` 并经默认删除器删除派生对象时要检查虚析构。

常见追问

追问：`unique_ptr` 能放进 `vector` 吗？答：可以，容器可通过移动转移元素所有权。插入已有指针通常用 `push_back(std::move(p))`，成功后 `p` 为空。不能直接复制一个 `vector<unique_ptr<T>>` 来获得独立副本。

易错点

`get` 得到的是借用地址，不能另建一个独占所有者再交给另一个 `unique_ptr`，也不能手动 `delete` 后保留原拥有者。两个所有者会导致重复释放。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 独占智能指针

ISO C++ 工作草案 • 独占指针修改操作

09. `shared_ptr` 的控制块与线程安全边界是什么？

所有权与资源 / 进阶

`shared_ptr` 通过控制块共享对象所有权，最后一个强拥有者释放对象。共享控制块的不同指针实例可安全维护引用计数，但被指向对象和同一个指针实例的并发修改仍需额外同步。

回答要点

1. 控制块记录强拥有计数、弱引用关联以及删除方式等信息。`make_shared` 常见实现会将对象与控制块放在一次分配中，有利于减少分配，但弱引用仍存活时也可能延后整块存储回收。
2. 不同 `shared_ptr` 实例即使共享控制块，维护自身引用计数不等于保护对象内容。多个线程通过它们修改同一个普通对象仍会发生数据竞争；必须用互斥锁或对象自身的同步机制。
3. 不能从同一裸指针分别构造两个独立 `shared_ptr`，它们会建立独立的控制块并重复销毁资源。需要自引用时使用 `enable_shared_from_this`，并且对象已经进入 `shared_ptr` 管理后才能使用。

常见追问

追问：`use_count()==1` 可以据此无锁修改对象吗？答：不应把它当并发独占证明，计数可能马上变化，其他线程也可能保有借用地址。`C++20 atomic<shared_ptr<T>>` 可同步指针发布，但仍不会自动保护 `T` 的内部状态。

易错点

“`shared_ptr` 线程安全”必须说清保护范围。别混淆控制块、指针实例与目标对象三种不同共享状态；共享所有权也会增加计数维护和生命周期推理成本。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 共享智能指针

ISO C++ 工作草案 • 原子共享指针

10. `weak_ptr` 怎样打破循环引用？为什么使用 `lock`？

所有权与资源 / 进阶

`weak_ptr` 观察 `shared_ptr` 管理的对象而不增加引用计数。将回指或观察关系改成弱引用，可以避免强所有权环；使用前通过 `lock` 原子地尝试取得临时强所有权。

回答要点

1. 若 `A` 和 `B` 互持 `shared_ptr`，外部拥有者消失后两者强计数仍非零，无法回收。先确定哪条边代表真正拥有关系，再将父节点回指或观察者边改为 `weak_ptr`，而非随意全部改弱引用。
2. `expired` 只报告检查时刻强计数是否为零，不能保证后续使用仍有效。`lock` 将检查与取得拥有权作为原子操作：成功返回有效 `shared_ptr`，失败返回空指针，成功期间目标对象保持存活。
3. `weak_ptr` 不让对象本身继续存活，但需要控制块来判断状态，控制块会保留到相关弱引用释放。对象销毁与控制块存储回收是两个时刻，不能简单认为弱引用完全不产生资源成本。

```
if (auto owner = weak.lock()) {
    // owner 存活期间才使用对象
    owner->work();
}
```

常见追问

追问：缓存为什么会使用 `weak_ptr`？答：缓存可以记录已存在对象但不主动延长其生命。查找时 `lock` 成功则复用，失败则重新构造并更新弱引用。并发场景还需要同步缓存索引，避免重复创建及结构数据竞争。

易错点

先调用 `expired` 再直接使用先前保存的裸指针，仍存在时间窗口。安全使用通常要检查 `lock` 的返回值，并持有得到的 `shared_ptr` 到访问结束。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 弱智能指针

ISO C++ 工作草案 • 弱引用观察操作

11. `std::move`、右值引用和完美转发有什么联系？

现代 C++ / 进阶

`std::move` 转换值类别，本身不搬运资源；是否移动由重载解析和类型能力决定。完美转发以转发引用和 `std::forward` 保留值类别，使包装函数正确选择下游重载。

回答要点

1. 左值通常表示有身份的对象表达式，右值包含纯右值和将亡值。具名的 `T&&` 参数表达式本身仍是左值；`std::move` 把表达式转为将亡值，随后才可能匹配移动构造或移动赋值。
2. 移动通常转移底层资源，但移动后的标准库对象一般只保证有效、状态未指定，除非类型有更强约定。不能统一断言所有 `string` 或 `vector` 移动后都为空，也不能无条件继续读取业务值。
3. 模板推导中的 `T&&` 可形成转发引用，配合引用折叠和 `std::forward<T>` 保留左值右值属性。普通固定类型的 `T&&` 不自动成为转发引用；移动 `const` 对象通常仍会走拷贝，因为常见移动接口要修改源。

常见追问

追问：`return std::move(local)` 总比 `return local` 快吗？答：不一定，显式 `move` 可能阻碍命名返回值优化。优先直接 `return local`，让编译器使用可选 NRVO 或隐式移动；C++17 的部分纯右值返回场景还有保证的拷贝消除。

易错点

把 `std::move` 翻译成“立即移动”会误判 `const`、无移动函数和重载情况。移动构造的 `noexcept` 属性也会影响容器重分配时是否选择拷贝。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • `move` 与 `forward`

ISO C++ 工作草案 • 拷贝消除

12. 模板、重载、特化与 C++20 concepts 怎么区分？

现代 C++ / 进阶

模板描述可按类型或值实例化的通用代码，重载为同名调用提供候选函数，特化为特定模板参数提供实现。C++20 concepts 用约束表达模板需求，使接口意图和错误定位更明确。

回答要点

1. 模板不是运行期解释器，编译器会在需要时进行推导与实例化。模板定义通常放在头文件，使使用位置能看到定义；特定场景也可通过显式实例化把支持的参数集合集中到源文件。
2. 函数模板可以重载并进行全特化，但不能像类模板那样直接做偏特化。实现部分类型差异时通常采用函数重载、类型特征或类模板偏特化，要区分候选选择与模板实例化两个阶段。
3. SFINAE 会在特定替换失败场景移除候选，不能把所有模板错误都当作替换失败。concepts 和 requires 提供更直接的约束表达；它们检查声明需求，不替你证明算法的业务正确性。

常见追问

追问：模板为什么可能增大二进制？答：不同参数可能实例化不同代码，过多组合增加编译时间和体积。可以把与类型无关的工作提取到普通函数，或采用有限显式实例化；同时也应让编译器优化与实际测量指导取舍。

易错点

不要把模板等同于宏：模板仍受类型系统与作用域规则约束。泛型代码的约束、实例化位置与重载规则需要一起解释，不能只背一个 template 语法。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 模板

ISO C++ 工作草案 • 模板约束

13. lambda 的值捕获、引用捕获与生命周期怎样判断？

现代 C++ / 基础

lambda 生成一个闭包对象，捕获决定它保存外部状态的方式。值捕获通常保存副本，引用捕获借用原对象；闭包被异步保存时，必须确保它借用的对象仍然活着。

回答要点

1. 值捕获在创建闭包时获取当时的值，默认调用运算符对捕获成员呈 const 访问；mutable 允许修改闭包中的副本，但不表示外部原变量被修改。引用捕获则通过借用访问原状态。
2. 捕获 this 保存的是对当前对象的访问关系，闭包不会因此自动拥有对象。C++17 的 [*this] 可复制对象本身，但复制的内部裸指针也不会自动深拷贝其资源，需要检查成员拥有语义。

3. 初始化捕获可以把 `unique_ptr` 移入闭包，明确延长所管对象生命。返回闭包或投递线程任务时，优先显式列出捕获对象；按引用捕获局部变量后让闭包逃离作用域会造成悬空访问。

常见追问

追问：捕获 `unique_ptr` 的 `lambda` 能放入 `std::function` 吗？答：在 C++17/20 常用 `std::function` 中，目标通常要求可复制，因此仅有移动能力的闭包不满足要求。可直接用模板参数承载闭包，或重新设计共享所有权，不能盲目复制。

易错点

默认 `[&]` 在同步短作用域内便利，在异步回调里却易隐藏借用关系。判断安全时先写出闭包的存活时间，再检查每个捕获对象的有效期。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • `lambda` 表达式

ISO C++ 工作草案 • `std::function` 构造

14. `vector` 为什么适合默认使用？扩容与失效规则是什么？

STL 与容器 / 基础

`vector` 提供连续存储与常数时间随机访问，尾部追加为摊还常数时间。发生重分配时，指向元素的引用、指针与迭代器都失效；没有重分配时也要按插入删除位置判断有效性。

回答要点

1. `size` 表示已经存在的元素数量，`capacity` 表示当前存储可容纳的数量。`reserve` 提前规划容量但不会增加 `size`，也不能把未构造容量当作现存元素访问；`resize` 才会改变实际元素个数。
2. 尾部追加可偶尔触发线性迁移，因此是摊还 $O(1)$ ，单次不保证常数。标准不规定固定两倍增长倍率；中间插入删除需要移动后方元素，即使容量充足也通常是线性成本。
3. 重分配会让所有元素地址和迭代器失效；没有重分配的插入仍会让插入点及其后迭代器失效，`erase` 也会影响删除点及之后。保存下标有时更方便，但插删仍可能改变逻辑对应关系。

常见追问

追问：`reserve` 后能保证所有地址不变吗？答：只要后续大小不超过预留容量，尾部追加可避免该阶段重分配；中间插删仍可能移动元素。调用 `reserve` 请求更大容量本身也可能重分配，缓存地址必须在正确时机更新。

易错点

`vector<bool>` 是特殊化，不应套用普通 `vector<T>` 的真实 `bool` 引用和连续 `bool` 对象假设。容器迁移策略还依赖元素拷贝、移动和异常保证。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • vector 规范
ISO C++ 工作草案 • vector 修改操作

15. list、deque 与 vector 如何选？

STL 与容器 / 基础

先按操作需求选择容器：vector 连续存储和随机访问强；deque 支持两端插入及随机访问但不承诺整体连续；list 支持已知位置的节点操作，却缺少随机访问且缓存局部性较差。

回答要点

1. list 的插入删除在已知迭代器位置通常为常数时间，并能保留未删除节点的迭代器与引用。若先按下标查找位置，遍历已经需要 $O(n)$ ，不能把整个业务操作描述为常数时间。
2. deque 通常采用分段结构，两端增长不要求搬迁全部元素；语言接口保证随机访问，但不能将整段数据当作连续数组传给 C 接口。两端插入的迭代器失效规则与元素引用规则也不同。
3. vector 通常是读多、遍历多场景的良好默认选择，连续存储可减少指针跳转与分配开销。list 的节点额外空间、分配成本和实际缓存表现都要计入，不能只看插入删除的渐进复杂度。

常见追问

追问：deque 尾部插入会让引用失效吗？答：两端插入会使迭代器失效，但不影响既有元素引用有效性；中间插入则会使迭代器与引用都失效。不能将同一个容器的迭代器与引用稳定性混为一谈。

易错点

“链表插入快”省略了定位成本和分配成本。选择容器应根据实际访问模式、地址稳定性要求和测量结果，而不是根据某个复杂度单项排名。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • list 规范
ISO C++ 工作草案 • deque 规范

16. map 和 unordered_map 的性能与键规则有什么差别？

STL 与容器 / 基础

map 维护按比较器定义的键顺序，查找通常为对数时间；unordered_map 通过哈希组织数据，查找平均常数、最坏线性。是否需要范围查询、稳定性能和遍历顺序是选择的重要依据。

回答要点

1. map 的比较器应满足严格弱序，键等价由双方都不小于对方判定，不必等同于 `operator==`。常见实现是红黑树，但标准关注接口、复杂度和失效保证，不强制这一具体树结构。

2. `unordered_map` 要求相等键具有相同哈希值；良好的散列分布和合适负载因子支持平均常数查找。恶意碰撞或糟糕哈希可导致线性退化，还需考虑字符串键的哈希计算成本。
3. `unordered_map` 重哈希通常使迭代器失效，但不使元素引用和指针失效。`operator[]` 在键不存在时会插入并默认构造值；仅想查询时可用 `find`，避免查询行为意外修改容器。

常见追问

追问：查找最快就总选哈希表吗？答：有序遍历、`lower_bound` 或键区间检索更适合 `map`；哈希表还可能有额外桶空间及碰撞退化。确定访问模式后再比较，不应把平均 $O(1)$ 当作任何数据上的确定低延迟。

易错点

不要在容器存储期间改变会影响排序或哈希等价的键状态。自定义比较器与哈希函数还应覆盖边界值，并且保持等价关系的一致性。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 有序关联容器要求

ISO C++ 工作草案 • 无序关联容器要求

17. 迭代器类别、算法要求与删除遍历怎样关联？

STL 与容器 / 进阶

迭代器把容器访问方式交给算法，但不同类别支持不同操作。调用前既要满足算法的迭代器要求，也要遵守修改容器后的失效规则；删除遍历应使用删除操作返回的新位置继续。

回答要点

1. 输入迭代器主要支持单遍读取，前向迭代器可多遍遍历，双向迭代器支持退后，随机访问迭代器支持位移与距离。不能因为容器能遍历，就认为任意算法或 `it+n` 表达式都适用。
2. `std::sort` 要求随机访问迭代器，因此 `vector` 和 `deque` 常可使用，而 `list` 应使用自身 `sort` 成员。比较器必须满足严格弱序；用 `<=` 写比较器会让相等值也互相小于，破坏算法前提。
3. 遍历时删除元素不要自增已经失效的迭代器，通常以 `it=container.erase(it)` 继续。C++17 可用 `erase-remove` 模式清理序列；C++20 的 `erase_if` 简化部分容器的条件删除接口。

```
for (auto it = values.begin(); it != values.end(); ) {
    if (*it < 0) it = values.erase(it);
    else ++it;
}
```

常见追问

追问：`lower_bound` 在 `list` 上也是 $O(\log n)$ 吗？答：比较次数可以是对数，但移动迭代器的次数可能是线性的，整体时间不能按随机访问数组理解。有序关联容器优先使用其成员 `lower_bound`，让搜索利用树结构。

易错点

不要把 `end` 当作有效元素解引用。算法比较次数与实际迭代器移动成本不是同一指标，判断复杂度时需写明容器和算法接口。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 迭代器要求

ISO C++ 工作草案 • 排序与二分算法

18. 异常安全的基本、强与不抛保证怎么区分？

可靠性与并发 / 进阶

基本保证要求失败后仍维持有效状态且不泄漏资源；强保证要求失败后可观察状态保持原样；不抛保证要求操作不让异常逃逸。异常安全要围绕状态提交与资源生命周期设计。

回答要点

1. 基本保证不等于没有任何变化，只保证对象不变量和资源管理仍可靠。强保证可以理解成成功则提交、失败则回滚；例如先把新数据建在临时对象中，全部成功后再以不抛的交换提交。
2. 栈展开会析构已经构造的自动对象，是 RAII 发挥作用的机制。析构若在异常展开过程中再抛出并逃逸，会导致终止；资源释放应尽可能不抛，失败信息可通过单独操作或其他通道报告。
3. `noexcept` 是对异常逃逸行为的承诺，不会把内部所有调用变成不会失败的调用。若异常逃出 `noexcept` 函数，会调用 `terminate`；条件 `noexcept` 可依据成员操作能力描述类型的移动和交换属性。

常见追问

追问：`vector` 为什么可能不用移动而改用拷贝？答：当移动可能抛异常而类型可拷贝时，重分配过程可能选择拷贝，以便失败后保留旧元素。若类型不可拷贝且移动抛出，部分操作的强保证会受限制，需要查具体规范。

易错点

`catch(...)` 后忽略错误并不自动提供强保证，数据可能已经部分修改。不要为追求表面无异常而随意加 `noexcept`，应先验证内部操作与终止语义是否符合需求。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 异常处理

ISO C++ 工作草案 • 异常说明

19. 未定义行为、越界、对齐与类型别名怎样理解？

可靠性与并发 / 进阶

未定义行为表示语言不再约束该次执行的结果，不能用一次正常输出证明安全。越界、悬空访问、部分不合法类型访问和有符号溢出都是典型风险，必须从语言规则判断。

回答要点

1. 数组下标要求落在有效元素范围内；尾后指针可用于比较和终止遍历，但不能解引用。已经释放的存储可能仍保留旧字节，这不能恢复对象生命周期，也不能让读取重新变得合法。
2. 不同类型对象对存储的对齐要求不同，把任意字节地址强转成某类型指针并解引用可能同时违反对齐、生命周期与类型访问规则。`reinterpret_cast` 只改变解释方式，不负责建立有效对象。
3. 不要依靠互不相关的类型指针读取同一对象表示来做常规类型转换。检查位表示可用允许的字节访问方式；需要表示层转换时可针对适用类型使用 `memcpy` 或 C++20 `bit_cast`，并满足相关前提。

常见追问

追问：未定义行为是不是必定崩溃？答：不是，可能暂时正常、输出错误或被优化后出现难以解释的结果。调试构建与优化构建差异也很常见。使用地址与未定义行为检查工具辅助发现问题，但工具未报错不能代替证明合法。

易错点

“机器允许未对齐读”不等于 C++ 规则允许该类型访问。把硬件现象、编译器扩展和语言保证分开说明，避免用平台巧合回答跨平台问题。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 值类别与类型访问

ISO C++ 工作草案 • 对齐要求

20. 数据竞争、atomic、mutex 与内存序是什么关系？

可靠性与并发 / 进阶

并发访问同一内存位置，至少一方写入且缺少适当同步，可能构成数据竞争并导致未定义行为。`atomic` 保护单项原子操作，`mutex` 可保护复合不变量，内存序决定跨线程可见性关系。

回答要点

1. `atomic` 读写解决该对象的原子访问，不能自动保护旁边的普通对象。即使两个字段分别是 `atomic`，也不能保证读取者看到的是同一版本组合；维护多个字段的一致性常用同一互斥锁。
2. `relaxed` 保证原子性和该原子对象的修改顺序，但不建立一般发布关系。`release` 写与读取到对应值的 `acquire` 读可以建立同步关系，使发布前写入的数据对接收线程可见，前提是协议完整。
3. 顺序一致序通常更易推理，但原子操作不保证无锁，也不保证算法无阻塞。`volatile` 不是线程同步机制；避免数据竞争还需要明确对象生命期、锁顺序和所有参与线程的访问约定。

```
#include <atomic>
std::atomic<int> count{0};
void record() {
    count.fetch_add(1, std::memory_order_relaxed);
}
```

常见追问

追问：i++ 为什么不能靠普通 int 完成并发计数？答：它包含读、修改和写，多个线程会互相覆盖且构成数据竞争。用 atomic 的 fetch_add 可完成原子计数；若计数还控制其他数据发布，应同时设计适当的内存序与生命周期协议。

易错点

不要只背 acquire/release 名称，要指出哪个发布与哪个读取同步。原子计数正确也不意味着整个业务操作正确，多步检查再修改仍可能需要锁。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 数据竞争

ISO C++ 工作草案 • 原子内存序

数据结构

从复杂度到结构与算法选择

20 题覆盖线性结构、查找排序、树图与常见算法模式。复杂度默认采用常见 RAM 模型，并写明平均、最坏和摊还条件；实现常数、内存与缓存表现仍需具体测量。

01. 时间复杂度、空间复杂度与摊还分析怎么区分？

复杂度与线性结构 / 基础

复杂度描述资源消耗随输入规模的增长，不能直接等同于某次运行耗时。最坏、平均和摊还分析回答不同问题；空间分析还要说明是否计入输入、输出以及递归调用栈。

回答要点

1. O 给出渐进上界， Θ 描述紧确增长量级；分析嵌套循环时应累加真实执行次数，不应看到两层循环就一律判成平方。若内层范围随外层变化，应先写出求和式再求增长阶。
2. 平均复杂度依赖输入或随机过程的概率假设，摊还复杂度分析一串操作的总体成本，不要求概率分布。动态数组倍增时，少数线性迁移的成本可分摊到很多尾部追加操作中。
3. 空间复杂度通常关注额外空间，但必须明确口径。递归深度 h 常产生 $O(h)$ 调用栈；输出一个含 n 个元素的答案至少需要相应输出空间，不能一边省略它一边声称完整任务只占常数空间。

常见追问

追问：动态数组追加为什么不是每次 $O(1)$ ？答：容量充足时可以常数完成，容量不足时要分配并迁移已有元素，单次可能 $O(n)$ 。按几何倍率扩容的一串 n 次追加总成本可为 $O(n)$ ，因此每次摊还 $O(1)$ ，这不是单次最坏保证。

易错点

没有输入分布就随意说“平均很快”并不严谨。还要区分数据规模、键长度和输出量等变量，字符串哈希不能总按与字符数无关的常数操作处理。

延伸阅读 / 原始参考资料

Open Data Structures • 动态数组与摊还成本

Princeton Algorithms • 复杂度速查

02. 数组与链表怎么选？链表插入真的是 $O(1)$ 吗？

复杂度与线性结构 / 基础

数组支持常数随机访问但中间插删要搬移，链表通过节点链接便于已知位置的结构调整，却需要线性查找。链表 $O(1)$ 插入删除通常以已经获得适当节点或前驱为前提。

回答要点

1. 连续数组可按下标直接定位，遍历通常有较好的缓存局部性；中间插入或保序删除需要搬移后续元素。若不要求保持顺序，有时可用尾元素覆盖删除位置，但会改变排列语义。
2. 链表第 k 个节点一般要从头或尾逐步查找。单链表删除给定节点时通常需要前驱，双链表有前后指针可在节点已知时直接调整；节点分配与额外指针也增加内存及常数开销。
3. 选择结构时先列出访问模式：随机读取多倾向数组，频繁在已知节点旁操作且要求节点地址稳定时可考虑链表。不能只比较插删复杂度，忽略定位、缓存、分配与维护引用的成本。

常见追问

追问：只有单链表待删节点指针能 $O(1)$ 删除吗？答：可在存在后继且允许改变节点内容时，将后继内容搬到当前节点再删后继，但这改变节点身份，尾节点也无法这样处理。若身份或外部引用有要求，就仍需要前驱信息。

易错点

链表的常数删除前提要说完整：已知位置、足够的相邻链接信息以及允许的节点语义。用一次搬移数值的技巧代替一般链表删除会漏掉尾节点和身份约束。

延伸阅读 / 原始参考资料

Open Data Structures • 链表取舍

Open Data Structures • 双向链表

03. 栈、队列、循环队列与双端队列有什么区别？

复杂度与线性结构 / 基础

栈按后进先出工作，队列按先进先出工作，双端队列允许两端操作。循环队列利用取模复用数组空间，关键是清楚定义头尾指针含义，以及如何区分满和空。

回答要点

1. 栈适合括号匹配、表达式处理和深度优先过程；队列适合广度优先搜索与按到达顺序处理。抽象数据类型约束的是操作与顺序，实现可以使用数组或链表，而不是某一种固定布局。
2. 循环数组队列让 `head` 指向待取元素，`tail` 指向下一写入位置是一种常见约定。可以额外记录 `size`，也可以留一个空槽区分满和空；容量计算和判满公式必须与选定约定一致。
3. 双端队列支持头尾插入删除，适合维护滑动窗口候选项。数组实现要处理环绕与扩容，链表实现要维护两端链接；接口复杂度还取决于固定容量、扩容策略和是否需要随机访问。

常见追问

追问：如何用两个栈实现队列？答：入队压入输入栈，出队在输出栈为空时把输入栈所有元素倒过去，再弹出输出栈顶。一次倒栈可能线性，但每个元素只经历有限次转移，整串操作的单次成本为摊还 $O(1)$ 。

易错点

不要混用尾指向“最后一个元素”和“下一写入位置”两种约定，否则容易少算容量或覆盖数据。并发队列还需要额外同步，环形结构本身不保证线程安全。

延伸阅读 / 原始参考资料

Open Data Structures • 数组队列

Open Data Structures • 数组双端队列

04. 哈希表为什么平均 $O(1)$ ，却可能退化？

查找与排序 / 基础

哈希表将键映射到桶或探测位置，借助良好散列与受控负载取得平均常数时间操作。碰撞无法完全避免，处理策略、负载因子和键的哈希成本决定实际表现，最坏可线性退化。

回答要点

1. 拉链法把同桶元素放在链表或其他结构中，查询需要在桶内检查键相等。负载因子等于元素数除以桶数；分布均匀且负载受控时，桶内平均工作量有限，但集中碰撞会拉长查询路径。
2. 开放寻址把元素留在桶数组内，碰撞后按照探测序列寻找位置。删除不能随意把槽位标成从未使用，否则会截断后续查询链；通常使用墓碑或调整探测簇，并适时清理与扩容。
3. 扩容重哈希需要重新布置大量元素，单次可能线性；按合适增长策略可分摊维护成本。字符串或组合键的哈希计算随键长度增长，自定义哈希还必须保证相等键具有相同哈希值。

常见追问

追问：碰撞是不是说明哈希函数错了？答：不是，有限输出范围映射更大键空间，碰撞通常不可避免。错误在于分布长期很差，或相等键被算出不同哈希导致查询失效。工程上要同时检查等价规则、负载和攻击性输入。

易错点

哈希相同不表示键相等，仍要用等价比较确认。平均常数结论依赖分布与负载条件；不能把一次重哈希或恶意碰撞的最坏延迟藏在 $O(1)$ 说法里。

延伸阅读 / 原始参考资料

Open Data Structures • 拉链哈希

Open Data Structures • 线性探测

05. 二分查找如何避免边界错误？能对答案二分吗？

查找与排序 / 基础

二分查找利用有序或单调条件反复缩小候选区间。先定义区间开闭和循环不变量，再统一更新边界；寻找第一个满足条件的位置时，返回值可能等于区间末尾，必须检查。

回答要点

1. 采用半开区间 $[left, right)$ 查第一个不小于目标的元素时， $left$ 左侧已知更小， $right$ 及其右侧已知不小。中点不满足则令 $left = mid + 1$ ，满足则令 $right = mid$ ，直到边界相遇。
2. 中点通常写成 $left + (right - left) / 2$ ，避免直接相加造成溢出；仍需选用能表示区间长度的类型。空数组、单元素、重复值、目标在两端之外，都应按照同一不变量得到明确结果。
3. 答案二分要求判定函数存在单调转折，例如容量足够后继续增大仍可完成任务。总复杂度是判定成本乘二分轮数；不能只说 $O(\log n)$ ，也不能对没有单调性的业务条件直接套用。

```
#include <vector>
#include <cstdint>
std::size_t lower(const std::vector<int>& a, int x) {
    std::size_t l = 0, r = a.size();
    while (l < r) {
        auto m = l + (r - l) / 2;
        if (a[m] < x) l = m + 1;
        else r = m;
    }
    return l;
}
```

常见追问

追问：二分找到目标后为什么还可能继续查？答：存在重复元素时，普通等值查找只保证找到某个位置。要找最左或最右边界，需要在满足条件时继续缩小相应一侧，或分别使用 `lower_bound` 与 `upper_bound`，形成重复值范围。

易错点

比较次数对数不等于所有容器上的总时间对数；链表定位中点可能需要线性移动。必须说明有随机访问支持，或算法能直接利用搜索树结构。

延伸阅读 / 原始参考资料

Princeton Algorithms • 二分查找

ISO C++ 工作草案 • 二分算法

06. 常见排序算法怎样比较？稳定性有什么意义？

查找与排序 / 基础

比较排序通常以 $O(n \log n)$ 为通用高效目标；选择时还要看最坏时间、额外空间、稳定性与输入特征。稳定排序保持相等键的原始相对顺序，在多关键字处理和记录排序中很有用。

回答要点

1. 插入排序最坏 $O(n^2)$ ，但对近乎有序的小数据常有效，常见实现可以稳定；选择排序通常 $O(n^2)$ 且不稳定。冒泡排序也常为平方时间，是否早停影响有序输入上的最佳表现。
2. 常见数组归并排序稳定且最坏 $O(n \log n)$ ，代价是 $O(n)$ 辅助数组；堆排序最坏 $O(n \log n)$ ，常见实现额外空间常数但不稳定。普通快速排序平均高效，最坏仍可能平方且不稳定。
3. 计数与基数排序利用整数范围或位数等额外结构，不能用来否定比较模型下的 $\Omega(n \log n)$ 下界。计数排序复杂度通常含键范围 K ，范围巨大时即便元素很少也可能付出很大空间成本。

常见追问

追问：怎样按两个键排序并保留顺序？答：可以直接提供字典序比较器；也可以先按次要键排序，再按主要键进行稳定排序，让主要键相等的记录保留次要键顺序。若第二次排序不稳定，之前的次序就可能丢失。

易错点

“原地”是否允许递归栈要明确，不要把快排的分区辅助空间与整个递归空间混为一谈。排序性能还受比较器成本、缓存与数据分布影响。

延伸阅读 / 原始参考资料

Princeton Algorithms • 排序特征

Open Data Structures • 计数与基数排序

07. 快速排序如何分区？为什么会退化，怎样优化？

查找与排序 / 进阶

快速排序选择枢轴，把元素按与枢轴的关系分区，再处理两侧。分区越均衡递归越浅，平均可达 $O(n \log n)$ ；若反复出现极不平衡分区，时间会退化为 $O(n^2)$ 。

回答要点

1. 先选定一种分区定义，例如枢轴最终归位，或双指针返回切分边界；递归范围必须与该方案一致。不能将 Hoare 分区返回值当作另一种方案中的最终枢轴位置，容易重复或漏掉元素。
2. 每次选最左元素会在特定有序输入上得到极不平衡分区。随机枢轴或先打乱输入可改善期望表现，但不提供普通随机快排的确切最坏上界；三数取中同样不能根除全部坏输入。
3. 大量重复值适合三路分区，将小于、等于和大于枢轴的区域分开，减少对相等元素的重复递归。小区间可用插入排序；需要确定上界时可用达到深度阈值后切换堆排的混合策略。

常见追问

追问：快排空间复杂度一定 $O(\log n)$ 吗？答：常规递归在均衡或平均情况下栈深对数，极端分区时可能线性。每次先递归较小分区、迭代处理较大分区，可限制递归栈深度，但这不自动消除总时间平方退化。

易错点

把随机化说成最坏 $O(n \log n)$ 是常见错误。还要测试重复值和边界相遇，保证每次分区后子问题严格缩小，避免死循环。

延伸阅读 / 原始参考资料

Princeton Algorithms • 快速排序

Open Data Structures • 分区与比较排序

08. 归并排序为何稳定？如何统计逆序对？

查找与排序 / 进阶

归并排序把问题分成两个子区间，分别排序后线性合并。合并相等键时先取左侧元素可保持稳定性；统计逆序对时，右元素小于当前左元素便与剩余左区间形成逆序。

回答要点

1. 两个已排序区间可以用双指针合并，每次取较小元素，任何一侧用尽后拷贝另一侧剩余部分。分治递推 $T(n)=2T(n/2)+O(n)$ ，得到最坏 $O(n \log n)$ ，不依赖初始是否有序。
2. 普通数组实现通常需要 $O(n)$ 合并缓冲区，递归还使用 $O(\log n)$ 栈。稳定性依赖相等元素的取舍规则，若相等时总先取右侧，就可能改变来自左右半区的原始相对次序。
3. 当右侧当前元素严格小于左侧当前元素时，它小于左半区剩余所有元素，可一次增加剩余数量。先递归统计两半内部逆序，再统计跨区间逆序，总计仍可在线性合并的基础上完成。

常见追问

追问：逆序对结果为什么要用宽整数？答： n 个元素最多有 $n(n-1)/2$ 个逆序对，规模稍大就可能超过 32 位整数。计算乘法时也要先转换到足够宽的类型，不能先在窄类型中溢出，再把错误结果转成宽类型。

易错点

逆序定义通常使用严格大于，相等值不应计数。稳定合并时使用左侧优先的相等规则，可同时让逆序统计条件清晰，避免把重复值算进答案。

延伸阅读 / 原始参考资料

Princeton Algorithms • 归并排序

Princeton 官方代码 • Merge

09. 二叉堆为何能 $O(n)$ 建堆？Top K 怎么做？

树与索引结构 / 基础

二叉堆以完全二叉树结构维持父子优先级关系，取堆顶 $O(1)$ ，插入和删除堆顶 $O(\log n)$ 。自底向上建堆为 $O(n)$ ，并不是对每个节点都按完整树高付费。

回答要点

1. 数组下标从零开始时，节点 i 的孩子为 $2i+1$ 和 $2i+2$ ，非根节点的父亲为 $(i-1)/2$ 。插入后上浮、移除堆顶后将末尾放到根再下沉，保持堆序及完全树的结构。
2. 从最后一个非叶节点向前下沉，绝大多数节点距离叶层很近。按不同高度累计节点数与下沉长度，总成本是线性的；若从空堆逐个插入所有元素，则一般上界为 $O(n \log n)$ 。
3. 求最大的 K 个数可维护大小 K 的小根堆，让当前最弱候选处于堆顶。新元素更大时替换堆顶并下沉，总时间 $O(n \log K)$ 、空间 $O(K)$ ；若需要有序输出，还需对结果再排序。

常见追问

追问：为什么不能在堆里 $O(\log n)$ 找任意值？答：堆序只约束父子，不保证整个数组有序，也不能根据一个节点值排除另一半区域。查任意值通常仍需 $O(n)$ ；若要高效修改指定元素，可额外维护键到位置的索引并同步交换。

易错点

“堆中元素有序”不准确，只有堆顶极值及父子关系得到保证。 K 为零、 K 大于输入量、重复值是否占名额和输出是否排序，均应先明确。

延伸阅读 / 原始参考资料

Open Data Structures • 二叉堆

Princeton Algorithms • 优先队列与堆

10. BST、AVL 与红黑树有什么区别？

树与索引结构 / 进阶

BST 依靠有序键关系缩小查找范围，但普通 BST 可能退化成链。AVL 约束子树高度差，红黑树通过颜色与黑高约束限制高度；二者都能提供对数级查找与更新上界。

回答要点

1. 普通 BST 的操作时间取决于树高 h ，顺序插入可能产生高度 n 的树。删除有两个孩子的节点时，可用中序后继或前驱替换，再处理较简单的实际删除位置，同时维护关联数据。
2. AVL 要求每个节点左右子树高度差至多一，通过单旋或双旋修复不平衡。旋转改变局部父子关系但保留中序键顺序；删除可能需要沿祖先路径继续修复，不能照搬插入停止条件。

3. 红黑树用红节点限制与相同黑高等规则保证高度为 $O(\log n)$ ，更新涉及旋转和重着色。AVL 更紧的高度控制与红黑树不同的修复策略形成取舍，不能不经测量断言某一种对所有负载更快。

常见追问

追问：有序 map 一定就是红黑树吗？答：红黑树是多种标准库的常见实现，但接口规范通常约束复杂度、顺序和迭代器保证，不强制内部结构。面试中应先讲数据结构原理，再明确具体库与实现层面的事实。

易错点

旋转不是重新排序所有节点；它应保留 BST 的中序顺序。重复键规则、父指针更新、根节点替换以及附加统计信息都需要在实现中一致维护。

延伸阅读 / 原始参考资料

Open Data Structures • 二叉搜索树

Open Data Structures • 红黑树

Yale 算法讲义 • Balanced Trees

11. 二叉树遍历与重建的条件是什么？

树与索引结构 / 基础

前序先访问根，中序在左右子树之间访问根，后序最后访问根；层序通过队列逐层处理。不同遍历揭示不同结构信息，重建是否唯一还依赖节点值唯一及给出的遍历组合。

回答要点

1. 前序适合先处理父节点，后序适合先汇总孩子结果，BST 的中序遍历可得到键的有序序列。迭代 DFS 用显式栈保存待处理状态，层序 BFS 用队列保存下一批节点，不能混淆二者访问顺序。
2. 无重复值且序列合法时，前序加中序或后序加中序可唯一重建。用中序值到位置的索引可减少反复查找；每次先确定根，再划分左右子区间，最后验证元素数量与对应位置。
3. 仅有前序和后序通常不能唯一确定普通二叉树，因为一个孩子究竟在左侧还是右侧可能无法辨别。重复值也会让中序根位置有歧义；题目若加满二叉树等限制，结论需要重新分析。

常见追问

追问：DFS 空间一定比 BFS 小吗？答：DFS 常需要 $O(h)$ 路径空间，BFS 常需要 $O(w)$ 层宽空间；链状树 h 可线性，宽树 w 也可线性。选择应结合树形和任务，而不是绝对说一种遍历更省空间。

易错点

区分完全二叉树、满二叉树与完美二叉树，不同教材中文译名可能有差异。重建时应直接说明每个节点孩子数量等结构条件，避免术语误解。

延伸阅读 / 原始参考资料

Open Data Structures • 二叉树遍历

12. Trie 如何支持前缀搜索？空间开销怎样优化？

树与索引结构 / 进阶

Trie 按字符或位逐层组织键，共享公共前缀。查找长度 L 的字符串通常需要 $O(L)$ 路径操作；它适合前缀查询和补全，但空间取决于字符集、节点表示与键的分布。

回答要点

1. 插入和查找逐字符沿边前进，完整键结束处要记录终止标志或值。一个字符串的路径存在不等于该字符串本身已经入库，例如存入 hello 后，hell 可能只是前缀而不是完整键。
2. 前缀检索先定位前缀节点，再遍历其子树收集结果。复杂度必须计入输出数量和输出字符串长度，不能只说前缀长度 $O(L)$ ，因为打印很多候选本身就需要相应时间和空间。
3. 每节点固定字符集数组可快速访问孩子，却浪费稀疏节点空间；映射或紧凑边列表可减少空间但影响访问成本。压缩 Trie 合并只有单个孩子的路径，须额外比较边上的字符片段。

常见追问

追问：Trie 一定比哈希表查字符串快吗？答：两者都可能需要读取字符，哈希表通常先算整个键哈希，Trie 可在不匹配处提前停止并支持前缀操作。实际速度还依赖节点分配、字符集大小、键分布和缓存局部性，应按需求选择。

易错点

必须明确字符编码，不能把 UTF-8 字节数量直接当成用户感知字符数量。固定 26 个孩子只适用于限定字母表，大小写、汉字与其他符号需要另行设计。

延伸阅读 / 原始参考资料

Princeton Algorithms • R-way Trie 官方实现

Princeton Algorithms • Trie 讲义

13. 图用邻接矩阵还是邻接表？BFS 与 DFS 如何选？

图算法 / 基础

邻接矩阵使用 $O(V^2)$ 空间并快速判断某条边，邻接表使用 $O(V+E)$ 空间更适合稀疏图。BFS 按边数逐层扩展，DFS 沿路径深入；遍历复杂度必须结合存储结构说明。

回答要点

1. 邻接矩阵查询某对顶点是否相连常为 $O(1)$ ，但枚举一个顶点的所有邻居通常需扫描 V 个位置。邻接表可直接枚举出边，但判断指定边存在可能需要遍历该顶点邻居或额外索引。

2. 在邻接表上，BFS 与 DFS 完整遍历一般为 $O(V+E)$ 。BFS 对无权图或等权图可找最少边数路径，发现节点时就标记可避免重复入队；DFS 常用于连通分量、环与递归结构分析。
3. 非连通图需对每个尚未访问节点启动遍历，单一起点只能覆盖可达部分。有向图与无向图的连通性、边回退和判环条件不同；无向 DFS 判断环时还要区分返回父边与真正回边。

常见追问

追问：BFS 找到的路径为什么最短？答：队列保证距离为 k 的节点先于距离为 $k+1$ 的节点处理，所以首次发现的边数最少。若边权不一致，这个按层结论不能直接代表总代价最小，应选择适合权重的算法。

易错点

不能把一个 `visited` 标志用于所有有向图判环场景，有向 DFS 还需区分正在递归路径上的节点与已经完成的节点。深图递归 DFS 也可能耗尽调用栈。

延伸阅读 / 原始参考资料

Open Data Structures • 邻接矩阵

Open Data Structures • 图表示与遍历

Princeton Algorithms • 无向图算法

14. 拓扑排序怎样检测有向环？顺序是否唯一？

图算法 / 进阶

拓扑排序将有向边的起点排在终点之前，存在完整拓扑序的充要条件是图为 DAG。Kahn 算法不断输出入度零顶点，若最终输出数小于顶点数，则剩余部分存在有向环。

回答要点

1. 先统计所有顶点入度，将入度为零的顶点入队；弹出一个顶点后遍历它的出边并减少终点入度，新的零入度顶点继续入队。邻接表实现总时间 $O(V+E)$ ，还需 $O(V)$ 辅助状态。
2. 另一种方法是 DFS 完成节点时记入后序，再将后序反转；必须同时检查递归路径中的回边来判环。对有环图仅输出反后序，并不能将它变成满足依赖约束的拓扑排序。
3. 拓扑序可能不唯一，若某一步同时有多个可选零入度顶点，就可能有不同选择。若要求字典序最小，可用小根堆替代普通队列，相应增加维护代价，并明确顶点比较规则。

常见追问

追问：任务依赖怎样建边才不会排反？答：若任务 B 必须等任务 A 完成，建 $A \rightarrow B$ ，B 的入度表示仍未完成的前置条件数量。拓扑排序输出是合法先后序，若还需要完成时间，应结合任务时长沿 DAG 做动态规划。

易错点

输出不到 V 个顶点时不能直接忽略剩余任务，也不能称已经得到完整执行顺序。还应检查自环、孤立顶点和重复边的入度计数是否与存储一致。

延伸阅读 / 原始参考资料

Princeton Algorithms • 拓扑顺序

Princeton Algorithms • DAG 与 Kahn 算法

15. BFS、Dijkstra、Bellman - Ford 与 Floyd 如何选择？

图算法 / 进阶

先看边权与查询范围：无权单源路径用 BFS，非负权常用 Dijkstra，含负边考虑 Bellman - Ford，小规模全点对可用 Floyd；负环会影响有限最短路。

回答要点

1. Dijkstra 每次确定当前暂定距离最小的顶点，并松弛其出边；非负权保证已经确定的距离不会被未来路径降低。采用索引二叉堆和邻接表的常见实现可达 $O((V+E) \log V)$ 时间。
2. Bellman - Ford 通过多轮松弛处理负边，常见最坏 $O(VE)$ ，并可识别源点可达负环。若负环还能到达目标，绕环可无限降低路径代价；与源点不可达的负环不影响该源点查询。
3. Floyd 枚举允许经过的中间顶点，依次更新 $dist[i][j]$ ，通常时间 $O(V^3)$ 、空间 $O(V^2)$ 。更新顺序要让中间顶点循环位于外层；不可达状态相加必须先检查，避免无穷哨兵加法溢出。

常见追问

追问：有负边但没有负环能用 Dijkstra 吗？答：一般不能保证正确，因为负边可能在某顶点已确定后带来更短路径。DAG 可按拓扑序进行线性松弛，即使存在负边也可处理；所以应先查看图结构，而不只根据是否负边机械选择。

易错点

SPFA 等队列优化不能默认承诺线性最坏时间。实现优先队列 Dijkstra 时，若插入重复距离项，还需跳过过期项，并根据具体实现重新说明时间与空间。

延伸阅读 / 原始参考资料

Princeton 讲义 • 单源最短路径

Princeton Algorithms • 负边与负环

16. 最小生成树与最短路不同在哪里？Prim 和 Kruskal 如何工作？

图算法 / 进阶

最小生成树在无向连通加权图中，用无环的 $V - 1$ 条边连接所有顶点并最小化总权重；最短路最小化特定起点到目标的路径代价。两种目标不同，算法结果不能互换。

回答要点

1. Kruskal 按边权从小到大考虑边，连接不同连通分量时才选入，通常借助并查集快速判环。排序常为 $O(E \log E)$ ，随后合并成本较低；图不连通时得到最小生成森林而非一棵生成树。
2. Prim 从一个顶点出发，反复选取当前树到外部的最小权重边来扩张。邻接表加适当二叉堆的常见版本为 $O((V+E) \log V)$ ，稠密图上矩阵版本 $O(V^2)$ 也可能更合适。
3. 割性质支撑贪心正确性：跨越某个割的最小边可成为某棵 MST 的安全选择。负权边不会像最短路负环那样破坏 MST；相同权重可能产生多棵不同的最优树，不能据此说无解。

常见追问

追问：最小生成树一定包含起点到每个节点的最短路径吗？答：不一定，它最小化整个连接网络的总权重，可能让某个点对的树内路径绕远。若需要源点最短路径树，应使用相应最短路算法；若追求建设总代价才考虑 MST。

易错点

不要给有向图直接套普通 Prim 或 Kruskal 求同一意义的 MST。有向的最小树形图是不同问题；边数 $V-1$ 也只是树结构条件，不能单靠条数证明连接且无环。

延伸阅读 / 原始参考资料

Washington CSE373 • 最小生成树讲义

University of São Paulo • Kruskal 官方代码文档

17. 并查集的路径压缩与按秩合并为什么有效？

图算法 / 进阶

并查集维护元素所属集合，find 找代表元，union 合并两个集合。路径压缩减少未来查询路径，按秩或按大小合并避免树过高；联合使用时，一串操作可获得接近常数的摊还成本。

回答要点

1. 每个集合用一棵父指针树表示，根节点作为代表元。判断两个元素是否同属集合，应比较 find 的结果，不能直接比较它们父指针；同一集合中不同节点的直接父亲可能不同。
2. 路径压缩在查询时让沿途节点更接近根，按大小合并把较小树接到较大树上。按秩合并则用秩指导方向，路径压缩后秩不再等于当前真实高度，但仍可用于正确的合并策略。
3. 两种优化联合时常见摊还界为 $O(\alpha(n))$ ， α 是增长极慢的反 Ackermann 函数；这不是每次严格 $O(1)$ 的证明。初始化父数组为 $O(n)$ ，统计集合数时只在成功合并不同根后递减。

```
#include <vector>
int find(std::vector<int>& parent, int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]];
    }
}
```

```
    x = parent[x];  
}  
return x;  
}
```

常见追问

追问：并查集能支持删除边吗？答：普通并查集擅长合并与连通查询，不擅长把集合拆开。离线动态连通可考虑倒序操作或回滚并查集；回滚版本通常避免普通路径压缩，以便记录有限的结构改变并恢复。

易错点

union 必须先找到根再修改根关系，直接改任意节点父亲可能破坏集合结构。不要把并查集当作保存原图所有路径信息的结构，它只维护特定连通关系。

延伸阅读 / 原始参考资料

Princeton Algorithms • 并查集复杂度

Princeton Algorithms • 路径压缩与合并

18. 动态规划的状态、转移、边界与遍历顺序怎么设计？

常见算法模式 / 进阶

动态规划复用重叠子问题的答案，核心是定义足够表达后续决策的状态，再建立无环依赖关系。依次说明状态含义、转移、边界、计算顺序、结果位置和复杂度，比先背代码更重要。

回答要点

1. 状态必须用完整句子解释，例如 $dp[i][j]$ 表示两个前缀的最长公共子序列长度。若当前决策还依赖前一个选择、剩余资源或是否持有物品，就需要把相应信息纳入状态，避免不同历史被错误合并。
2. 转移要覆盖所有合法决策，并证明子问题已经包含求解所需信息。最优化问题常需要最优子结构，计数问题则要避免重复计数；记忆化递归与自底向上是处理同一依赖关系的不同计算方式。
3. 复杂度通常等于状态数量乘每个状态的转移成本。只依赖上一行时可用滚动数组压缩空间，但更新顺序必须保持读取的是旧状态；需要还原实际方案时，还要保留选择或前驱信息。

常见追问

追问：0/1 背包一维数组为什么倒序遍历容量？答：每件物品只能使用一次，倒序让 $dp[j-weight]$ 仍来自处理当前物品前的状态。正序更新会读到本轮已更新的状态，可能重复使用同一件物品，更接近完全背包的语义。

易错点

看起来有递推式并不保证状态正确。先验证信息是否足够，以及转移是否重复使用当前对象；空间压缩后测试结果长度正确，也不代表方案还原仍可用。

延伸阅读 / 原始参考资料

19. 单调栈、双指针与滑动窗口为什么能做到线性？

常见算法模式 / 进阶

这些模式通过单向推进或淘汰不会再成为答案的候选，避免重复扫描。单调栈常找相邻更大更小元素，滑动窗口维护连续区间；线性时间需要每个元素进出次数有界及更新成本受控。

回答要点

1. 单调栈维持候选值的单调关系，新元素到来时弹出已经被它解决或支配的候选。寻找下一个更大元素可存索引，遇到更大值便为被弹出的索引确定答案；每个索引最多入栈出栈一次。
2. 滑动窗口常维护区间和、频率或最大值，右边界扩张、左边界按条件收缩。两指针总步数可线性，但前提是不会回退并且状态维护足够高效；条件若缺乏单调性，就不能随意套用。
3. 窗口最大值可用双端队列保留按值递减的候选索引：队首移除过期项，队尾移除被新值支配的项。新值更大且更晚过期，被淘汰的旧值不可能再次成为未来窗口的必要最大候选。

常见追问

追问：求和至少为目标的最短子数组能总用普通滑动窗口吗？答：全部元素非负时，扩张不会减小和，收缩不会增大和，容易保持单调判断。若含负数，这个性质失效，可能需要前缀和结合其他结构，例如单调队列。

易错点

重复值的弹出条件用 $<$ 还是 $\leq$ 会改变保留哪个索引，应按答案要求和过期规则选择。 $O(n)$ 推理要统计整个过程的进出次数，不能只看代码里是否有 `while` 嵌套。

延伸阅读 / 原始参考资料

Cornell CS2110 • 单调栈

Python 官方文档 • deque 双端操作

20. 怎样设计平均 $O(1)$ 的 LRU 缓存？

常见算法模式 / 进阶

LRU 淘汰最久未被使用的项，常用哈希表定位节点、双向链表维护访问次序。命中或更新后移到最近使用端，容量超限从另一端淘汰；平均常数时间依赖哈希表操作的平均复杂度。

回答要点

1. 哈希表保存键到链表节点或迭代器的映射，双向链表从最近访问到最久访问排列。查询命中后把节点摘下并移到头部，指针调整是常数操作；若只用链表，按键定位仍需要线性扫描。

2. 插入新键时先创建节点并更新索引，超过容量就删除尾节点，同时从哈希表移除相同键。更新已有键应改值并移动，而不是生成第二个同键节点；哨兵节点可减少头尾特殊分支。
3. 容量为零、返回值与缺失值冲突、节点所有权和异常路径都要明确。并发访问中命中也会修改访问链，因此所谓只读查询同样可能需要锁；大规模缓存可能采用近似策略减少维护竞争。

常见追问

追问：LRU 和 LFU 有什么区别？答：LRU 看最近访问时间，LFU 看访问频率，两者对一次性扫描与热点的反应不同。LRU 也不自动提供过期时间或内存字节上限，需要将 TTL、容量口径和淘汰策略作为不同需求设计。

易错点

不能只从链表删节点而遗漏哈希索引，否则后续查询会拿到失效地址。严格 $O(1)$ 要避免误写，应说明哈希部分是平均时间，碰撞和重哈希仍可能增加单次成本。

延伸阅读 / 原始参考资料

Oracle Java SE 21 • [LinkedHashMap 访问顺序与 LRU](#)

CMU 论文 • [基于双向链表的 LRU 维护](#)

操作系统

从进程到持久化

以通用操作系统原理为主，结合 Linux/POSIX 接口讨论进程、内存、并发与 I/O；具体调度器和内核细节需注明版本。

01. 进程和线程有什么区别？

进程与调度 / 基础

进程组织地址空间和资源，线程组织执行流。同一进程的线程通常共享代码、堆和文件描述符，但各自保留寄存器、栈及调度状态；共享使通信方便，也引入同步问题。

回答要点

1. 进程间默认有地址空间隔离，某个进程的普通内存写入不会直接覆盖另一个进程；线程共享地址空间，因此一个线程的非法写入可能破坏整个进程的运行。
2. 每个线程都要保存程序计数器、寄存器和自己的栈，局部变量并非自动可被其他线程安全访问；通过指针共享栈对象还必须保证对象生命周期覆盖访问过程。
3. 比较创建与切换成本时应说明系统实现：同进程线程切换常可保留地址空间，跨进程可能切换页表并影响 TLB；缓存冷却和调度策略也会改变实际成本。

常见追问

追问：多线程一定比多进程快吗？答：不一定。线程减少部分资源开销，但共享数据会带来锁竞争和缓存争用；多进程隔离更强。应以任务粒度、通信量、故障隔离需求和实测结果选择。

易错点

易错点：把线程等同于轻量进程后就认为它没有独立栈，或认为线程切换完全不进入内核。用户态协程和内核线程的调度机制需要分别讨论。

延伸阅读 / 原始参考资料

OSTEP • Concurrency and Threads

Linux 手册 • pthreads(7)

02. 进程状态和上下文切换如何理解？

进程与调度 / 基础

经典模型包括运行、就绪和阻塞。就绪表示具备运行条件但暂未获 CPU，阻塞表示等待事件。切换保存旧任务的执行现场并恢复新任务，事件完成通常先转为就绪。

回答要点

1. 时间片耗尽或更高优先级任务到达，可使运行任务回到就绪；等待 I/O、锁或条件的任务进入阻塞，不能只靠给它更多时间片让等待条件立即满足。
2. 上下文通常涉及寄存器、程序计数器、栈指针及内核维护的调度信息；跨地址空间切换还可能改变页表根，间接造成 TLB 和 CPU 缓存命中率变化。
3. 状态图是教学抽象，Linux 等系统还区分可中断睡眠、不可中断睡眠、停止与僵尸等状态；回答时先说明通用模型，再解释所讨论系统的扩展状态。

常见追问

追问：系统调用必然发生进程切换吗？答：不会。系统调用首先是同一任务从用户态进入内核态，处理完成后可能直接返回；只有阻塞、抢占或主动让出等条件触发调度时才切换任务。

易错点

易错点：混淆模式切换和上下文切换；I/O 完成也不意味着该任务马上占用 CPU，它通常需要进入就绪队列等待调度器选择。

延伸阅读 / 原始参考资料

OSTEP • The Process

OSTEP • Limited Direct Execution

03. 常见 CPU 调度算法怎么比较？

进程与调度 / 基础

先来先服务简单但可能出现护航效应；短作业优先在已知运行时间的模型下改善平均周转；时间片轮转改善响应。实际调度需权衡公平、吞吐、响应和饥饿。

回答要点

1. FCFS 可让短任务被长任务阻挡；SJF 需要预测任务长度，抢占式版本按剩余时间选择任务。说明理论最优时必须同时给出到达时间和长度已知等模型假设。
2. 轮转使用时间片让就绪任务交替执行，较小时片有利于响应但增加切换成本；较大时间片逐渐接近 FCFS。时间片并非越小越好，应联系切换开销和交互需求。
3. 多级反馈队列根据历史行为调整优先级，周期性提升可缓解饥饿；真实 Linux 调度器随版本演进，不能把教学算法直接宣称为所有版本的实际实现。

常见追问

追问：周转时间和响应时间有什么区别？答：周转从到达到完成，响应从到达到首次获得响应；一个算法可能让首次响应很快，却因反复切换拉长总完成时间。面试时应先明确优化目标。

易错点

易错点：把高优先级等同于短作业，或声称短作业优先在任何实际系统中都最优。未知任务长度、公平性与实时约束会改变算法选择。

延伸阅读 / 原始参考资料

OSTEP • Scheduling

OSTEP • Multi-Level Feedback Queue

04. fork、exec、wait 分别做什么？

进程与调度 / 基础

POSIX 的 `fork` 创建子进程，`exec` 系列替换当前进程的程序映像，`wait` 系列获取子进程退出状态并回收相应记录。`fork` 后父子返回值不同，执行先后由调度决定。

回答要点

1. `fork` 后父进程得到子进程 PID，子进程得到零，失败则返回错误；父子拥有独立的逻辑地址空间。Linux 常用写时复制推迟数据页复制，页表等仍有管理开销。
2. `exec` 成功后不返回原调用点，而是运行新程序；PID 通常保持不变。文件描述符是否保留受 `close-on-exec` 标志影响，不能说 `exec` 无条件清空全部进程资源。
3. 子进程退出后保留状态供父进程读取，未回收会形成僵尸；孤儿指父进程先退出的存活子进程。两者描述不同维度，回收机制需结合系统和子进程管理方式。

常见追问

追问：为什么 shell 常先 `fork` 再 `exec`？答：shell 需要保留自身的命令解释循环，同时让子进程执行目标程序；还可以在子进程执行前设置重定向等环境，父进程按前台或后台语义决定是否等待。

易错点

易错点：`exec` 创建新进程、僵尸还在占用 CPU、父进程退出后子进程必然立即退出，这三个说法都不准确。应区分程序替换、退出状态保存与进程收养。

延伸阅读 / 原始参考资料

OSTEP • Process API

Linux 手册 • `fork(2)`

Linux 手册 • `execve(2)`

05. 常见 IPC 方式如何选择？

进程与调度 / 基础

IPC 可通过管道、消息队列、共享内存、信号和套接字完成。选择时关注是否跨机器、消息边界、数据规模和同步需求；共享内存省去部分复制，却需要自行约束并发访问。

回答要点

1. 管道适合字节流，普通匿名管道常用于有亲缘关系的进程；FIFO 可通过路径让无亲缘进程通信。读写端关闭、阻塞规则和缓冲容量都应纳入协议设计。
2. 共享内存让多个地址空间映射同一存储区域，适合大数据量交换；数据布局、初始化、对象生命周期和同步必须有约定，映射共享区域并不会自动提供互斥。
3. 消息队列强调离散消息，套接字可实现本机或跨网络通信，信号更适合有限事件通知；不同机制支持的边界、持久性和权限模型不同，应按问题选择。

常见追问

追问：共享内存为什么还要加锁？答：多个进程可能同时修改同一队列索引或读取尚未发布的数据；共享只解决可访问性，锁、信号量或正确设计的原子协议才负责互斥、顺序和可见性。

易错点

易错点：认为共享内存绝对没有任何复制开销，或把所有管道写入都当作原子消息。管道是字节流，写入原子性还受到 PIPE_BUF 等条件约束。

延伸阅读 / 原始参考资料

Linux 手册 • pipe(7)

Linux 手册 • shm_overview(7)

Linux 手册 • unix(7)

06. 虚拟内存解决了什么问题？

内存管理 / 基础

虚拟内存为每个进程提供逻辑地址空间，通过地址转换和权限检查映射到物理资源。它支持隔离、共享、按需分配和换出；核心并非单纯把磁盘当作更大的内存。

回答要点

1. 程序使用虚拟地址，硬件与内核依据页表完成转换并检查读写执行权限；不同进程可以使用相同虚拟地址映射到不同物理页，简化程序装载并实现隔离。
2. 代码、堆、栈和映射文件组成地址空间，各区域的权限与生命周期不同；虚拟区域被预留或映射，并不保证每一页立即分配物理内存，更不保证未来分配必成功。
3. 共享库、共享内存和写时复制可让多个虚拟映射引用同一物理页；是否能写、何时复制和何时回收由映射属性决定，隔离与共享可以同时存在于一个系统。

常见追问

追问：64 位程序就能使用全部 2 的 64 次方字节吗？答：不意味着如此。CPU 实际实现的虚拟地址位宽、操作系统布局、进程限制和可用物理资源都会限制地址范围及可成功分配的容量。

易错点

易错点：把虚拟内存大小、驻留集大小和实际物理占用混为一谈。讨论内存不足时应同时检查映射、驻留页、共享页和系统提交策略。

延伸阅读 / 原始参考资料

OSTEP • Address Spaces

OSTEP • Address Translation

07. 分页、分段和多级页表有什么区别？

内存管理 / 基础

分页使用固定大小页和页框，分段依据代码、数据等逻辑区域划分可变大小段。多级页表把页表自身按层组织，稀疏地址空间可只建立需要的下级表，但增加查表层次。

回答要点

1. 分页将地址拆成虚拟页号与页内偏移，转换只替换页号，偏移保持不变；固定页框减少传统外部碎片，但末页未用空间仍形成内部碎片。
2. 分段为不同逻辑区域配置基址、界限和权限，表达逻辑结构直观；可变大小区域的分配容易出现外部碎片。现代体系也可能结合分段和分页，不能只用二选一模型。
3. 多级页表按地址位段逐层索引，未使用的大地址区域无需分配完整叶表；代价是未命中 TLB 时的页表遍历。大页可减少层级或条目，但会带来分配与碎片权衡。

常见追问

追问：4 KiB 页的偏移需要多少位？答：需要十二位，因为 4096 等于 2 的 12 次方；地址其余有效位用于页号和页表层级索引。具体各级索引位数由所讨论的体系结构和页表格式决定。

易错点

易错点：认为多级页表一定比单级页表占用更少内存。它主要适合稀疏地址空间；若地址空间密集使用，上层索引结构也需要额外空间。

延伸阅读 / 原始参考资料

OSTEP • Paging

OSTEP • Advanced Page Tables

OSTEP • Segmentation

08. TLB 是什么？TLB miss 等于缺页吗？

内存管理 / 基础

TLB 缓存最近使用的地址转换及相关权限，减少查页表成本。TLB miss 表示缓存没有该转换；页表仍可能指向驻留物理页。缺页是地址访问触发的异常，两者不能混同。

回答要点

1. TLB 命中时可快速得到物理页号；未命中时由硬件或软件遍历页表补充转换，机制依体系结构而异。页表遍历本身可能访问缓存和内存，因此会引入延迟。
2. 进程切换后旧转换不能误用于新地址空间，可通过刷新或地址空间标识区分；映射被修改时还必须使相关旧转换失效，多核系统需要协调相关 CPU 的 TLB。
3. TLB 容量有限，跨大量页面的随机访问可能使转换频繁失效；大页提高一次转换覆盖的范围，但增加内存分配难度，并可能浪费较小对象周围的空间。

常见追问

追问：数据全部在 CPU cache 中，为什么访问仍然慢？答：数据缓存命中不代表地址转换缓存命中。如果访问跨越很多页，TLB miss 及页表遍历仍可成为开销，应结合数据访问和地址转换的指标分析。

易错点

易错点：把 TLB 说成保存完整数据页的缓存，或者把任何 TLB miss 都解释成从磁盘读页。它主要缓存地址转换，实际缺页还要检查页表状态。

延伸阅读 / 原始参考资料

OSTEP • Translation Lookaside Buffers

09. 缺页异常如何处理？什么是写时复制？

内存管理 / 进阶

缺页处理先判断地址和权限是否有效，再决定分配零页、加载文件或换入页面；无效访问可能被终止。写时复制让页面先共享，发生写入后才为写入方建立私有副本。

回答要点

1. 合法地址未驻留时，内核可以分配物理页并填充数据，更新页表后让指令重试；读取磁盘只是其中一种路径，首次访问零填充页也可能发生缺页而无需磁盘读取。
2. 权限异常可能来自真正非法写入，也可能是写时复制的保护安排；内核需要查看映射属性，不能把所有保护异常都判定为程序错误，也不能无条件允许写入。

3. fork 后共享的可写数据页常暂时标为只读，并记录写时复制关系；任一进程写入时建立私有页，其他进程保持原内容。引用计数和并发处理决定页面何时可回收。

常见追问

追问：minor fault 和 major fault 怎么区分？答：Linux 统计中主要看处理该缺页是否需要相关存储 I/O；轻微缺页也可能涉及分配、页表更新和复制，因此数量高仍会耗费 CPU，不能认为完全免费。

易错点

易错点：malloc 成功不等于所有页面已经驻留；缺页不等于非法访问。应区分地址有效性、页面驻留状态、权限保护和具体后备存储。

延伸阅读 / 原始参考资料

OSTEP • Swapping Mechanisms

Linux 手册 • getrusage(2)

Linux 手册 • mmap(2)

10. 页面置换算法和抖动是什么？

内存管理 / 进阶

页面置换在物理内存紧张时选择牺牲页，目标是减少未来缺页。最优算法需要未来访问序列，LRU 用过去近似未来，Clock 用访问位降低维护成本；频繁换入换出称为抖动。

回答要点

1. FIFO 按进入时间淘汰，未必照顾热点，可能出现分配更多页框却缺页更多的 Belady 异常；真正的 LRU 属于栈算法，但精确记录每次访存顺序的成本较高。
2. Clock 环形扫描页面并参考访问位，给近期使用的页第二次机会；实际系统还区分匿名页、文件页、脏页以及回收成本，不能用单一教学算法概括全部实现。
3. 当活跃工作集超过可提供的内存时，刚换入的页可能很快再次被淘汰，I/O 时间占比升高；降低并发、优化局部性或增加资源可能比只调整置换算法有效。

常见追问

追问：缓存命中率高为什么仍会卡顿？答：缺页的代价差别很大，少量需要存储 I/O 的缺页可能主导尾延迟；还要观察换页频率、工作集、存储延迟和内存压力，不能只看平均命中率。

易错点

易错点：把操作系统实际回收策略直接说成精确 LRU，或认为加内存一定按线性比例提速。置换效果取决于访问模式、工作集与页面后备类型。

延伸阅读 / 原始参考资料

OSTEP • Swapping Policies

11. 互斥锁、自旋锁、读写锁怎么选？

并发与同步 / 基础

互斥锁保护临界区，竞争时通常可以阻塞等待；自旋锁在等待时持续检查；读写锁允许多个读者但限制写者。选择取决于持锁时间、竞争程度、运行环境和公平性要求。

回答要点

1. 自旋适合预期很短且持锁者能尽快继续运行的等待；长时间持锁、单核竞争或持锁者被抢占时，自旋浪费 CPU。内核自旋锁还受到是否允许睡眠等上下文限制。
2. 阻塞式互斥锁可节省等待 CPU，但睡眠与唤醒产生调度成本；实际锁实现常结合用户态快速路径、短暂自旋和内核等待，不能把每次加锁都说成一次系统调用。
3. 读写锁在大量并发读取且写入较少时可能有效，但维护读者状态也有开销；写者饥饿、锁升级和长读临界区都需分析，读多并不自动意味着读写锁更快。

常见追问

追问：临界区只有一个自增为什么还要同步？答：普通自增通常包含读、修改和写，多线程交错会丢失更新。可以使用锁或适当原子操作；正确的选择还要考虑被保护的不变量是否涉及多个变量。

易错点

易错点：自旋锁不是忙等越久性能越好；读写锁也不是万能的读多优化。首先缩短临界区、减少共享，再基于测量选择等待方式和锁粒度。

延伸阅读 / 原始参考资料

OSTEP • Locks

Linux 手册 • pthread_mutex_lock(3)

12. 信号量和互斥锁、条件变量的关系？

并发与同步 / 基础

信号量维护许可数量，等待操作获取许可，发布操作归还或增加许可。互斥锁表达临界区所有权，条件变量表达等待某个受锁保护的条件；三者语义不同，不能随意替换。

回答要点

1. 计数信号量可限制同时访问资源的任务数，初值对应许可数量；生产者消费者中可分别计数空位和已有元素，还需要互斥保护共享队列的索引与存储。
2. 二值信号量可以实现部分互斥效果，但通常没有互斥锁的同等所有权要求；互斥锁强调由持有者解锁，具体递归、错误检查和优先级继承能力依 API 而异。

3. 条件变量自身不保存资源计数，唤醒后还要重新检查谓词；信号量的许可可在没有等待者时累积。把条件通知当成可永久保存的事件，会导致丢失等待协议。

常见追问

追问：限制同时最多十个下载任务用什么？答：可用初值十的计数信号量，任务开始前获取许可，成功、失败或取消路径都归还许可；资源管理应保证异常情况下不泄漏许可，避免并发额度逐渐耗尽。

易错点

易错点：只设置一个表示队列非空的信号量，却没有处理容量和共享索引并发。计数、互斥和条件等待是不同职责，要按不变量组合。

延伸阅读 / 原始参考资料

OSTEP • Semaphores

Linux 手册 • sem_overview(7)

13. 为什么条件变量要在 while 中等待？

并发与同步 / 基础

条件变量等待必须与受互斥锁保护的谓词配合：持锁检查，条件不满足就等待，醒来后重新检查。while 处理虚假唤醒和其他线程抢先消耗条件，通知只表示可以重试。

回答要点

1. 等待操作需要原子地释放互斥锁并进入等待，再在返回前重新获得锁；这避免检查条件后、真正睡眠前发生通知而丢失唤醒。手动先解锁再睡眠无法保证同等协议。
2. 一次通知不意味着当前线程一定获得资源，其他消费者可能先拿走元素；广播会唤醒多个等待者，而可用资源可能有限。醒来必须重新确认谓词而不是直接继续。
3. 改变谓词的一方也应按约定持有同一锁，先更新共享状态再通知；多个不同等待条件可以使用多个条件变量，但仍要设计对应锁和谓词，避免无效唤醒和遗漏通知。

常见追问

追问：为什么用 if 会出错？答：消费者收到通知后还需竞争锁，在它获得锁前队列可能再次为空；if 只检查了睡眠前的状态。while 重检能让它安全继续等待，虚假唤醒同样适用这个理由。

易错点

易错点：认为通知次数就是资源数量，或者认为被通知线程直接获得执行权。通知、调度和获得互斥锁是不同步骤，程序正确性应建立在谓词上。

延伸阅读 / 原始参考资料

OSTEP • Condition Variables

Linux 手册 • pthread_cond_wait(3)

14. 死锁的四个必要条件和处理方式？

并发与同步 / 进阶

经典死锁需要互斥、占有并等待、不可抢占和循环等待同时成立。可以通过设计破坏条件、分配前避免不安全状态，或运行时检测后恢复；忙碌但无进展的活锁需另行区分。

回答要点

1. 统一锁顺序可破坏循环等待，例如所有路径先拿 A 再拿 B；一次申请全部资源可减少占有并等待，但限制并发。措施必须覆盖全部访问路径，包括错误恢复和回调。
2. 银行家算法根据最大需求判断状态是否安全，适用于需求可预先描述的资源模型；一般应用的锁依赖未必满足这个前提，不能把它作为无需约束的通用解决方案。
3. 检测等待图后可以终止任务、回滚事务或释放可恢复资源，但恢复动作有业务成本；超时能打断部分等待，也可能因频繁重试造成活锁，应设置退避并保证回滚完整。

常见追问

追问：一次观察到循环等待就一定是死锁吗？答：单实例资源且边代表确定等待时，等待图环是有力依据；多实例资源图的环不必然构成死锁。应说明图中节点、边和资源数量的模型。

易错点

易错点：把四个必要条件当成任意一个即可证明死锁，或认为加超时自动保证业务一致性。超时后的资源释放与已发生的操作仍需要正确处理。

延伸阅读 / 原始参考资料

OSTEP • Concurrency Bugs

15. 阻塞、非阻塞、同步和异步 I/O 怎么区分？

I/O 与文件系统 / 基础

阻塞描述调用是否等待才能返回，异步描述操作完成如何通知调用方，两者不是同一维度。非阻塞 read 无数据时可立即返回 EAGAIN；完成通知型异步 I/O 则在操作完成后报告结果。

回答要点

1. 阻塞 I/O 可让调用线程休眠直到数据或结果可用；非阻塞 I/O 要处理暂时不可操作、部分读写和重试。反复忙轮询虽然不阻塞调用，却可能消耗大量 CPU。
2. select、poll、epoll 主要通知文件描述符是否适合执行 I/O，通常随后还需 read 或 write；就绪通知与已完成读写不同，因此不能直接把 epoll 等同于完成通知异步 I/O。

3. 事件循环可以用少量线程管理大量等待中的连接，但 CPU 密集处理和阻塞文件操作仍可能阻塞整个循环；可把这类工作放到工作线程，并设计结果回传和背压。

常见追问

追问：epoll_wait 能阻塞，为什么还叫事件驱动？答：事件循环可以在没有就绪事件时休眠，有事件后统一处理；事件驱动指控制流按通知推进，并不要求每个调用都立即返回，也不代表完全无需线程。

易错点

易错点：把非阻塞等同于异步，把同步等同于始终阻塞。应先说明通知的是就绪还是完成，以及哪个阶段会等待、由谁执行数据传输。

延伸阅读 / 原始参考资料

OSTEP • Event-based Concurrency

Linux 手册 • aio(7)

Linux 手册 • read(2)

16. epoll 的 LT 与 ET 有什么区别？

I/O 与文件系统 / 进阶

Linux epoll 用兴趣集合和就绪集合管理事件。LT 在条件仍就绪时可再次通知，ET 主要在就绪变化时通知；ET 通常配合非阻塞描述符，处理到 EAGAIN 再等待。

回答要点

1. 如果一次事件到来后只读取部分数据，LT 可以继续报告可读，ET 却可能没有新变化可报告，导致剩余数据滞留；ET 处理器应循环读取，同时正确处理断开、错误和中断。
2. 非阻塞套接字的写入可能只接受部分数据，剩余内容要放进应用缓冲区；需要写事件时再关注可写就绪，避免常驻可写通知产生忙循环，并限制缓冲区实施背压。
3. epoll_wait 返回的事件数量影响单次处理工作量，注册、唤醒和应用处理也有开销；不能因为内部维护就绪集合，就声称所有操作、所有负载下都严格为 O(1)。

常见追问

追问：ET 会不会丢事件？答：常见问题是应用没有消耗到 EAGAIN 就再次等待，或者错误管理状态与重新武装；事件通知不能被当作字节计数。可使用明确的连接状态机，保证每次处理都能继续推进。

易错点

易错点：把 ET 描述为每个数据包恰好通知一次。通知可合并，且网络数据包与套接字可读字节流不是一一对应关系；应按就绪条件理解。

延伸阅读 / 原始参考资料

Linux 手册 • epoll(7)

Linux 手册 • [epoll_ctl\(2\)](#)

Linux 手册 • [epoll_wait\(2\)](#)

17. 文件描述符、打开文件描述和 inode 的关系？

I/O 与文件系统 / 基础

文件描述符是进程用于引用打开对象的整数；内核打开文件描述保存偏移与状态，inode 保存文件元数据。
dup 或 fork 可让多个描述符共享同一打开文件描述，因此也共享文件偏移。

回答要点

1. 描述符表属于进程资源视图，描述符数字可以在关闭后复用，不能把数字当作永久对象身份；读写套接字、管道和普通文件都可能通过文件描述符接口进行。
2. 同一文件被分别 open 两次，通常得到不同打开文件描述和独立偏移；dup 得到的新描述符引用同一描述，因此一个描述符读取后另一个会观察到推进的偏移。
3. inode 保存类型、权限、大小及数据定位等信息，目录负责把名字关联到 inode；文件名并不是 inode 必须保存的字段，多个硬链接可对应同一 inode。

常见追问

追问：fork 后父子都读同一个继承描述符会怎样？答：它们共享打开文件描述，因而共享偏移，读取会消耗同一个顺序位置；独立 open 或使用显式偏移的 pread 可以表达不同读取需求，并仍需考虑共享数据的变化。

易错点

易错点：把描述符表、系统打开文件描述和 inode 说成同一层，或认为 dup 会复制文件内容。它主要建立另一个引用，不创建独立文件副本。

延伸阅读 / 原始参考资料

OSTEP • [Files and Directories](#)

Linux 手册 • [open\(2\)](#)

Linux 手册 • [dup\(2\)](#)

18. inode、目录和软硬链接怎么理解？

I/O 与文件系统 / 基础

目录项把名字映射到文件标识，inode 描述文件自身。硬链接新增同一文件的名字，符号链接是保存路径的独立对象；删除一个名字不一定立刻释放文件内容。

回答要点

1. 硬链接指向同一 inode，多个名字读取的是同一份文件数据；常规文件系统通常不允许跨文件系统硬链接，并限制对目录创建硬链接，避免层次结构出现复杂循环。
2. 符号链接保存目标路径，目标可以不存在，也可跨文件系统；相对目标通常相对于符号链接所在目录解析。目标被移动或删除时可能形成悬空链接。
3. unlink 移除目录项并减少链接计数，文件还可能被打开文件描述引用；通常在链接计数归零且没有仍需保持对象的引用时，系统才最终回收相应数据与元数据。

常见追问

追问：删除正在写入的日志后磁盘空间为何没释放？答：进程可能仍持有已经没有目录名字的文件对象，继续通过描述符写入。关闭相关引用后空间才有机会回收，排查时要确认被删除文件的打开引用。

易错点

易错点：把符号链接说成另一个共享 inode 的文件名，或认为 unlink 必然立即销毁文件。目录项、文件对象和进程引用有各自的生命周期。

延伸阅读 / 原始参考资料

OSTEP • File System Implementation

Linux 手册 • link(2)

Linux 手册 • symlink(2)

Linux 手册 • unlink(2)

19. 文件系统日志为什么能帮助崩溃恢复？

I/O 与文件系统 / 进阶

文件系统更新可能涉及多个数据块与元数据块，崩溃会留下部分完成的状态。日志先记录事务意图或更新内容，再按规则提交和应用，使恢复能够重放或丢弃未完成事务。

回答要点

1. 例如扩展文件需要修改数据块、分配位图和 inode，任意写入顺序都可能在崩溃后产生不一致；日志把关联修改组织成事务，并通过提交标记判断是否应恢复。
2. 元数据日志与数据日志提供不同保证，不能认为所有日志文件系统都保护所有用户数据；写回缓存、设备持久化语义和写入排序也会影响断电后的结果。
3. write 成功通常说明数据已被系统接收，并不等于落到持久介质；需要持久化时按接口语义使用 fsync 等操作，涉及创建或重命名时还需考虑目录项的持久化。

常见追问

追问：用了日志还需要 fsync 吗？答：需要依据持久化要求判断。日志主要解决文件系统一致性，不自动保证每次应用写入都在断电前持久化；业务必须明确提交边界，并正确处理同步操作返回的错误。

易错点

易错点：把日志文件系统一致性、应用事务原子性和数据永不丢失混为一谈。它们是不同的层级的保证，需要说明日志模式和实际持久化接口。

延伸阅读 / 原始参考资料

OSTEP • Crash Consistency

Linux 手册 • fsync(2)

20. 用户态、内核态、系统调用和中断的关系？

I/O 与文件系统 / 基础

用户态限制特权操作，内核态允许管理受保护资源。系统调用通过受控入口请求内核服务，中断响应外部事件，异常来自执行过程；进入内核后是否调度其他任务取决于处理结果。

回答要点

1. 普通程序不能任意更改页表或直接控制设备，保护边界由硬件权限与内核共同维护；系统调用入口校验参数和权限，再把允许的操作转换成内核内部处理。
2. 外部设备完成、定时器等通常通过中断通知 CPU；除零、非法指令和缺页属于异常。不同体系结构对 trap、fault 等术语的细分不同，回答时应注明所用分类。
3. 系统调用有进入退出、参数检查和可能的数据传输成本，但库函数不必每次都触发系统调用；例如缓冲 I/O 可能先在用户空间完成，之后批量提交给内核。

常见追问

追问：为什么不能让应用直接操作硬件以提高速度？答：缺少受控隔离会让一个错误程序破坏其他任务或系统资源；高性能接口可减少调用次数或共享缓冲区，但仍需要权限、生命周期和资源控制协议。

易错点

易错点：认为进入内核态必然切换进程，或每个 C 库函数都是系统调用。权限级别变化、内核服务入口和调度切换应分别描述。

延伸阅读 / 原始参考资料

OSTEP • Limited Direct Execution

OSTEP • I/O Devices

计算机网络

从可靠传输到 Web 协议

覆盖 TCP/IP 与 Web 通信的高频考点。TCP 以 RFC 9293 为基础，HTTP 使用 9110 系列，明确区分 TLS 1.3、HTTP/2 与 HTTP/3。

01. OSI 七层与 TCP/IP 模型怎么对应？

网络基础 / 基础

OSI 是分层参考模型，TCP/IP 是实际互联网协议体系。可把应用、表示、会话功能归入应用层，再分传输、网际和链路层；部分教材将链路拆成数据链路与物理两层。

回答要点

1. 应用层处理 HTTP、DNS 等业务语义，传输层通过 TCP 或 UDP 为进程通信提供端口和传输能力，IP 负责跨网络寻址与转发，链路层处理同一链路内的帧传递。
2. 发送侧逐层封装首部，接收侧反向解析；路由器主要依据网络层信息转发，交换机常依据链路地址转发。实际设备也可能实现更高层功能，不能仅凭设备名称限定层级。
3. 分层是一种功能组织方式，现实协议可能跨越传统边界，例如 QUIC 在 UDP 上实现可靠流和安全握手；描述它时应说明其作用，而不强行套入唯一的 OSI 层号。

常见追问

追问：端口、IP 和 MAC 分别解决什么问题？答：端口帮助区分主机上的传输端点，IP 用于网络间寻址与路由，MAC 用于特定链路内交付。跨路由时链路首部常改变，端到端地址也可能受 NAT 影响。

易错点

易错点：认为 OSI 七层就是互联网每个报文必须严格走过的七个独立模块，或把 HTTP 的所有功能固定解释成会话层。参考模型和协议实现应区分。

延伸阅读 / 原始参考资料

RFC 1122 • Internet Host Communication Layers

MDN • HTTP Overview

02. TCP 与 UDP 的区别和使用场景？

TCP 与 UDP / 基础

TCP 提供面向连接、有序可靠的字节流，并包含流量和拥塞控制；UDP 提供保留消息边界的数据报，不内置可靠有序传输。选择取决于应用是否需要时效、顺序与自定义传输策略。

回答要点

1. TCP 负责排序、丢失恢复和重复处理，但连接中断时仍可能出现结果不确定，不能替业务提供事务成功保证；应用还需约定消息格式、超时以及必要的幂等或去重机制。
2. UDP 每次发送对应数据报边界，但网络仍可能丢失、乱序、重复，较大报文还可能面临分片问题；应用应控制消息大小，并在需要时实现反馈、重试和拥塞响应。
3. 实时媒体可优先考虑过期数据无需重传的策略，可靠文件传输常选择 TCP；QUIC 则在 UDP 上实现可靠传输，说明 UDP 可作为更高层可靠协议的承载，而非只能用于不可靠业务。

常见追问

追问：UDP 为什么不保证比 TCP 快？答：它减少部分协议机制，但实际延迟取决于链路、拥塞、重试和应用处理。若应用自己补齐可靠性，也会产生开销；应比较符合业务语义的完整实现及测量结果。

易错点

易错点：TCP 可靠不等于业务绝不失败，UDP 无握手不等于应用没有会话，也不等于其上层协议不能实现可靠传输。可靠性保证必须说明所在层级。

延伸阅读 / 原始参考资料

RFC 768 • User Datagram Protocol

Linux 手册 • tcp(7)

RFC 9000 • QUIC Transport

03. TCP 为什么通常需要三次握手？

TCP 与 UDP / 基础

常规握手依次交换 SYN、SYN+ACK、ACK，使双方同步各自初始序列号并确认对方已收到自己的建连信息。第三步避免服务器仅因旧 SYN 就把连接视为已完全建立。

回答要点

1. 客户端发送 SYN 并携带初始序列号 x ，服务器用 SYN+ACK 确认 $x+1$ ，同时公布自己的序列号 y ；客户端再确认 $y+1$ 。SYN 消耗一个序列号空间，方便可靠确认。
2. 三次交互让双方分别获知对端的发送初始状态和确认信息；若只用两次，服务器可能无法确认客户端是否收到其 SYN，也难以正确区分某些旧连接请求带来的情况。
3. 常规流程不是所有连接建立情况的唯一报文排列，同时打开、TCP Fast Open 和重传会呈现不同交互；回答应先使用 RFC 9293 的基础模型，再说明扩展与异常路径。

$C \rightarrow S$: SYN, seq= x

$S \rightarrow C$: SYN+ACK, seq= y , ack= $x+1$

$C \rightarrow S$: ACK, ack= $y+1$

常见追问

追问：第三次 ACK 丢失怎么办？答：服务器仍可能处在等待确认状态并重传 SYN+ACK；客户端收到后再次确认。客户端后续携带正确确认号的数据也可能推进状态，但不能据此假设第三步从来不重要。

易错点

易错点：只背诵双方确认能收能发，却不解释序列号同步与旧报文；或者把三次握手说成必须恰好三个网络数据包，忽略重传和特殊建连流程。

延伸阅读 / 原始参考资料

RFC 9293 • TCP — Connection Establishment

04. TCP 关闭为什么常见四次挥手？

TCP 与 UDP / 基础

TCP 是全双工，两方向的数据流需要分别终止。某端发送 FIN 表示该方向不再发送，另一端确认后仍可继续发送，待其结束再发 FIN；确认与 FIN 有时可以合并。

回答要点

1. 主动关闭方发送 FIN 后等待确认，对端确认只说明收到这一方向的终止声明，不意味着它也准备终止发送；这种半关闭允许一端结束请求后继续接收剩余响应数据。
2. 对端最终发送 FIN，主动方确认并进入相应结束状态；FIN 与 SYN 一样消耗序列号。正常有序关闭、异常 RST 和应用超时触发的关闭，其状态与数据保证不同。
3. 若对端已经没有待发送数据，它的 ACK 与 FIN 可放在同一报文中；同时关闭也有其他状态迁移，所以四次挥手是常见教学流程，而不是不可变化的报文数量定律。

常见追问

追问：CLOSE_WAIT 很多通常看什么？答：它常说明对端已结束发送，而本地应用尚未完成关闭，应该检查读到 EOF 后的处理、资源生命周期和异常路径；状态本身不是证据，仍需结合连接持续时间与业务。

易错点

易错点：对端 ACK 了 FIN 就等于两个方向同时关闭，或者所有关闭都走四个独立报文。半关闭、合并报文和 RST 需要分别解释。

延伸阅读 / 原始参考资料

Linux 手册 • shutdown(2)

Linux 手册 • close(2)

RFC 9293 • TCP — Connection Termination

05. TIME_WAIT 为什么存在？大量出现怎么办？

TCP 与 UDP / 进阶

TIME_WAIT 为已关闭连接保留一段状态，帮助处理对端 FIN 的重传并避免旧报文影响后续同标识连接。典型主动关闭方进入该状态，但同时关闭等情况需按状态机分析。

回答要点

1. 最后确认可能在网络中丢失，对端会重传 FIN；保留状态的一方能再次确认，帮助对端完成关闭。若立即销毁全部状态，可能无法按正常关闭语义响应这些重传。
2. 等待窗口还有隔离旧连接报文的作用，教学通常用 2MSL 表示；MSL 是最大报文生存期模型参数，真实系统的等待时长和复用策略依实现而异，不应固定背一个秒数。
3. 大量短连接和主动关闭会积累 TIME_WAIT，可优先使用连接复用、调整请求模式并排查临时端口压力；是否需要参数调优应看实际耗尽症状，而不是仅凭数量多就关闭保护。

常见追问

追问：TIME_WAIT 一定只在客户端吗？答：不是。角色名称与关闭方向无必然关系，服务器也可主动关闭而进入 TIME_WAIT，同时关闭时双方也可能经历该状态；判断要依据实际 FIN 和状态转换。

易错点

易错点：认为 TIME_WAIT 是内存泄漏，或可无代价强制删除。盲目复用连接标识可能破坏旧报文隔离，应明确系统版本、连接生命周期与真实瓶颈。

延伸阅读 / 原始参考资料

RFC 1337 • TIME-WAIT Assassination Hazards

Linux 手册 • tcp(7)

06. TCP 如何检测丢包并重传？

TCP 与 UDP / 进阶

TCP 依据确认和定时机制恢复丢失数据。超时重传采用随 RTT 更新的 RTO，重复确认可触发快速重传，SACK 提供已收到的数据范围；它们处理信息不足的不同情况。

回答要点

1. 发送方维护尚未被确认的序列范围，确认号通常表示下一个期望字节；没有及时收到确认可能因数据丢失，也可能因确认丢失或网络延迟，所以超时不能直接证明哪个报文丢失。
2. RTO 结合平滑 RTT 和波动估计，而非简单固定为一次 RTT；超时后需要退避。重传报文的 RTT 采样存在歧义，标准算法对这种样本有专门处理，避免错误缩短定时器。

3. SACK 让接收方描述不连续但已收到的范围，帮助发送方只恢复缺失片段；经典快速重传参考重复 ACK，现代实现还可能使用其他丢失检测机制，不能视为永远固定的一套规则。

常见追问

追问：收到三个重复 ACK 一定丢包吗？答：不一定，报文重排序也可能产生重复确认；算法根据线索推断丢失，并存在误判和恢复措施。解释时应区分经典规则与具体系统实际采用的检测算法。

易错点

易错点：把 ACK 数量直接等同于丢包数量，或认为超时重传固定等待一秒。RTT、抖动、退避和选择性确认会影响实际重传时机与范围。

延伸阅读 / 原始参考资料

RFC 6298 • Computing TCP's Retransmission Timer

RFC 2018 • TCP Selective Acknowledgment

RFC 5681 • TCP Congestion Control

07. 流量控制、滑动窗口与零窗口探测？

TCP 与 UDP / 进阶

流量控制避免发送方压垮接收方缓冲区。接收端通告可接收窗口 `rwnd`，发送方据此限制未确认数据；零窗口不表示连接断开，需要探测窗口是否重新开放。

回答要点

1. 滑动窗口让发送方无需每发一段就停下来等待，已确认数据腾出发送范围，未确认数据继续受窗口限制；窗口单位是序列空间中的字节，不能简单看成固定个数的数据包。
2. 接收应用读取慢会占满接收缓冲，通告窗口可能降到零；发送方暂缓正常发送，并通过持续探测避免窗口更新丢失后永久等待，应用仍应设置合适的业务超时。
3. TCP 首部基础窗口字段位宽有限，窗口扩大选项可支持更大的接收窗口；该能力在握手时协商。实际允许发送的数据还要同时受拥塞窗口和已在途数据量约束。

常见追问

追问：扩大接收缓冲一定增加吞吐吗？答：当瓶颈是窗口不足且链路带宽时延积较大时可能有用；若瓶颈在拥塞、应用处理、CPU 或磁盘，更大缓冲可能仅增加排队延迟，需要结合整个路径测量。

易错点

易错点：把 `rwnd` 与 `cwnd` 混成一个窗口，或认为接收窗口为零后只能断连。流量控制反映接收能力，拥塞控制反映网络路径允许的发送量。

延伸阅读 / 原始参考资料

RFC 7323 • TCP Extensions for High Performance

08. 拥塞控制与流量控制有什么区别？

TCP 与 UDP / 进阶

流量控制保护接收端，拥塞控制限制网络中过量在途数据。经典 TCP 通过窗口和阈值组织慢启动、拥塞避免及恢复；实际系统也可能采用 CUBIC、BBR 等算法。

回答要点

1. 发送量通常受 $\min(\text{cwnd}, \text{rwnd})$ 与在途数据共同约束；cwnd 是发送方根据网络反馈维护的状态，不是接收方直接通告，也不能通过扩大接收窗口就完全绕过。
2. 经典慢启动在确认驱动下快速扩大窗口，接近阈值后采用更谨慎的增长；发生超时或明确拥塞线索后降低发送量。具体增长、恢复和阈值更新规则依算法与事件不同。
3. 拥塞线索可以包括丢失、时延和 ECN 标记，不同算法关注的信号不同；讲解 Reno 风格规则时应指出这是经典模型，不将其行为直接套到所有操作系统和网络连接。

常见追问

追问：网络没有丢包是否表示不拥塞？答：不一定。队列可能已经增长而尚未溢出，表现为延迟上升；支持 ECN 的路径也可以标记拥塞。应同时观察吞吐、RTT、队列和显式反馈，而非只看丢包。

易错点

易错点：把慢启动说成增长最慢，或把拥塞避免描述成永远固定的线性增长。阶段名称、经典算法模型与实际部署算法是三个需要说明的层面。

延伸阅读 / 原始参考资料

RFC 5681 • TCP Congestion Control

RFC 9438 • CUBIC

09. TCP 粘包与拆包如何解决？

TCP 与 UDP / 基础

TCP 提供字节流，不保留应用 send 调用的消息边界。一次接收可能包含半条、多条或若干条消息的片段；所谓粘包与拆包是应用消息解析问题，需要明确封装协议。

回答要点

1. 长度前缀先给出载荷长度，接收方缓冲到完整消息再解析；必须限制最大长度并处理整数溢出、非法字段和提前断连，避免攻击者利用长度制造巨大分配或无限等待。

2. 分隔符适合文本协议，但载荷若包含分隔符需转义或有明确语法；固定长度简单但灵活性有限。选择取决于数据类型、吞吐、调试需求和协议可演进性。

3. `read` 或 `recv` 的返回值才是本次实际读取长度，不能假设一次请求会全部满足；发送端也要处理部分写入。接收解析器应保留剩余字节，持续读取并输出完整消息。

消息格式：4 字节长度（网络字节序） + N 字节载荷

接收循环：积累字节 → 校验长度 → 提取完整消息 → 保留余量

常见追问

追问：关闭 Nagle 能解决粘包吗？答：不能。它影响小块发送的聚合策略，字节流语义仍然不保留消息边界。即使每次写入立刻发送，网络分段、缓冲和读取行为也会使接收边界发生变化。

易错点

易错点：用 `sleep` 强行分隔消息，或按数据包长度直接推断业务消息长度。这些做法依赖偶然时序，真正可靠的边界来自应用协议与增量解析。

延伸阅读 / 原始参考资料

Linux 手册 • `recv(2)`

Linux 手册 • `send(2)`

Linux 手册 • `tcp(7)`

10. HTTP/1.1 的连接复用和队头阻塞？

HTTP 与安全 / 基础

HTTP/1.1 使用消息格式在连接上表达请求与响应，默认允许持久连接。复用减少重复建连开销；流水线请求仍要求按顺序返回响应，前面的慢响应可能阻塞后续响应。

回答要点

- 持久连接意味着一次传输连接可服务多个请求，但不能因此说浏览器和服务器永远保持连接；超时、Connection 语义、资源管理和网络错误都会影响实际连接生命周期。
- 消息边界需按协议处理 Content-Length、分块传输和关闭定界等规则；代理与服务器对边界解析不一致可能产生安全问题，因此不能简单以空行后所有字节都当作正文。
- HTTP/1.1 流水线允许尚未收到前一响应就发送下一请求，但响应仍有顺序约束；实际客户端可能使用多个连接缓解等待，不应把 HTTP/2 的多路复用能力赋予 HTTP/1.1。

常见追问

追问：Keep-Alive 和 TCP keepalive 一样吗？答：不是。HTTP 持久连接用于复用应用请求，TCP keepalive 是传输层存活探测机制。开启探测不能保证业务响应及时，也不能替代应用级请求超时。

易错点

易错点：HTTP/1.1 持久连接不等于同时乱序返回多个响应；也不要吧 HTTP 连接复用、TCP 保活和应用心跳混为同一个功能。

延伸阅读 / 原始参考资料

RFC 9112 • HTTP/1.1

11. HTTP/2 怎样多路复用？还会队头阻塞吗？

HTTP 与安全 / 进阶

HTTP/2 将消息编码为帧，在同一连接上用流标识交错承载多个请求，并使用 HPACK 压缩字段。它缓解应用层顺序响应限制，但 TCP 有序字节流仍可使丢失影响多个流。

回答要点

1. 一个流可包含首部与数据帧，不同流的帧能交错发送，接收方按流标识重新组织消息；多路复用减少为并发请求开多条 TCP 连接的需求，却不代表没有资源限额。
2. HTTP/2 同时具备连接级与流级流量控制，主要约束 DATA 帧；服务端还可限制并发流数量。客户端和服务端需要处理流取消、窗口变化及长连接资源占用。
3. 如果 TCP 丢失一个片段，后续字节即使到达也需等待缺失部分恢复，多个 HTTP/2 流可能共同停顿；因此应用层多路复用并没有自动消除传输层队头阻塞。

常见追问

追问：HTTP/2 必须使用 TLS 吗？答：协议本身定义了不同建立方式，不能从标准层面说只允许 TLS；主流浏览器的部署要求与实际支持通常更严格。回答要区分协议能力和具体客户端的实现策略。

易错点

易错点：认为 HTTP/2 每个流对应独立 TCP 连接，或已经消除所有队头阻塞。流独立性存在于帧组织和应用语义层，底层仍共享连接与网络容量。

延伸阅读 / 原始参考资料

RFC 9113 • HTTP/2

RFC 7541 • HPACK

12. HTTP/3、QUIC 与 UDP 是什么关系？

HTTP 与安全 / 进阶

HTTP/3 在 QUIC 上表达 HTTP 语义，QUIC 在 UDP 上提供可靠多路流、安全和连接管理。各流可靠交付相对独立，减少 TCP 层跨流队头阻塞。

回答要点

1. QUIC 包承载帧和流数据，接收方可独立交付不同流的已到达数据；一个流缺少字节仍要等待该流恢复，但通常不需要把其他流的可用数据一起挡在 TCP 字节序列之后。
2. QUIC 使用 TLS 1.3 完成密钥协商与身份验证，传输参数与安全握手结合；连接标识支持特定条件下路径迁移，而迁移还需要路径验证，并不是可无条件接受任意地址。
3. HTTP/3 使用 QPACK 处理字段压缩，避免照搬 HPACK 的连接顺序依赖；UDP 被阻断时可能无法建立 QUIC，客户端需要考虑基于 TCP 的 HTTP 版本回退，收益应按网络情况测量。

常见追问

追问：HTTP/3 还有队头阻塞吗？答：单个可靠流内部仍受有序交付约束，字段解压也可能因动态表依赖等待；消除的是部分跨流的传输阻塞，拥塞、带宽和应用依赖仍然可能共同限制请求。

易错点

易错点：UDP 不可靠不代表 QUIC 不可靠，也不要说 HTTP/3 在任何网络都更快。握手复用、丢包率、部署环境与回退成本决定实际体验。

延伸阅读 / 原始参考资料

RFC 9114 • HTTP/3

RFC 9000 • QUIC

RFC 9001 • TLS for QUIC

13. HTTP 方法、幂等性和状态码怎么回答？

HTTP 与安全 / 基础

HTTP 方法描述操作语义，状态码描述请求处理结果。安全方法原则上不请求改变服务器状态，幂等方法重复同样请求的预期效果与一次相同；它们不等于响应完全相同或业务零副作用。

回答要点

1. GET 与 HEAD 具有安全语义，PUT 和 DELETE 具有幂等语义，POST 一般不保证幂等；实现必须符合这些约定，但日志与统计等附带行为不改变安全语义的基本定义。
2. 2xx 表示成功，3xx 表示进一步行动或重定向，4xx 多为请求相关问题，5xx 多为服务器无法完成有效请求；细分语义要结合具体码，例如 401 与 403、201 与 204 不能互换。
3. 网络超时可能发生在服务端已经完成操作之后，客户端不知道结果；重试非幂等业务应使用业务幂等键或查询结果等机制，不能只依靠 HTTP 方法名称决定交易是否重复执行。

常见追问

追问：DELETE 第一次返回 204、第二次返回 404 还幂等吗？答：可以。幂等关注预期资源状态的效果，不要求每次响应相同；删除后资源都不再存在，具体错误和响应表达仍由接口契约规定。

易错点

易错点：把幂等理解为任意请求可以安全重试，或认为 GET 参数只能在 URL、POST 天然更安全。协议语义、业务约束和传输安全应分别讨论。

延伸阅读 / 原始参考资料

RFC 9110 • HTTP Semantics

14. HTTP 强缓存与协商缓存如何工作？

HTTP 与安全 / 基础

新鲜缓存可直接复用，过期后可向服务器验证。Cache-Control 管理策略，ETag 等字段充当验证器；304 允许复用本地表示，不携带完整正文。

回答要点

1. max-age 描述响应的新鲜寿命，no-cache 要求复用前验证，no-store 要求不存储；名称不能只按日常含义猜测，还需考虑共享缓存、私有缓存及授权响应的限制。
2. ETag 配合 If-None-Match 按表示标签验证，Last-Modified 配合 If-Modified-Since 按时间验证；弱验证器与强验证器适用场景不同，不能一概用于所有范围请求或并发更新检查。
3. 缓存键不仅可能包括 URL，还可能根据 Vary 区分请求字段；忽略语言、编码或认证相关差异会返回错误版本甚至泄漏内容。更新策略要明确版本化资源与动态数据的不同需求。

常见追问

追问：no-cache 会不会完全不缓存？答：它允许存储，但再次使用前必须按规则验证；no-store 才限制存储。已有缓存的处理、浏览器历史机制和应用存储并非都由一个响应字段直接控制。

易错点

易错点：把 304 当成服务器没有数据，或者认为 ETag 一定是文件 MD5。ETag 是服务器选择的表示验证标签，生成方式属于实现细节。

延伸阅读 / 原始参考资料

RFC 9111 • HTTP Caching

15. HTTPS 与 TLS 1.3 握手如何理解？

HTTP 与安全 / 进阶

HTTPS 通过 TLS 为 HTTP 连接提供机密性、完整性与身份验证。TLS 1.3 通常通过临时密钥交换建立会话密钥，用证书与签名验证服务器，再用对称算法保护后续应用数据。

回答要点

1. 客户端发送支持的算法、密钥份额及扩展，服务端选择参数并提供相应握手信息；证书链、域名、有效期和信任根验证共同决定是否信任服务端，不只是看到证书就算安全。
2. 密钥交换、身份签名与记录加密承担不同职责，不能说证书公钥直接用于加密所有 HTTP 数据；TLS 1.3 移除了旧式 RSA 密钥传输，描述旧 TLS 流程时应注明版本。
3. 会话恢复可以减少握手成本，0-RTT 早期数据存在重放风险，应用需限制可接受的操作；TLS 保护传输链路，但终端被入侵、错误授权或业务逻辑漏洞仍由应用负责处理。

常见追问

追问：HTTPS 能隐藏所有网络信息吗？答：不能。观察者通常仍能看到 IP、连接规模与时间等元数据，传统 DNS 和部分握手信息也可能暴露域名线索；应区分加密内容和可观察的流量特征。

易错点

易错点：把 TLS 1.2 的 RSA 传输流程直接套到 TLS 1.3，或认为 HTTPS 自动保证网站业务可信。握手版本、证书验证与应用安全属于不同层次。

延伸阅读 / 原始参考资料

RFC 8446 • TLS 1.3

16. DNS 解析过程和缓存、UDP/TCP 的关系？

网络基础 / 基础

DNS 把域名映射到资源记录。客户端常向递归解析器查询，后者根据缓存和委派关系访问根、顶级域及权威服务器；缓存受 TTL 约束，DNS 并非只使用 UDP。

回答要点

1. 递归查询要求解析器尽力返回结果，迭代响应可给出下一步应询问的服务器；普通客户端通常不自己遍历所有层级，递归解析器会复用委派和记录缓存减少请求。
2. A、AAAA、CNAME、MX 等记录表达不同信息，域名解析不必只返回一个 IP；负缓存也能保存不存在的结果。TTL 是缓存允许保存的时间参数，不保证所有客户端同时更新。
3. 传统 DNS 常以 UDP 发起查询，响应截断等情况下可改用 TCP，区域传送通常使用 TCP；DoT 和 DoH 还可通过加密传输承载 DNS。讨论连接开销时应明确实际协议。

常见追问

追问：修改 DNS 后为什么有人还访问旧地址？答：递归解析器、系统或应用可能仍持有有效缓存，浏览器和连接池也可能继续使用旧连接。应检查记录 TTL、权威响应、实际解析器和应用连接复用。

易错点

易错点：认为一次查询必然依次访问根和全部权威层级，或者 DNS 永远只用 UDP。缓存、委派、截断回退和加密 DNS 都会改变实际流程。

延伸阅读 / 原始参考资料

RFC 1034 • DNS Concepts and Facilities

RFC 1035 • DNS Implementation

RFC 7766 • DNS over TCP

17. IP 地址、子网掩码与 CIDR 如何计算？

网络基础 / 基础

CIDR 用前缀长度标识网络部分，例如 IPv4 /24 有二十四位前缀和八位主机部分。地址与掩码按位与得到网络地址，是否同网段要结合前缀判断，不能只比较点分十进制外观。

回答要点

1. 例如 192.168.10.34/24 的网络地址为 192.168.10.0，传统该广播子网的广播地址为 192.168.10.255；前缀越长，地址范围越小，路由匹配通常优先选择更长前缀。
2. 常规 IPv4 子网的可用主机数常按 2 的主机位次方减二，但 /31 点到点和 /32 主机路由有特殊语义；回答公式必须指出适用范围，不能对全部前缀机械套用。
3. IPv6 地址长度与 IPv4 不同，也没有 IPv4 式广播机制；网络前缀、路由和地址配置规则要按具体协议说明。NAT、私网地址和子网划分是不同概念，不应混为一谈。

192.168.10.34 & 255.255.255.0 = 192.168.10.0
/24 共 256 个地址；常规广播子网通常可用 254 个

常见追问

追问：两个地址看起来前三段相同就同网段吗？答：不一定，还需要各自配置的前缀长度；/25 会把原 /24 分成两个范围。主机发包时依据路由表决定下一跳，判断不只依赖地址字符串。

易错点

易错点：把 /24 看成二十四台主机，或者对 /31 仍一律减去网络地址和广播地址。先确定地址位宽、前缀与链路语义，再做计算。

延伸阅读 / 原始参考资料

RFC 4632 • CIDR

RFC 3021 • 31-Bit Prefixes

18. 路由、最长前缀匹配和 NAT 的区别？

网络基础 / 基础

路由选择数据包下一跳，最长前缀匹配优先选择最具体的匹配路由；NAT 改写地址或端口实现映射。路由本身不意味着修改端到端地址，NAT 也不是通用安全控制的替代。

回答要点

1. 主机向远端发送时通常把链路帧交给网关，IP 目的仍是远端；路由器按表选择输出接口和下一跳，并递减 TTL，必要时返回 ICMP 错误，避免报文无期限循环。
2. 若同时匹配默认路由和更具体前缀，先选更长前缀；同长度匹配再按系统的优先级、度量或策略处理。策略路由等扩展可能加入源地址和其他条件，应说明普通模型范围。
3. NAT 表记录转换关系，多个内网连接可通过不同端口共用公网地址；这种映射影响入站连接、连接追踪与超时。防火墙过滤规则和 NAT 地址改写应分别理解。

常见追问

追问：为什么内网主动连公网通常容易，公网主动连内网却困难？答：主动连接可以建立 NAT 映射并放行对应返回流量，未知入站连接没有对应目标映射；端口转发或其他连接建立机制可改变这个行为。

易错点

易错点：认为路由器逐跳修改目的 IP 为下一跳 IP，或只要做 NAT 就等于安全。普通转发改变的是链路交付目标，而网络层目的通常仍指向最终目标。

延伸阅读 / 原始参考资料

RFC 1812 • IPv4 Routers

RFC 3022 • Traditional NAT

19. ARP 如何工作？IPv6 用什么？

网络基础 / 基础

ARP 在 IPv4 的本地链路上解析下一跳 IP 对应的链路地址，常先查缓存再广播请求。跨网段访问通常解析网关而非远端主机的 MAC；IPv6 使用邻居发现协议。

回答要点

1. 发送方先按路由确定下一跳，再在本地链路查询该下一跳的地址映射；广播 ARP 请求通常被同链路主机接收，持有目标地址的一方响应，得到映射后才能构造链路帧。
2. ARP 缓存减少每次发送前的广播，条目需要更新和失效；ARP 缺少强认证机制，错误或恶意应答可能改变映射，应结合链路隔离、检查机制和实际部署考虑防护。

3. IPv6 邻居发现基于 ICMPv6，除地址解析外还涉及邻居可达性和路由器发现等功能；不能把 IPv4 ARP 报文直接用于 IPv6，也不能把两者视为报文格式完全相同。

常见追问

追问：访问远程网站需要知道服务器的 MAC 吗？答：一般不需要。主机需要本地下一跳的链路地址，路由器逐跳重新组织链路交付；服务器 MAC 不会作为统一端到端标识穿越整条路由路径。

易错点

易错点：认为 ARP 在互联网范围广播寻找服务器，或 ARP 只查最终目标。它主要服务本地链路交付，目标由路由选择的下一跳决定。

延伸阅读 / 原始参考资料

RFC 826 • Address Resolution Protocol

RFC 4861 • IPv6 Neighbor Discovery

20. 输入 URL 后发生什么？socket 在哪里？

实践与排查 / 基础

浏览器解析 URL、检查策略与缓存，取得目标地址并建立或复用连接，完成必要安全握手后发 HTTP 请求。socket 是程序使用传输服务的接口，页面显示还需要资源加载、解析与渲染。

回答要点

1. TCP 服务端常按 socket、bind、listen、accept 组织连接，客户端使用 connect；accept 返回已连接的通信描述符，监听描述符继续负责接收新连接，两者用途不能混淆。
2. 浏览器可能命中 DNS、HTTP 缓存或连接池，HTTPS 可能使用 TCP+TLS 或 QUIC；重定向会触发新的目标处理，实际链路不必每次从头进行全部步骤，要说明缓存和协议分支。
3. 收到 HTML 后解析 DOM、样式并发起子资源请求，布局、绘制和合成影响可见时间；网络请求完成并不等于页面可交互，排查应分别看 DNS、连接、首字节、下载和渲染阶段。

常见追问

追问：connect 成功能证明服务业务正常吗？答：不能，它主要说明传输连接建立，服务器应用可能繁忙、挂起或返回错误；应该检查业务请求与响应，并设置连接超时和请求超时，避免把探活层次混淆。

易错点

易错点：把 URL 流程背成每次都重新查 DNS 和三次握手，或把 accept 当成创建监听服务。连接复用、HTTP/3 和缓存会改变步骤，接口职责也需明确。

延伸阅读 / 原始参考资料

Linux 手册 • socket(7)

Linux 手册 • accept(2)

计算机组成原理

理解硬件如何执行程序

从数据表示、指令执行和存储层次到流水线、多核一致性与性能分析。教学 CPU 用于解释原理，实际 x86、Arm、RISC-V 的细节需以对应手册为准。

01. 冯·诺依曼结构与哈佛结构有什么区别？

指令与数据表示 / 基础

冯·诺依曼模型把指令与数据放在统一存储体系，CPU 按存储的程序执行；哈佛结构区分指令和数据存储或通路。现代处理器常在统一地址空间下使用分离的一级指令与数据缓存。

回答要点

1. 存储程序思想把程序本身编码为可存储的信息，程序计数器指出后续执行位置，控制逻辑协调取指、运算和数据访问；它并不要求每条指令都严格按五个固定硬件周期执行。
2. 共享存储通路可能让取指与访存争用资源，形成带宽瓶颈；分离的指令和数据通路可减少这类冲突，但会增加设计复杂度。瓶颈应结合实际缓存和总线结构分析。
3. 现代 CPU 可同时具有统一主存、分离 L1 缓存、流水线与乱序执行；不能只用冯·诺依曼或哈佛标签推出性能与所有细节，应分别说明地址空间和实际物理访问结构。

常见追问

追问：代码和数据共用内存为什么还能禁止写代码？答：统一存储模型不意味着权限相同，系统可以通过页面权限区分可写和可执行区域；代码修改还涉及指令缓存同步等机制，具体规则依体系结构而异。

易错点

易错点：把现代 CPU 归为一种模型后就认为指令和数据一定共享每一级缓存，或者哈佛结构必然更快。教学分类不能代替具体微架构分析。

延伸阅读 / 原始参考资料

MIT 6.004 • Programmability

MIT 6.004 • Building the Beta

02. 一条指令从取指到完成经历什么？

指令与数据表示 / 基础

教学模型常把执行分为取指、译码、执行、访存和写回。控制单元解释操作码，数据通路完成运算与访问；现代处理器可能把指令拆为内部操作并乱序执行，最终维护架构语义。

回答要点

1. 取指依据程序计数器读取指令，译码识别操作及源、目的位置；跳转、异常和中断可能改变后续取指位置，因此程序计数器并不是永远按一个固定值线性增加。
2. ALU 负责整数与逻辑运算，加载存储单元负责相应数据访问，寄存器保存当前操作数据；并非所有指令都需要访存，也并非每条指令都在同样长度的周期内完成。
3. 流水化、超标量和内部微操作改变执行组织，但软件观察的指令效果仍由 ISA 定义；对现代乱序机器，应区分发射、执行完成和退休，不能把三者当成同一时刻。

常见追问

追问：为什么除法和加法可能耗时不同？答：两种运算的数据通路和算法不同，依赖长度、硬件实现及操作数也可能影响延迟；即便指令表面同样只有一条，其执行延迟与可持续吞吐也未必相同。

易错点

易错点：把五阶段教学流水线描述成所有 CPU 的真实结构，或把单条指令等同于一个时钟周期。回答先注明所用模型，再区分微架构实现。

延伸阅读 / 原始参考资料

MIT 6.004 • Building the Beta

Intel • Architecture Software Developer Manuals

03. ISA、微架构与 RISC/CISC 怎么区分？

指令与数据表示 / 基础

ISA 定义软件可见的指令、寄存器和行为，微架构定义硬件如何实现这些行为。RISC 与 CISC 描述不同设计传统，但现代实现会融合多种技术，不能单凭标签判断性能和功耗。

回答要点

1. 同一个 ISA 可以由顺序或乱序、不同流水深度和缓存配置的处理器实现，程序兼容性来自架构契约；扩展指令是否可用仍需检查具体 CPU 与运行环境的能力。
2. RISC 常强调较规整编码与加载存储组织，CISC 常允许更复杂的指令与寻址；现代 x86 可译码为内部操作，RISC-V 也有压缩指令扩展，所以固定长度和简单实现并非绝对。
3. 比较性能应看编译结果、指令数量、每条指令平均代价、频率以及存储访问；指令更少未必更快，更长指令也未必更低效，复杂度可能在编译器与硬件之间重新分配。

常见追问

追问：RISC 是不是只能用很少几种指令？答：不是，它描述设计取向而不是指令总数的严格阈值；基础 ISA 与扩展可覆盖整数、浮点、向量和原子操作。应说明具体 ISA 版本及所讨论的扩展集合。

易错点

易错点：认为 RISC 指令一定单周期完成、CISC 一定更慢，或把 ISA 与某款 CPU 的流水线等同。架构语义、实现成本和实际性能需要分别讨论。

延伸阅读 / 原始参考资料

MIT 6.004 • Designing an Instruction Set

RISC-V • RV32I 2.1

04. 补码、符号扩展和整数溢出怎么解释？

指令与数据表示 / 基础

n 位补码表示范围为负 2 的 $n-1$ 次方到正 2 的 $n-1$ 次方减一。负数可由对应正数按位取反再加一得到；符号扩展复制符号位保持数值，硬件位运算规则与语言规则需区分。

回答要点

1. 补码把加减统一到模 2 的 n 次方的加法结构，并仅保留一个零；最高位既参与编码也指示符号，但不能把负数解释为一个独立符号位加上其余位的绝对值。
2. 例如八位的负一为全一，扩展到十六位后仍需全一；无符号数则使用零扩展。读取同样位模式时，有符号和无符号解释不同，混合比较与类型转换可能造成意外结果。
3. 无符号加法的最高进位与有符号溢出不是同一判断；两个同号数相加得到异号结果可提示补码有符号溢出。C++ 有符号溢出的语言语义不能直接假设为硬件自然回绕。

8 位补码：+5 = 00000101, -5 = 11111011

范围：[-128, 127]；-1 符号扩展后仍是全 1

常见追问

追问：最小负数为什么取相反数可能溢出？答：补码负数范围比正数多一个值，八位可表示负一百二十八但不能表示正一百二十八；更宽中间类型或显式边界检查才能安全处理这个不对称范围。

易错点

易错点：把符号扩展写成一律补零，或把 CPU 产生的位结果直接视为 C++ 有符号溢出的保证。位模式、数学值与语言行为是不同层面。

延伸阅读 / 原始参考资料

RISC-V • Integer Computational Instructions

05. 大端、小端和内存对齐是什么？

指令与数据表示 / 基础

端序决定多字节值在内存中的字节顺序，小端把低有效字节放在低地址，大端相反。对齐要求对象或访问地址满足边界条件；跨平台存储与网络协议应显式规定编码而非直接复制结构体。

回答要点

1. 四字节数 0x12345678 在小端低到高地址为 78、56、34、12，大端为 12、34、56、78；端序描述字节排列，不表示每个字节内部的二进制位也被反转。
2. 不同 CPU 对未对齐访问支持不同，可能正常完成但增加代价，也可能触发异常；跨缓存行或页面的访问还可能有额外成本，不能把某一平台的容忍行为当成通用保证。
3. 结构体可能含填充字节，字段大小和端序也可能变化；传输或持久化应逐字段序列化，使用明确宽度和边界校验。网络字节序一般采用大端，但协议正文仍以自身规定为准。

常见追问

追问：能把一个 C++ 结构体直接发到网络吗？答：只有双方严格共享布局和编码约定等条件下才可能工作，通常应显式序列化；指针字段尤其只是本进程地址，不能被远端当作可用对象引用。

易错点

易错点：把大端理解成最高位位于最低地址，忽略单位是字节；或认为对齐只影响性能不影响合法性。语言对象访问规则与硬件容忍能力都需考虑。

延伸阅读 / 原始参考资料

Linux 手册 • endian(3)

MIT 6.004 • Machine Language, Assemblers, and Compilers

06. IEEE 754 浮点数为什么有误差？

指令与数据表示 / 基础

常见二进制浮点使用符号、指数和有效数字表示值，有限位数只能精确表达部分实数。十进制 0.1 的二进制展开无限，因此会舍入；浮点加法还可能因舍入次序而不满足结合律。

回答要点

1. binary32 的典型字段是一个符号位、八个指数位和二十三个显式小数位，规格化值还有隐含前导位；非规格化值、零、无穷大和 NaN 使用特殊编码规则。
2. 较大数与很小数相加时，小数的贡献可能在舍入中消失；相近数相减会放大已有相对误差。分析误差应关注量级与算法稳定性，不只是笼统说电脑计算不准确。
3. 比较计算结果常需结合绝对与相对容差，且单独处理 NaN、无穷和特殊业务边界；金融金额等场景可按单位使用整数或适合的十进制表示，但也需定义舍入与范围。

常见追问

追问：double 比 float 更精确，是否就没有误差？答：不会。更多有效位减少很多舍入误差，但有限二进制表示仍无法精确覆盖所有十进制小数；换类型不能替代数值稳定算法和明确的比较、舍入规则。

易错点

易错点：认为所有浮点值都是近似值，实际上很多整数与二的幂分数可精确表示；也不用固定 epsilon 处理任意数量级而忽略相对误差。

延伸阅读 / 原始参考资料

Oracle • Floating-Point Arithmetic

RISC-V • F Extension 2.2

07. 存储层次与局部性为什么有效？

存储层次 / 基础

寄存器、各级缓存、主存和外部存储形成速度、容量与成本的层次。时间局部性表示近期数据可能再次使用，空间局部性表示附近数据可能被使用；缓存通过保留热点降低平均访问代价。

回答要点

1. 靠近执行单元的存储通常更快但容量更小，每一层为更慢的一层保留部分内容；命中率只是一个因素，还需考虑命中延迟、未命中代价、并发访问与可持续带宽。
2. 顺序访问数组可利用缓存行和预取，随机追逐指针可能因为地址不可预测且存在依赖而难以并行；同样的大 O 复杂度也可能因数据布局与访问模式呈现不同实际耗时。
3. 层次结构的精确配置依处理器而异，L1 指令和数据缓存常分离，末级缓存可能共享或分片；不能背诵固定延迟或容量来代表所有 CPU，应注明型号与测量条件。

常见追问

追问：链表插入是 $O(1)$ ，为什么不一定快于数组？答：找到位置本身可能需要遍历，节点分散导致缓存未命中和依赖访问；数组搬移虽然涉及更多元素，却常能连续批量访问。应比较完整操作与数据规模。

易错点

易错点：认为 cache 命中就没有延迟，或把主存和磁盘访问混成同等代价。缓存收益来自访问模式，数据大小和局部性比笼统的复杂度标签更能解释差异。

延伸阅读 / 原始参考资料

MIT 6.004 • The Memory Hierarchy

Intel • Architecture Optimization Manuals

08. cache 的直接映射、组相联和全相联？

存储层次 / 基础

cache 把地址映射到缓存行。直接映射每块只有一个候选位置，组相联可放到固定组内多个位置，全相联允许任意行；更多选择减少冲突，但增加比较、替换和硬件成本。

回答要点

1. 典型地址拆为标记、组索引和块内偏移：索引确定候选组，标记确认当前缓存行对应的内存块，偏移定位行内字节；位数取决于地址位宽、行大小、组数和映射方式。
2. 直接映射硬件简单，但两块热点若映射到同一位置会反复互相淘汰；组相联提供多路候选，全相联更灵活，替换策略负责在候选位置已满时选择被逐出的行。
3. 增大相联度通常降低部分冲突，却可能增加命中延迟、能耗和设计复杂度；实际最优点取决于访问模式。容量相同也不代表冲突行为相同，应区分容量与相联度。

示例：32 KiB、64 B 行、8 路组相联

组数 = $32768 / (64 \times 8) = 64$

块内偏移 6 位，组索引 6 位

常见追问

追问：总容量相同，为什么程序改变数组间隔后变快？答：地址间隔可能改变热点映射到的 cache 组，减少互相冲突；也可能同时改变 TLB 或预取行为。需要用控制实验与计数器验证，不能仅凭一次时间比较。

易错点

易错点：把组相联中的一路理解为一个独立完整缓存，或认为提高相联度必然全方面更快。比较应同时考虑命中率、命中时间与替换成本。

延伸阅读 / 原始参考资料

MIT 6.004 • Cache Issues

09. cache 写直达、写回与写分配怎么区分？

存储层次 / 进阶

写直达在写 cache 时把更新传播到下层，写回先标脏、淘汰时再写下层。写分配决定写未命中时是否装入缓存行，它与传播策略是不同维度；一致性和持久化也不能混同。

回答要点

1. 写直达便于保持下层较新，但频繁写入会增加流量，常配写缓冲降低停顿；写回可合并多次修改，节约带宽，却需要脏位并在逐出时处理尚未传播的数据。

2. 写分配在写未命中时取入对应行，随后在 cache 修改；不写分配可直接把写送往下层。常见组合有规律但不是强制规则，应说明所讨论缓存层次和指令行为。
3. CPU 缓存中的更新被其他核看到，通常依一致性协议而非每次立刻写回主存；普通写回主存也不意味着断电持久化，持久内存或存储设备还需要对应的刷新与排序协议。

常见追问

追问：写回 cache 的数据未进 DRAM，其他核还能读到新值吗？答：可以，一致性协议可从拥有最新缓存行的核获得数据或协调状态；新值的可见性与必须立即更新 DRAM 是不同要求，还需满足程序的同步顺序。

易错点

易错点：认为写回就是数据丢失，或写直达自动解决多线程同步。缓存传播、缓存一致性、内存顺序和持久性有各自的保证范围。

延伸阅读 / 原始参考资料

Intel • Architecture Software Developer Manuals

Intel • Architecture Optimization Manuals

MIT 6.004 • Cache Issues

10. 如何分析 cache miss 与平均访问时间？

存储层次 / 进阶

cache miss 可按首次访问、容量不足和映射冲突等原因分析。简单模型用命中时间加未命中率乘未命中代价估算平均访问时间；现代 CPU 能重叠访问，实际性能还受依赖与带宽限制。

回答要点

1. 首次访问对应冷未命中，工作集大于缓存容量产生容量压力，有限映射候选产生冲突；多核共享还可因一致性失效产生额外未命中，不能强行塞进单核三类模型。
2. 多级缓存应说明每一级的局部未命中率和到最终层的总体未命中率，不能把数字直接混用；一次 L1 miss 不意味着必然到 DRAM，也可能在 L2 或末级缓存找到。
3. 连续独立访问可形成多个在途请求，隐藏部分延迟；指针链等串行依赖则更容易被单次延迟限制。优化应选择改善布局、分块、减少工作集或增加可并行访问等具体方向。

简单单级模型： $AMAT = \text{命中时间} + \text{miss 率} \times \text{miss 代价}$

若命中 1 ns，miss 率 5%，额外代价 60 ns： $AMAT = 4 \text{ ns}$

常见追问

追问：未命中率降了一半，程序就快一倍吗？答：不一定。未命中只占整体耗时的一部分，部分访问原本与计算重叠，优化还可能增加指令或容量；应比较端到端时间并确认关键路径和瓶颈是否改变。

易错点

易错点：把理论平均访存公式当作完整的现代 CPU 性能预测器，或只看命中率不看代价。吞吐、依赖链、带宽和多级缓存会改变实际表现。

延伸阅读 / 原始参考资料

Intel • Architecture Optimization Manuals

MIT 6.004 • The Memory Hierarchy

11. MMU、页表与内存保护如何配合？

存储层次 / 基础

MMU 按页表把虚拟地址转成物理地址并检查权限，TLB 缓存转换。页表由操作系统管理，硬件支持与系统策略共同实现隔离；缺页异常把无法直接完成的访问交给内核处理。

回答要点

1. 虚拟地址可以拆成页号与偏移，页表提供物理页号及存在、读写、执行等属性；同样数值的虚拟地址在不同地址空间可对应不同物理资源，不能直接用于设备 DMA。
2. TLB 是转换缓存，不是数据 cache，未命中可能只是需要遍历页表；页表项无效或权限不符才可能触发相应异常。不同体系结构的权限组合与异常细节需查对应手册。
3. 用户态访问受到地址空间和权限约束，内核可以改变映射实现共享、按需分配或写时复制；更新页表后要使相关旧转换失效，多核系统还需协调其他 CPU 的转换缓存。

常见追问

追问：为何同一物理页能同时映射到多个进程？答：多个页表项可以指向相同物理页，并按需要设置读写权限；共享库或共享内存利用这种机制。映射共享并不保证应用并发访问安全，仍需同步和生命周期管理。

易错点

易错点：虚拟地址不能与物理地址、DMA 地址直接互换，也不要吧 TLB miss 当作缺页。转换缓存、页表状态和访问权限是不同检查环节。

延伸阅读 / 原始参考资料

MIT 6.004 • Virtual Memory

Linux • DMA Address Spaces

12. 流水线提高的是吞吐还是单条延迟？

流水线与执行 / 基础

流水线把执行过程划成可重叠阶段，让多条指令同时处于不同阶段，主要提高持续吞吐。单条指令仍需走过多个阶段，还增加级间寄存器开销，不保证它的端到端延迟缩短。

回答要点

1. 理想 k 级流水线处理 n 个相同任务，需要约 $k+n-1$ 个周期；启动和排空使短任务达不到稳态收益，真实情况还受到阶段不均衡、停顿与资源共享影响。
2. 时钟周期由最慢阶段及寄存器、时序约束等共同决定，继续细分可提高频率但增加寄存器成本；更深流水线也使预测失败和异常恢复可能付出更多工作丢弃代价。
3. 延迟是一个任务从进入到完成的时间，吞吐是单位时间完成任务数量；一条依赖链更受延迟影响，大量独立任务可能更能发挥吞吐，解释优化时必须明确测量对象。

常见追问

追问：五级流水线每周期完成一条指令，为什么一条仍需五周期？答：不同指令在各阶段交错执行，稳态每周期完成的是其中一条；这一输出间隔不等于同一条指令从进入到结束的总时间。

易错点

易错点：把 CPI 接近一解释成所有指令一周完成，或认为流水线深度翻倍性能必翻倍。启动、阶段时序和冒险会限制理想收益。

延伸阅读 / 原始参考资料

MIT 6.004 • Pipelining

13. 结构、数据和控制冒险如何处理？

流水线与执行 / 进阶

结构冒险来自资源冲突，数据冒险来自指令间依赖，控制冒险来自下一条路径未确定。可通过增加资源、旁路、停顿、调度与预测等手段处理；旁路不能消除所有依赖延迟。

回答要点

1. 若取指和数据访问争用同一资源，可分离通路或调度使用时间；结构冒险讨论的是同时需要某资源却不能满足，而不是数据值是否正确，不同问题需使用不同手段。
2. 后一指令需要前一结果形成 RAW 真依赖，旁路可把尚未写回的结果直接送到需要处；但 load-use 等情况仍可能来不及提供数据，必须停顿或安排独立指令填充。
3. 分支结果尚未确定时，取指可能走错路径，停顿最简单但降低吞吐；预测先猜测路径，错误时恢复。WAR、WAW 等名字依赖在特定乱序环境下可由寄存器重命名缓解。

常见追问

追问：寄存器重命名能消除 RAW 吗？答：不能。RAW 表示真正需要先前计算结果，换个寄存器名字不会生成尚未算出的值；重命名主要去掉复用同一架构寄存器产生的名字依赖，仍需遵守真实数据关系。

易错点

易错点：把所有数据冒险都用旁路即可解决，或把三类冒险混为 CPU 太慢。必须指出冲突资源、依赖指令和需要等待的值或控制信息。

延伸阅读 / 原始参考资料

MIT 6.004 • Pipelining the Beta

MIT 6.004 • Pipeline Issues

14. 分支预测、乱序执行与顺序退休？

流水线与执行 / 进阶

分支预测在结果确定前选择可能路径，乱序执行让已就绪的独立操作提前利用资源；顺序退休确保对软件提交的架构状态符合程序顺序，并帮助维持可恢复的精确异常。

回答要点

1. 预测器利用历史等信息猜测分支方向和目标，推测路径上可先做工作；错误时丢弃相关结果并恢复正确路径。更高预测命中率能减少浪费，但收益依分支频率和恢复代价而异。
2. 乱序调度依据操作数和执行资源是否就绪，不代表忽略程序依赖；寄存器重命名减少名字冲突，重排序缓冲等结构跟踪结果与退休顺序，具体设计依微架构不同。
3. 精确异常希望发生错误时，前面的指令已经完成提交，出错及其后的指令尚未改变可见架构状态；微架构缓存等痕迹则可能保留，安全分析不能只看架构结果是否回滚。

常见追问

追问：既然最终顺序退休，乱序有什么价值？答：退休顺序限制的是结果提交，不禁止独立操作在执行阶段重叠；当一条指令等待数据时，其他就绪操作能继续推进，从而减少空闲周期并提高资源利用。

易错点

易错点：把乱序执行理解为程序结果可以任意乱序，或推测失败后硬件所有内部状态都恢复到未执行时。架构状态与微架构影响必须分别描述。

延伸阅读 / 原始参考资料

MIT 6.004 • Pipeline Issues

Intel • Architecture Optimization Manuals

15. 中断、异常与精确异常怎么区分？

I/O 与多核 / 基础

中断通常由外部设备或定时器等异步事件触发，异常通常与当前指令执行相关。CPU 按机制保存必要现场并转到处理入口；精确异常让处理程序看到可解释、可恢复的架构状态。

回答要点

1. 外设中断让 CPU 不必不停轮询设备，定时器中断可支持抢占调度；是否立即响应取决于屏蔽、优先级和处理器状态，不应说设备一发信号就立刻执行全部后续处理。
2. 非法指令、访问保护和缺页属于典型同步异常，不同架构对 fault、trap 和 abort 等术语划分不同；系统调用也可通过特定受控指令进入内核，不能统称为外设中断。
3. 中断入口要保护寄存器和返回信息，处理程序完成后恢复被中断的执行，或者由内核选择其他任务；保存现场与切换到新任务不是相同动作，中断并非必然导致调度切换。

常见追问

追问：为什么中断处理应尽量短？答：过长处理可能影响其他中断响应与任务延迟，且部分上下文不能睡眠；常把必须立即完成的工作与可延后的处理分开，但具体机制、允许操作和优先级取决于系统。

易错点

易错点：把异常都理解成程序必须终止，缺页等可以正常恢复；也不要一次中断等同于一次进程切换。事件入口、异常语义和调度策略是不同层面。

延伸阅读 / 原始参考资料

MIT 6.004 • Devices & Interrupts

Intel • Architecture Software Developer Manuals

16. DMA 为什么减少 CPU 开销？IOMMU 做什么？

I/O 与多核 / 进阶

DMA 让设备按描述符等安排直接搬运内存数据，CPU 负责配置、管理缓冲与处理完成。IOMMU 可转换并限制设备访问的地址空间；DMA 地址不必等于 CPU 虚拟或物理地址。

回答要点

1. CPU 逐字节搬运会占用执行资源，DMA 可让设备批量传输并与计算重叠；建立映射、提交描述符和完成处理仍有成本，较小传输不一定足以抵消管理开销。
2. 设备使用的总线地址可能由 IOMMU 映射到物理内存，驱动应使用平台 DMA API 获取正确地址；简单把用户或内核指针交给设备，可能访问错误位置或违反隔离边界。
3. CPU cache 与 DMA 的数据一致性依平台能力和映射模式不同，需要遵守缓冲所有权、同步与内存排序规则；缓冲区在传输完成前不能被提前释放或不受约束地复用。

常见追问

追问：DMA 完成后 CPU 能立刻读取缓冲吗？答：需按驱动协议确认完成，并执行该平台或 API 要求的同步和排序；一致性 DMA 与流式映射的规则不同，不能因某款 x86 上看起来正常就推广到所有平台。

易错点

易错点：DMA 不代表 CPU 完全不参与，也不意味着设备可使用普通指针。地址转换、缓存一致性、生命周期和完成通知共同决定正确性。

延伸阅读 / 原始参考资料

Linux • Dynamic DMA Mapping Guide

Linux • DMA API

17. 缓存一致性、MESI 与内存一致性模型？

I/O 与多核 / 进阶

缓存一致性协调多个核对同一内存位置的缓存副本，MESI 用修改、独占、共享、无效等状态组织权限。内存一致性模型则约束多个位置的读写顺序；有缓存一致性仍需要程序同步。

回答要点

1. 某核要写共享缓存行时，协议需取得合适权限并使其他副本失效或更新，其他核读取时获取最新许可的数据；协议可通过总线侦听或目录组织，真实处理器未必只有四个 MESI 状态。
2. 一致性常按缓存行管理，一行的修改可能影响同一行中的其他字段；最新数据也可在缓存之间传递，不必每次先写主存再读取，状态表示比单纯的主存是否最新更丰富。
3. 单地址修改顺序正确，不自动保证读者按程序预期看到多个地址之间的发布顺序；编译器和 CPU 可采用允许的优化与缓冲，锁或带内存顺序的原子操作才建立所需同步关系。

常见追问

追问：硬件会保持缓存一致，为什么 C++ 仍可能出现数据竞争？答：语言的并发规则不仅要求缓存副本最终协调，还要求冲突访问有正确的同步关系；普通非原子并发读写可能触发未定义行为，不能靠硬件协议修复。

易错点

易错点：把缓存一致性说成所有核每次都从 DRAM 读取最新值，或认为 MESI 等于跨地址顺序一致。行权限协调、可观察顺序与语言规则是不同保证。

延伸阅读 / 原始参考资料

MIT 6.004 • Parallel Processing

Linux • Memory Barriers

18. 什么是伪共享？如何发现与改善？

I/O 与多核 / 进阶

多个核访问同一缓存行内不同字段，至少一个频繁写入时，可能因行级一致性产生无意义的失效与转移，这称为伪共享。数据逻辑上独立不代表硬件管理粒度独立。

回答要点

1. 例如每个线程维护自己的计数器，但计数器紧邻放在同一行，写入各自字段仍会让整行在核间争用；只有只读访问通常不会出现同等写失效开销，必须明确写入条件。
2. 可将高频写字段按缓存行分隔，或让线程先在本本地累计再批量汇总；填充增加内存占用并可能恶化缓存和 TLB 压力，因此应优先减少共享写，并针对真实热点调整布局。
3. Linux 下可借助 `perf c2c` 等工具分析缓存行转移与访问位置，再结合结构布局确认字段是否冲突；HITM 等事件也可来自真正共享，不能仅凭计数就断言全是伪共享。

常见追问

追问：给结构体加 `alignas` 就一定解决了吗？答：不一定。对齐只保证起始位置满足边界，仍需检查大小、数组元素间隔以及被其他字段占据的行；还要确认平台行大小，并测量收益是否超过空间开销。

易错点

易错点：把多个线程使用同一个变量的真实共享称为伪共享，或固定认为所有 CPU 缓存行都是六十四字节。根因是无关数据落入同一硬件一致性粒度。

延伸阅读 / 原始参考资料

Linux • False Sharing

Intel • False Sharing Performance Pattern

19. 原子指令、内存屏障和 `acquire/release`?

I/O 与多核 / 进阶

原子操作保证所定义操作不会被观察为部分完成，内存顺序与屏障约束允许的可观察先后。

`acquire/release` 可通过同一同步对象发布数据；原子性不等于所有周围访问都自动顺序一致。

回答要点

1. 原子读改写可支持锁和无锁结构，但竞争会产生缓存行转移，未必比锁更快；CAS 还需处理失败重试和 ABA 等算法问题，LR/SC 与 CAS 的实现条件也各有差异。
2. 发布方先写数据再 `release` 写标志，读取方以 `acquire` 观察到对应发布后，建立相关的同步关系；只使用 `relaxed` 标志通常不能发布普通数据，必须说明读到哪个写及对象生命周期。
3. 编译器屏障与 CPU 屏障负责不同约束，屏障也不是一律把缓存清空；用户态 C++ 应优先按语言原子与锁的规则编码，再理解编译器映射到特定 ISA 的指令，避免依赖偶然硬件行为。

常见追问

追问：volatile 能替代原子和锁吗？答：普通 C++ volatile 不能提供线程间需要的原子性与同步关系，主要涉及特定访问语义；应使用 `std::atomic` 或锁，并选择足以建立不变量的内存顺序，而不是仅阻止某些优化。

易错点

易错点：认为原子自增就自动保护所有旁边变量，或屏障就是强制立即写 DRAM。单操作原子性、跨操作顺序与缓存传播必须分别分析。

延伸阅读 / 原始参考资料

RISC-V • A Extension 2.1

Linux • Memory Barriers

20. CPU 性能、Amdahl 定律、SIMD 与 NUMA 怎么分析？

性能分析 / 进阶

CPU 时间可由指令数、平均 CPI 和频率作基础分析，Amdahl 定律限制局部优化收益。SIMD 改善数据并行运算，NUMA 改变访存成本；最终需测量端到端表现。

回答要点

1. 频率提高不保证程序同比加速，等待内存、I/O 或同步的时间可能不随频率缩短；平均 CPI 也取决于指令组合和执行环境，不能把不同 ISA 的指令数直接当作公平速度比较。
2. 若仅部分耗时被优化，整体加速受剩余部分限制，例如原耗时的一半提速两倍，总体约为一又三分之一倍；增加线程还会带来调度、通信和竞争，理论公式只是理想边界。
3. SIMD 让一条指令作用于多个数据元素，但要求适合的数据布局与依赖关系；NUMA 中本地与远端内存代价可能不同，线程放置和内存分配会影响带宽与延迟，应结合硬件拓扑测量。

CPU 时间 = 指令数 × 平均 CPI / 时钟频率

Amdahl 加速比 = $1 / ((1-p) + p/s)$

$p=0.5, s=2 \rightarrow$ 总体加速约 1.33 倍

常见追问

追问：CPU 利用率接近百分之百是否就证明算力不足？答：不证明。忙等锁、无效重试和缓存争用也会让 CPU 忙碌，却没有增加有效工作；应查看热点、等待、带宽、缓存事件及业务吞吐，确认资源用于什么。

易错点

易错点：用主频、核心数或 MIPS 一个数字判断性能，或按理想 Amdahl 公式忽略并行管理开销。比较应固定任务、输入、环境，并同时关注延迟与吞吐。

延伸阅读 / 原始参考资料

MIT 6.004 • Cost/Performance Tradeoffs

MIT 6.004 • Parallel Processing

Linux • What Is NUMA?

Intel • Architecture Optimization Manuals

MySQL

索引、事务与查询优化

以 MySQL 8.4 LTS 与 InnoDB 为基准，覆盖从 B+ 树与 MVCC 到慢 SQL、复制一致性和表设计的高频追问。

01. 为什么 InnoDB 使用 B+ 树索引？

索引与存储 / 基础

InnoDB 普通索引采用面向页的 B 树结构，面试通常称为 B+ 树。高扇出降低树高，叶子保存索引记录并支持有序遍历，因此同时适合等值定位和范围查询。

回答要点

1. 页是磁盘与缓冲池管理的重要单位，一个内部节点可容纳大量键和子页指针。相同记录数下，多路树比二叉树层数更少，能减少随机访问页面的次数，而不是简单靠比较次数取胜。
2. 范围查询先定位起点，再按叶子记录的顺序向后扫描。哈希结构擅长等值查找，却无法天然支持有序输出、范围条件和前缀查找，因此不适合作为通用关系表的唯一索引结构。
3. 普通索引和空间索引不能混为一谈，空间索引使用 R 树。默认索引页大小为 16KB，但实例初始化可选择其他支持的页大小，树高还取决于键长度、记录大小以及页面填充情况。

常见追问

追问：索引越多越快吗？不是。每次插入、删除及索引列更新都要维护相关索引，并占用存储和缓冲池。应根据高频查询、选择性与写入比例保留有效索引，用实际计划和负载验证收益。

易错点

不要背出固定的“几层支持多少亿行”作为保证。树高由页大小与键宽度等共同决定，数据库命中缓冲池后，也不一定每层都产生磁盘 I/O。

延伸阅读 / 原始参考资料

MySQL 8.4 • 索引物理结构

02. 聚簇索引、二级索引与回表有什么关系？

索引与存储 / 基础

InnoDB 聚簇索引叶子保存行数据，二级索引记录包含索引列及主键。通过二级索引找到主键后再定位整行称为回表，覆盖索引可以在适用时减少这一步。

回答要点

1. 有显式主键时采用主键作为聚簇索引；没有主键时优先选所有列非空的第一个唯一索引；仍没有合适索引时，InnoDB 生成隐藏递增行标识。因此“没有主键就没有聚簇索引”错误。
2. 二级索引使用主键定位聚簇记录，而非保存一个永久的物理文件地址。主键越宽，各二级索引附带的主键空间通常越大，所以设计主键要兼顾稳定性、长度与插入访问模式。
3. 自增主键通常让插入更集中，减少随机插入造成的页面分裂，但也可能形成写入热点。随机 UUID 作为主键会增大键宽并分散写入，是否可接受应由真实规模与业务需求判断。

常见追问

追问：SELECT id WHERE email=? 会回表吗？若 email 二级索引已包含所需的 id，通常可以覆盖查询。覆盖描述的是所需列可由索引提供，但 MVCC 可见性判定等情况仍可能需要访问聚簇记录。

易错点

“聚簇”描述数据与索引组织方式，不代表表文件按主键连续无缝排列。也不能把任何数据库的二级索引都解释为携带主键，这是 InnoDB 的具体实现。

延伸阅读 / 原始参考资料

MySQL 8.4 • 聚簇与二级索引

MySQL 8.4 • 多版本机制

03. 联合索引为什么有最左前缀原则？

索引与存储 / 基础

联合索引按列顺序进行字典序排列。索引 (a, b, c) 可直接利用 a、(a, b)、(a, b, c) 的连续前缀定位；列顺序应围绕常用过滤、范围及排序需求设计。

回答要点

1. 先按 a 排序，a 相同时按 b，再按 c 排序。没有 a 条件时，b 的值分散在不同 a 区间内，不能把 b 条件一般化为一个连续索引范围，所以常规定位需要前导列约束。
2. 等值条件通常能继续利用后续列，范围条件会限制后续列进一步缩小扫描区间的能力。但后续列仍可能用于索引条件过滤、覆盖和排序判断，不能说遇到范围后就完全失效。
3. WHERE 条件书写顺序通常不决定索引使用顺序，优化器会分析谓词。MySQL 还有索引跳跃扫描等优化，因此缺少最左列并不等于绝对不能访问该索引，需要查看执行计划。

常见追问

追问：(status, created_at, id) 怎么选？若主要查询某状态按时间翻页，该顺序可将状态等值、时间范围与稳定排序结合。若主要按时间跨状态查询，前导 status 反而不适合，应基于查询占比调整或增设索引。

易错点

不要把“选择性最高的列永远放第一”当唯一规则。相等条件、范围条件、排序、覆盖、写入成本与查询组合都需要一起考虑。

延伸阅读 / 原始参考资料

MySQL 8.4 • 联合索引

MySQL 8.4 • 范围访问

04. 覆盖索引与索引条件下推有什么区别？

索引与存储 / 进阶

覆盖索引让查询所需列由索引提供，减少取整行；索引条件下推则让存储引擎先用索引中的列过滤，再决定是否读取整行。二者解决的阶段不同，可共同降低访问成本。

回答要点

1. 例如二级索引 (city, age) 可提供查询 city、age 和主键所需的值。如果还要读取 name 且它不在索引中，就仍需要取得整行，覆盖应结合 SELECT 投影与过滤条件具体判断。
2. ICP 把可由索引列检查的部分条件交给存储引擎执行。它不一定减少索引记录扫描数量，但能在获取完整行之前丢弃不满足条件的记录，从而降低随机读取聚簇页的成本。
3. 计划中的 Using index 通常提示覆盖访问，Using index condition 提示索引条件下推。它们不是同义词，也不等于查询已经最优，还应结合实际扫描行数、返回量与执行时间。

常见追问

追问：为什么 SELECT * 不利于优化？它增加传输和取行需求，让小而精确的覆盖索引难以成立；表结构变化还会改变查询输出。应列出业务真正需要的列，再判断是否值得加入索引，避免为了覆盖而堆宽索引。

易错点

不要认为 ICP 直接把二级索引变成覆盖索引。ICP 只是提前过滤，符合条件且缺少投影列的记录仍要取整行。

延伸阅读 / 原始参考资料

MySQL 8.4 • 索引条件下推

MySQL 8.4 • 执行计划字段

05. 哪些 SQL 写法可能降低索引利用率？

索引与存储 / 基础

索引能否有效使用取决于谓词可定位性、类型匹配、索引列序以及成本估计。对列做变换、前置通配符或返回大量行，常使普通索引无法高效定位，但不能一概称为索引失效。

回答要点

1. 对普通索引列套函数或表达式，例如 `DATE(created_at)=日期`，通常不如半开时间范围容易建立范围访问。若确实需要表达式查询，可评估生成列或函数索引，并检查版本限制。
2. 字符串列与数字比较等隐式类型转换，可能改变比较语义和访问路径；不同字符集或排序规则也会影响连接。应让参数类型与列定义一致，避免把类型问题误认为数据量太大。
3. `LIKE 'abc%'` 在合适条件下可构建索引范围，`LIKE '%abc'` 通常不能使用普通 B 树快速定位。即便可以访问索引，优化器也可能因返回比例高而认为表扫描成本更低。

```
-- 避免在普通索引列上套 DATE()
WHERE created_at >= '2026-10-09'
AND created_at < '2026-10-10'
```

常见追问

追问：OR 一定不用索引吗？不一定。优化器可能使用 Index Merge 或其他访问方式。应分别观察各分支的索引、选择性和合并成本，必要时比较等价的 UNION 写法，保证去重与 NULL 语义一致。

易错点

不要用“索引失效清单”代替验证。全索引扫描也属于使用索引，但未必高效；索引统计信息、数据分布和查询投影都可能影响最终计划。

延伸阅读 / 原始参考资料

MySQL 8.4 • 范围访问

MySQL 8.4 • EXPLAIN

06. ACID 分别由什么机制保证？

事务与并发 / 基础

ACID 是原子性、一致性、隔离性和持久性。InnoDB 通过撤销、日志、并发控制与恢复机制提供事务能力；业务一致性还依赖约束和正确的事务逻辑，不能只靠开启事务获得。

回答要点

1. 原子性要求事务修改整体提交或撤销，undo 支持回滚；持久性依赖 redo、提交刷盘策略及可靠存储。改变刷盘参数可能提高吞吐，但也改变进程故障、系统故障下的数据丢失窗口。
2. 隔离性由隔离级别、MVCC 与锁共同决定，不是所有事务都完全串行。数据库允许的并发异常随级别变化，应用需要根据业务约束选择锁定读、原子更新或更强隔离策略。
3. 一致性可以用“转账前后余额总和保持、库存不能为负”解释。数据库维护内部结构与声明的约束，应用则必须把扣款和入账纳入同一事务，并检查条件更新结果与失败处理。

常见追问

追问：提交成功就绝不丢数据吗？要结合日志刷盘参数、操作系统与存储设备保证讨论。严格持久性通常需提交时同步 redo 与 binlog，设备还必须真实兑现持久写入；跨机器故障切换另有复制一致性问题。

易错点

事务不会自动修复错误业务逻辑。例如先读取库存再无条件写入，仍可能丢失更新。先说明具体不变量，再选择原子条件更新或锁定策略。

延伸阅读 / 原始参考资料

MySQL 8.4 • ACID 模型

MySQL 8.4 • 撤销日志

07. MVCC 与 Read View 如何实现快照读？

事务与并发 / 进阶

MVCC 通过行版本与 undo 历史，在不阻塞写入的情况下提供一致性视图。Read View 根据事务可见性判断版本，当前版本不可见时沿历史版本查找，而不是复制整张数据库。

回答要点

1. 聚簇记录保存最近修改事务信息和回滚指针。更新产生可用于重建旧版本的 undo，读取时根据视图选择可见版本；自己的修改通常可见，所以快照并非对所有变化都一概隐藏。
2. RC 每次一致性读建立新的快照，RR 通常复用事务内第一次一致性读建立的快照。开始事务本身不等于普通情况下已经创建快照，WITH CONSISTENT SNAPSHOT 等语句要单独讨论。
3. 长时间运行的事务可能让旧版本持续被需要，阻碍 purge 清理并增大 undo 历史与空间压力。排查时要看活跃事务及历史积压，而不是只看当前 SQL 是否还在执行。

常见追问

追问：MVCC 能解决写写冲突吗？不能。快照读取减少读写互相阻塞，但多个事务修改同一行仍需锁协调。锁定读和更新使用与普通快照不同的读取语义，不能用某个旧快照直接推断更新会作用于哪些行。

易错点

不要说 Read View 是“当前最大事务号”一个数字。它包含活跃事务等可见性边界；也不要吧 MVCC 等同于所有 SELECT 都不加锁。

延伸阅读 / 原始参考资料

MySQL 8.4 • 多版本机制

MySQL 8.4 • 一致性非锁定读

08. RC 与 RR 的关键区别是什么？

事务与并发 / 基础

InnoDB 默认 RR。RC 的一致性读逐语句刷新快照，RR 通常在同一事务复用首次快照；锁定范围和间隙锁行为也不同，因此选择隔离级别同时涉及可见性、并发与业务约束。

回答要点

1. RC 可以看到前一条语句之后其他事务已提交的修改，所以同一行重复读取可能变化。RR 普通一致性读稳定于事务快照，但自己的写入可见，混合快照读与当前读需要谨慎解释。
2. RR 对许多范围锁定操作使用 next-key 锁来限制区间插入。RC 对一般搜索和索引扫描关闭间隙锁，但外键约束检查和重复键检查仍可能使用，不能说 RC 完全没有间隙锁。
3. 更高隔离级别可能增加等待、降低并发，但也简化某些约束推理。选择 RC 不能免除事务、唯一约束或库存校验；选择 RR 也不能自动防止跨多行的所有业务并发异常。

常见追问

追问：降低到 RC 能解决死锁吗？它可能减少范围锁竞争，但记录锁、外键检查和不同更新顺序仍可构成死锁。应先分析锁依赖、缩短事务并统一访问顺序，再根据业务允许的异常决定是否调整隔离级别。

易错点

不要只背“四级隔离表”回答 InnoDB 的具体行为。SQL 标准描述允许的异常，存储引擎实现还涉及快照、锁定读和索引扫描范围。

延伸阅读 / 原始参考资料

MySQL 8.4 • 事务隔离级别

09. RR 是否彻底解决幻读？

事务与并发 / 进阶

要区分读取方式：RR 普通快照读用稳定视图避免看到其他事务后来插入的行；范围锁定读可用 next-key 阻止相关插入。混合快照读、当前读及自身更新，不能简单断言所有场景结果永远不变。

回答要点

1. 如果事务反复执行普通 SELECT，且未改变自身数据，第一次快照之后其他事务提交的新行通常不可见。这是可见性控制，并不是把查询条件对应的所有区间都加锁禁止插入。
2. SELECT ... FOR UPDATE 和 UPDATE 等需要读取并锁定当前状态。RR 范围搜索通过记录和间隙锁约束相关区间，能阻止其他事务插入满足锁定范围的新记录，锁范围由访问路径决定。

3. 先快照查询为空，随后当前读可能看到其他事务已提交的新记录；事务若更新这些新记录，自己的修改又会对后续快照查询可见。因此分析必须明确语句顺序、读类型和自身写入。

常见追问

追问：先查询不存在再插入如何防重复？优先建立正确的唯一约束，并处理重复键错误。若还需控制某范围的业务规则，可在合适索引上使用锁定读或设计约束表，不能依赖一次普通 SELECT 的空结果保证无人插入。

易错点

不要把“RR 的快照读没有新行”说成“RR 下其他事务不能插入”。快照控制能看见什么，范围锁控制能修改什么，是两种机制。

延伸阅读 / 原始参考资料

MySQL 8.4 • 幻影行与 next-key

MySQL 8.4 • 锁定读

10. 记录锁、间隙锁与 next-key 锁怎么区分？

事务与并发 / 进阶

记录锁锁索引记录，间隙锁限制区间插入，next-key 锁同时覆盖记录及其之前的间隙。InnoDB 行锁建立在索引上，实际范围取决于隔离级别、谓词、索引唯一性与执行路径。

回答要点

1. 记录锁针对索引中的某条记录；即使表没有显式索引，InnoDB 也会借助隐藏聚簇索引实现。共享锁与排他锁的兼容性不同，锁定读和普通一致性读也不能混为一种访问。
2. 间隙锁主要阻止往间隙插入，多个事务的间隙锁可以共存。next-key 把记录锁与前方间隙锁合在一起，范围搜索可能锁到边界附近，不能只按 WHERE 最终返回的行数估计锁数量。
3. 使用完整唯一键等值匹配到已存在的唯一记录时，通常只需记录锁；未命中、非唯一搜索或只使用复合唯一键部分列时不能套用该结论。缺少合适索引会扩大扫描和锁定影响范围。

常见追问

追问：意向锁有什么作用？事务在持有行共享或排他锁前，先声明对应的表级意向锁，帮助表锁与行锁兼容性检查。意向锁本身通常不是把整张表的每一行锁住，它与记录锁是不同层级。

易错点

不要说“没用索引就自动升级成表锁”。通常是扫描并锁定大量索引记录造成近似全表阻塞，而非一定发生数据库意义上的锁升级。

延伸阅读 / 原始参考资料

MySQL 8.4 • InnoDB 锁

11. 死锁如何发生、排查与处理？

事务与并发 / 进阶

死锁是多个事务形成循环等待。InnoDB 通常检测并回滚一个事务打破循环，应用必须能重试；预防重点是统一资源访问顺序、缩短事务并降低锁定范围，超时与死锁要分开分析。

回答要点

1. 例如事务甲先锁行 A 再等 B，事务乙先锁 B 再等 A，就构成循环。锁的数量少也会死锁，行锁、间隙、外键和不同索引访问组合都可能参与，不只有大批量更新才发生。
2. 通过 `SHOW ENGINE INNODB STATUS` 查看最近死锁，也可开启全部死锁日志并结合锁等待信息。把 SQL、索引、隔离级别、锁模式及事务内先前语句连起来，重建实际等待关系。
3. 让批量事务按稳定主键顺序更新，事务中避免远程调用和用户等待，并给条件加合适索引。重试整个事务应有次数上限和退避，外部副作用要幂等，不能只机械重发最后一条 SQL。

```
SHOW ENGINE INNODB STATUS;
```

常见追问

追问：锁等待超时会回滚整个事务吗？默认通常只回滚超时语句，死锁受害者则回滚整个事务；`innodb_rollback_on_timeout` 等配置会改变超时行为。应用应明确失败后的事务状态并执行回滚或整体重试。

易错点

死锁检测关闭后可能依赖等待超时解开阻塞，这不等于死锁消失。不要用无限重试掩盖持续的锁顺序问题，也不要把所有等待都判定为死锁。

延伸阅读 / 原始参考资料

MySQL 8.4 • 死锁

MySQL 8.4 • 错误处理

12. redo 与 undo 分别解决什么问题？

日志与恢复 / 基础

redo 记录支持崩溃恢复所需的修改信息，让已提交变化可重放；undo 保存撤销和旧版本信息，用于事务回滚与一致性读。二者职责不同，修改 undo 页本身也需要受到恢复保护。

回答要点

1. 数据库先把修改相关的 redo 按日志规则持久化，再逐步刷新脏数据页，通常称为先写日志。顺序日志写入比每次提交同步所有随机数据页更高效，重启可从检查点之后恢复必要变化。

2. undo 用于恢复修改前的逻辑状态，更新形成的历史还服务于 MVCC。事务提交后不代表所有 undo 立即删除，仍被旧快照需要的版本必须保留，最终由 purge 等后台机制清理。
3. redo 具有有限容量并不断推进检查点，日志空间压力可能迫使脏页刷新；undo 生命周期与事务和快照有关。解释两类日志时，应同时说明恢复、回滚和空间回收三个维度。

常见追问

追问：有 redo 还要 doublewrite 吗？redo 记录变化，但损坏或部分写入的数据页可能无法作为可靠恢复基础。doublewrite 保护页面写入完整性，帮助处理 torn page，与 redo 的逻辑职责不同；具体配置受设备原子写能力影响。

易错点

不要把 redo 称为“SQL 原文日志”，也不要说 undo 只在失败时有用。提交成功后，其他事务的快照读仍可能依赖历史版本。

延伸阅读 / 原始参考资料

MySQL 8.4 • 重做日志

MySQL 8.4 • 撤销日志

MySQL 8.4 • 双写缓冲

13. binlog 与 redo 如何通过两阶段提交协调？

日志与恢复 / 进阶

binlog 属于 MySQL 服务层，支持复制与时间点恢复；redo 属于 InnoDB，支持引擎崩溃恢复。提交协调让引擎状态与 binlog 事务记录一致，通常用 prepare、写 binlog、commit 来概括。

回答要点

1. row 格式记录行变化，statement 格式记录语句，mixed 按规则选择。复制和恢复依赖 binlog，但它不能替代 InnoDB redo 对本地数据页的恢复职责，日志格式也会影响体积与可重现性。
2. 引擎先进入准备状态，随后 binlog 提交记录持久化，再完成引擎提交。恢复时检查准备事务与有效 binlog 的事务标识，决定完成提交还是回滚，避免两份日志对同一事务意见相反。
3. 概念上的三个步骤并不意味着每笔事务都独自刷盘三次，组提交可合并同步。严格讨论持久性时，需要结合 sync_binlog 与 innodb_flush_log_at_trx_commit，并考虑操作系统和硬件故障。

常见追问

追问：只开启 binlog 就能恢复到任意时间吗？需要一份可靠的一致性全量备份，以及从该备份起连续可用的 binlog。恢复先装载备份，再重放到目标事务边界；日志被清理或时间范围有缺口就无法任意回放。

易错点

不要把内部提交协调等同于应用显式发出的分布式 XA 全流程。面试可以用两阶段图解释核心，但要承认组提交与恢复规则使实现比背诵的三行顺序更复杂。

延伸阅读 / 原始参考资料

MySQL 8.4 • 二进制日志

MySQL 8.4 • 内部与外部 XA

14. 如何读懂 EXPLAIN 和 EXPLAIN ANALYZE?

查询优化 / 基础

EXPLAIN 展示优化器选择的访问方式和估计；EXPLAIN ANALYZE 实际执行可分析语句并提供耗时、行数和循环。应比较估计与实际，而不是只看是否出现某个索引或某个 Extra 标签。

回答要点

1. 观察 join 顺序、访问 type、possible_keys 与实际 key、估计 rows 和 filtered。rows 是估计扫描量，并非结果条数；多层循环需要结合各层循环次数判断总工作量，不能孤立比较单个数字。
2. Using filesort 表示未直接用索引顺序完成排序，不等于必然写磁盘；Using temporary 也需要看数据量和实现。小表扫描可能合理，索引随机回表太多时甚至比顺序扫描更贵。
3. EXPLAIN ANALYZE 对照实际耗时、返回行和 loops，有助发现统计偏差与重复访问。因为会实际执行并消耗资源，应在可控环境或合适负载时分析，不能当成零成本的静态展示。

```
EXPLAIN FORMAT=TREE
SELECT id FROM orders WHERE user_id=42;

EXPLAIN ANALYZE
SELECT id FROM orders WHERE user_id=42;
```

常见追问

追问：计划选错索引怎么办？先核对参数类型、数据分布、统计信息和索引设计，可比较更新统计后的计划与真实耗时。强制索引可以做诊断对比，但把它长期写死可能在数据分布变化后阻止优化器选择更好路径。

易错点

type 的传统顺序是提示而非性能总排名。一次 const 查询也可能被上层重复百万次；一次 ALL 扫描小表则可能非常快，必须看整体工作量。

延伸阅读 / 原始参考资料

MySQL 8.4 • EXPLAIN

MySQL 8.4 • 计划字段

15. JOIN 的语义与实现有哪些高频问题？

查询优化 / 基础

INNER JOIN 保留匹配组合，LEFT JOIN 还保留左侧未匹配行并补 NULL。物理执行可使用嵌套循环、索引查找或哈希连接，MySQL 8.4 的实现不能照搬旧版本只有嵌套循环的结论。

回答要点

1. 对 LEFT JOIN 的右表列在 WHERE 中施加排除 NULL 的条件，常使未匹配行被过滤，结果等价于某些内连接。若想保留全部左行，应明确条件应放 ON 还是 WHERE，并验证业务语义。
2. 索引嵌套循环利用外侧结果逐次查找内侧索引，适合外侧筛选后较少的情况。哈希连接可先构建哈希表再匹配另一侧，内存不足可能使用临时文件，具体计划取决于代价估计。
3. 连接键数据类型和排序规则应一致，先减少可安全下推的输入数据，并避免无意的一对多结果膨胀。常说小表驱动大表，更准确应指过滤后的有效输入及访问成本，而非原始总行数。

常见追问

追问：IN、EXISTS、JOIN 哪个更快？没有固定赢家。优化器可能把子查询转换为半连接等同类执行路径。先保证重复行和 NULL 语义等价，再对真实参数与数据查看计划及实际耗时，不要只按语法名称决定性能。

易错点

不要认为 LEFT JOIN 一定比 INNER JOIN 慢，也不要为了性能随意把外连接改为内连接。两者结果语义不同，优化首先必须保持正确结果。

延伸阅读 / 原始参考资料

MySQL 8.4 • 哈希连接

MySQL 8.4 • 嵌套循环连接

MySQL 8.4 • 外连接简化

16. 慢 SQL 的排查步骤是什么？

查询优化 / 进阶

先确认慢发生在执行、锁等待、网络还是连接队列，再定位高耗时或高频 SQL，检查计划与实际扫描量。优化应围绕最大成本，并在同样负载与参数下比较，不能只加索引或盲目加机器。

回答要点

1. 结合慢查询日志、Performance Schema 和应用链路，检查总耗时、锁等待、扫描与返回行数。单次快但频率极高的查询也可能消耗主要资源，只筛最长的一条会遗漏整体吞吐瓶颈。
2. 核对是否扫描过多、排序聚合过大、重复回表、连接结果膨胀或存在热点锁。优先缩小查询范围、只取必要列和调整索引，再考虑缓冲池、缓存或架构变化，避免优化不在瓶颈上的环节。

3. 评估 P95/P99 延迟、吞吐及写入代价，使用有代表性的参数和冷热缓存。线上加索引还要评估磁盘、DDL 并发和部署窗口，改写 SQL 必须检查重复、NULL、排序与事务语义。

常见追问

追问：为何缓存命中后仍慢？可能在锁等待、CPU 排序、扫描大量缓存页或应用传输解析上耗时。内存命中只减少磁盘读取，不能消除查询计算与竞争，应先用执行统计确认具体耗时位置。

易错点

慢日志的 Query_time 与端到端响应时间并不相同，也不是所有日志字段都包含完整等待。不要用一次测试的最快耗时证明优化有效。

延伸阅读 / 原始参考资料

MySQL 8.4 • 慢查询日志

MySQL 8.4 • 缓冲池

17. 深分页为什么慢，游标分页怎么做？

查询优化 / 进阶

LIMIT offset,size 往往要访问并丢弃前 offset 条，页数越深成本越高。游标分页以最后一条排序键作为下一页边界，用索引直接继续扫描，适合顺序浏览，但不提供任意页码跳转。

回答要点

1. 即使排序用到了索引，较大的 OFFSET 仍可能需要扫描大量记录，若伴随回表成本更高。可以先用覆盖索引拿目标主键再取行，但它降低取行成本，不会消除遍历深层偏移的本质。
2. 按 created_at,id 的稳定唯一顺序分页时，游标记录上一页末尾的两个值，下一页使用字典序边界并同样排序。id 用于解决时间相同的并列，否则会漏读或重复，方向也必须一致。
3. 并发插入或更新排序键可能影响跨页结果。若需要导出一致的完整集合，应考虑事务快照、固定上界或版本标识；普通无限滚动与严格审计导出不能使用同一种一致性假设。

```
SELECT id, created_at
FROM orders
WHERE user_id = 42
  AND (created_at > ? OR (created_at = ? AND id > ?))
ORDER BY created_at, id
LIMIT 20;
-- 对应索引: (user_id, created_at, id)
```

常见追问

追问：游标分页能直接跳第 1000 页吗？通常不能，需要已有边界或预先维护定位信息。它把按页码随机访问换成高效顺序推进；产品若必须跳页，可限制最大页数、另建检索接口或接受偏移查询成本。

易错点

不要只写 `WHERE id > last_id` 却继续按另一列排序。游标条件、索引顺序和 `ORDER BY` 必须协调，同时把唯一的并列键纳入边界。

延伸阅读 / 原始参考资料

MySQL 8.4 • LIMIT 优化

MySQL 8.4 • 排序优化

18. 主从复制与读写分离有哪些一致性问题？

复制与建模 / 进阶

常规 MySQL 复制由主库 binlog 向副本传送并应用，默认异步，因此副本可能落后。读写分离增加读吞吐，却可能让用户写后立即读到旧值；半同步确认也不等于副本已应用完成。

回答要点

1. 副本接收主库日志形成 relay log，再由应用线程执行，网络接收与 SQL 应用是不同阶段。大事务、慢查询争用、磁盘或网络压力都会增加延迟，判断一致性需要知道事务是否已经应用。
2. 需要写后读一致时，可短时间或按会话路由到主库，也可依据 GTID 等事务位置等待副本追上并设置超时回退。固定睡眠只能降低概率，无法在负载和故障变化下给出可靠证明。
3. 半同步让主库等待副本接收并持久记录所需事件的确认，降低特定故障下的丢失风险，但超时可回退异步且确认不等于副本完成应用。故障切换仍需选主、日志完整性与旧主隔离策略。

常见追问

追问：副本越多写入越快吗？不会。写入通常仍由主库承担，更多副本还带来日志传输和管理成本。副本适合扩展容忍一定延迟的读；写热点和容量瓶颈可能需要优化事务、分区或更复杂的数据分片方案。

易错点

不要把“复制线程在线”当成“数据实时一致”。Seconds_Behind_Source 等延迟指标也不是所有故障下的完美证明，应结合日志位置、应用状态与业务一致性要求。

延伸阅读 / 原始参考资料

MySQL 8.4 • 半同步复制

MySQL 8.4 • GTID 复制

19. 范式、反范式与表设计如何取舍？

复制与建模 / 基础

范式通过约束函数依赖减少冗余和更新异常；反范式为读性能或历史语义有意保存重复数据。设计应先表达实体、键与约束，再根据查询成本决定冗余，并明确唯一事实来源和更新责任。

回答要点

1. 面试可先解释一范式避免重复组，二范式去除非主属性对候选键的部分依赖，三范式关注非主属性的传递依赖。分析依据是候选键与函数依赖，而非简单按列数或是否只有单列主键判断。
2. 订单项保存成交时价格和商品名称快照，通常属于历史事实，即使商品表以后变化也不应自动更新。把它误判为应消除的重复字段，会破坏历史语义，冗余是否错误必须结合业务含义。
3. 读多写少的聚合可维护汇总表以降低实时连接和统计成本，但需定义更新事务、幂等重放、补偿与校验。没有同步责任的重复字段会产生不一致，不能只为了少 JOIN 就复制所有数据。

常见追问

追问：为何不全部用 JSON 一列存储？JSON 适合结构灵活的局部属性，但核心筛选、连接、唯一约束与关系仍需要清晰建模。大量字段隐藏在文档中会增加索引、类型校验和数据治理成本，应按访问与约束需求选型。

易错点

不要认为三范式永远比反范式快，也不要把加一个代理 id 视为自动消除函数依赖问题。候选键及业务依赖仍然存在，模型应能够解释更新异常。

延伸阅读 / 原始参考资料

MySQL 8.4 • 数据大小与规范化

RPI CSCI 4380 • 数据库规范化课程

20. 如何防止 SQL 注入并管理数据库连接？

复制与建模 / 基础

SQL 注入的根源是把不可信输入拼入 SQL 语法。使用参数化查询绑定值，动态表名或排序列采用白名单；再配合最小权限、正确连接池与超时控制，既保护数据也避免资源耗尽。

回答要点

1. 参数绑定让用户输入作为数据而非语法解释，适用于值参数。表名、列名和排序方向通常不能用值占位符代替，应从预设选项映射为可信标识符，不能把它们直接拼接进查询。
2. 参数化之外，还需限制数据库账号权限、避免应用使用管理员账号，并检查日志不要输出密码或敏感数据。转义和输入格式校验可作为辅助，但不能替代驱动支持的参数绑定。
3. 连接池复用会话以减少建立连接成本，但必须归还连接、回滚未结束事务并重置会话状态。连接数过多会增加服务器压力，应同时设置连接等待、执行与事务期限并观测排队情况。

-- 值由驱动绑定，禁止字符串拼接

```
SELECT id, name FROM users WHERE email = ?;
```

常见追问

追问：ORM 就一定安全吗？普通参数化接口通常安全，但原生 SQL、字符串拼接及动态排序仍可能绕过保护。应检查最终调用是否真正绑定值，动态标识符是否使用白名单，事务异常后连接是否带着未提交状态返回池中。

易错点

不要把正则过滤引号当完整防护。输入有多种编码和语法边界，正确的查询构造、参数绑定及权限隔离比黑名单更可靠。

延伸阅读 / 原始参考资料

MySQL 8.4 • 预处理语句

MySQL 8.4 • PREPARE

Redis

缓存、持久化与分布式边界

以 Redis Open Source 为范围，覆盖常用结构、原子操作、复制与 Cluster，以及缓存一致性和锁的真实保证边界；涉及新能力时注明版本差异。

01. Redis 常用数据类型分别适合什么场景？

数据类型与执行 / 基础

String 适合值缓存与计数，Hash 适合字段对象，List 适合顺序列表，Set 适合去重集合，Sorted Set 适合带分数排名。选型应围绕访问操作、规模和可靠性要求，不只记住五个名字。

回答要点

1. String 保存字节序列，可存文本、序列化对象或整数计数；Hash 把对象拆成字段便于局部读取和修改。大对象整体反序列化与多个小字段访问各有成本，应根据读取和更新粒度选择。
2. List 支持头尾操作，Set 支持成员判断及集合运算，Sorted Set 按分数排序且成员唯一。排行榜需要考虑同分排序、分页成本和分数更新，集合交并运算要注意参与集合规模。
3. Bitmap、HyperLogLog、GEO 与 Stream 等扩展了场景；版本和部署还影响 JSON、概率结构等能力。HyperLogLog 给出近似去重计数，Stream 支持消费状态，不能把所有结构都当精确集合使用。

常见追问

追问：排行榜为什么不用 List？List 保存位置，但插入或更新某用户排名需要寻找和移动逻辑；Sorted Set 可以按成员更新分数并按分数取区间。最终复杂度还取决于命令和返回数量，不能笼统说所有 ZSet 操作都是常数时间。

易错点

类型是对外语义，底层编码是实现，两者不可混为一谈。用 List 做队列还要另行考虑取出后崩溃、确认、重试与消息保留。

延伸阅读 / 原始参考资料

[Redis 官方](#) • 数据类型

[Redis 官方](#) • 有序集合

02. Redis 的类型与底层编码有什么区别？

数据类型与执行 / 进阶

同一数据类型可按内容和规模采用不同编码。紧凑编码减少对象与指针开销，达到阈值后会转换；讨论 `listpack`、`intset` 或跳表时应注明版本，不能沿用旧 `ziplist` 结论描述所有实例。

回答要点

1. 小 Hash 和小 Sorted Set 在新版本常使用 `listpack` 等紧凑结构，节省大量独立分配与指针空间。阈值受元素数及值大小配置影响，具体编码可通过 `OBJECT_ENCODING` 观察而非猜测。
2. 整数集合、哈希表或有序集合的字典与跳表服务不同访问需求。紧凑结构有时需要扫描或移动数据，规模扩大后会转换，优化的本质是在小规模内存收益和操作成本之间平衡。
3. Redis 7 前后的压缩列表演进、Redis 7.2 的集合紧凑编码变化都说明实现有版本边界。不要为省内存无限调大紧凑阈值，编码转换和线性操作可能扩大尾延迟，应做代表性负载测试。

常见追问

追问：为什么一万个小 key 比一个 Hash 更占内存？独立 key 需要重复字典条目、对象及分配元数据，聚合可摊薄部分开销。不过也会让过期、分片和大对象操作粒度变粗，不能只看省下的字节就无条件合并。

易错点

不要把 `ziplist` 当当前所有集合的统一底层实现。也不要认为一旦采用紧凑编码就必然更快，它主要优化小对象空间，访问复杂度仍需看具体命令。

延伸阅读 / 原始参考资料

Redis 官方 • 内存优化

Redis 官方 • 编码观察

03. Redis 是单线程吗，为什么能很快？

数据类型与执行 / 基础

Redis 核心命令通常在主执行线程按序处理，但进程还包含后台任务和可配置的 I/O 线程。内存访问、高效结构和事件驱动降低开销；慢命令仍会阻塞其他请求，因此“全部只有一个线程”不准确。

回答要点

1. 大量工作是短小的内存结构操作，事件循环复用网络连接，减少频繁线程切换和锁竞争。性能还依赖命令复杂度、值大小、网络往返与硬件，不能脱离负载承诺固定每秒请求数。
2. 后台线程可承担 AOF 同步、异步释放等工作，部分版本支持网络 I/O 线程；持久化重写或快照还可能使用子进程。I/O 并行并不意味着所有修改数据的命令都在多个线程并行执行。
3. 单个长脚本、巨大集合运算或大返回值会占用主执行路径，让其他请求排队。扩展可通过减少单次工作、批量网络传输以及多实例分片实现，单纯增加客户端并发并不会消除热点。

常见追问

追问：开启 I/O 线程能解决慢 Lua 吗？不能。I/O 线程主要改善网络读写相关成本，耗时脚本仍占用数据执行路径。应缩短脚本、限制处理数量或改造数据模型，同时用延迟和命令统计判断瓶颈是否真在网络。

易错点

不能用“单线程所以不可能并发问题”推导应用安全。多个请求仍会交错，客户端多命令流程、持久化和故障切换也都有独立一致性边界。

延伸阅读 / 原始参考资料

Redis 官方 • 延迟诊断

Redis 官方 • 线程与运行信息

04. 单条命令原子是否意味着业务操作原子？

数据类型与执行 / 基础

INCR 等命令的读改写在 Redis 内部完成，其他请求不会插在该命令中间；客户端分开 GET、计算、SET 则会发生交错。原子执行只描述可观察的命令边界，不自动包含持久化或跨系统业务成功。

回答要点

1. 两个客户端先读取同一个计数，再各自写入加一结果会丢失一次更新。用 INCR 可把读取和更新合成服务端操作，但它要求值可按整数解释，数值范围和错误返回仍要正确处理。
2. INCR 后再单独 EXPIRE 存在客户端崩溃后未设置 TTL 的窗口，且反复刷新 TTL 可能改变限流语义。应在短 Lua、事务或适合的命令组合中统一定义计数及过期规则。
3. 超时只说明客户端没拿到确定结果，命令可能已经执行。盲目重试计数、扣减会重复生效；若要求业务幂等，需要请求标识与去重状态，或让权威数据库约束最终结果。

```
INCR page:views
SET cache:user:42 '{"name":"Lin"}' EX 300
```

常见追问

追问：Redis 原子扣库存就能保证不超卖吗？它可在单实例内原子检查并扣减，但预热库存、重复请求、故障丢写和数据库落账仍可能破坏业务约束。需定义幂等、补偿与最终库存的权威来源，不能仅靠一条命令完成全链路保证。

易错点

原子性、隔离性和持久性不是一个概念。命令执行期间没有交错，不等于断电不会丢失，也不等于多个 Redis 节点或数据库修改同时提交。

延伸阅读 / 原始参考资料

Redis 官方 • 原子计数

05. MULTI、EXEC、WATCH 能提供什么保证？

数据类型与执行 / 进阶

MULTI 把命令排队，EXEC 按序连续执行；WATCH 检查关注的键是否在提交前发生变化，提供乐观并发控制。Redis 事务不会像关系数据库那样自动回滚已成功的命令，必须检查每个返回结果。

回答要点

1. 排队期间命令返回 QUEUED，通常尚未执行；EXEC 执行时其他客户端命令不会插入该组执行。语法或排队错误可导致整组拒绝，运行时的类型错误则可能仅影响对应命令，其余继续执行。
 2. WATCH 在读取前监视键，再由客户端计算并在 MULTI/EXEC 中提交。监视状态变化会使 EXEC 失败，需要重新读取和有限重试；连接专属状态意味着使用连接池时要保证操作落在同一连接。
 3. 事务内无法把尚未执行的查询结果取回后继续客户端分支判断，因此复杂读改写常更适合短 Lua。
- Cluster 中相关多 key 通常需同槽，事务的连续执行保证也不提供跨槽两阶段提交。

```
WATCH balance
GET balance
MULTI
SET balance 90
EXEC
-- EXEC 若因监视键变化而失败，重新读取并计算
```

常见追问

追问：WATCH 失败说明出错了么？通常说明乐观条件已被其他修改打破，是预期冲突。应重新读取状态再计算，而非只重发旧写入。高争用场景可能频繁失败，需要退避、原子服务端操作或调整热点设计。

易错点

不要说 MULTI 是“执行失败全部回滚”。运行时错误不会撤销之前已成功的命令；DISCARD 是放弃尚未执行的队列，也不是回滚已执行写入。

延伸阅读 / 原始参考资料

Redis 官方 • 事务

Redis 官方 • WATCH

06. Lua 脚本的原子性与风险是什么？

数据类型与执行 / 进阶

Lua 能把校验与修改放到服务端连续执行，减少网络交错与往返。但脚本执行期间会占用执行路径，运行错误也不自动回滚此前写入。应限制工作量，传入显式键并处理脚本缓存失效。

回答要点

1. 适合比较锁令牌后删除、库存条件扣减或限制窗口计数等短逻辑。每个脚本都应先检查参数与数据类型，再执行必要写入，避免后续错误留下部分修改，且不要用无界循环处理整个数据库。
2. 键名通过 KEYS 显式传入，其他值通过 ARGV 传入，不要在脚本中拼出未声明访问键。这样客户端和集群可理解涉及的键；Cluster 多键脚本通常要求这些键属于同一 hash slot。
3. EVALSHA 只发送脚本摘要可减少重复传输，但脚本缓存不是永久资产，重启、切换或缓存清理后可能出现 NOSCRIPT。客户端应准备重新加载或 EVAL 回退，并留意特定版本的脚本缓存行为。

```
-- KEYS[1] 为锁, ARGV[1] 为本次持有者令牌
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('DEL', KEYS[1])
end
return 0
```

常见追问

追问：脚本超时会自动停止并回滚吗？不会把已发生的写入自动撤销。长脚本会造成忙状态，能否用 SCRIPT KILL 安全结束取决于是否已经写入；更可靠的做法是事先限制规模，并避免把耗时计算塞进 Redis。

易错点

“原子”表示执行时其他请求不交错，不代表事务式失败回滚。脚本内已经执行的写入，在后续命令报错时仍可能保留。

延伸阅读 / 原始参考资料

Redis 官方 • Lua 脚本

07. RDB 快照如何工作，有什么代价？

持久化与内存 / 基础

RDB 保存某一时刻的数据快照，文件紧凑，适合备份与较快装载。后台快照通常通过 fork 子进程生成，但 fork 暂停、写时复制及磁盘 I/O 仍会影响服务；故障会丢失快照之后的变化。

回答要点

1. BGSAVE 通常创建子进程读取快照视图，主进程继续处理请求；父子共享页面直到写时复制发生。数据集大且写入多时，额外内存和 fork 时间可能明显，后台不等于完全零阻塞。
2. 快照适合定期备份、复制传输和可容忍一定丢失窗口的场景。保存周期越长，未持久化修改越多；保存频繁则增加 fork 与 I/O 成本，应按恢复点目标和负载选择而非机械套配置。
3. 备份文件应复制到独立故障域并验证可装载，还要监控最近快照状态、耗时与磁盘空间。只有本机一份 RDB 无法应对整机或存储损坏，启用持久化也不等于已经具备恢复流程。

常见追问

追问：SAVE 和 BGSAVE 有什么区别？SAVE 在当前执行路径同步生成快照，过程中阻塞正常请求；BGSAVE 通常使用子进程，减少长时间阻塞但仍有 fork 等代价。生产选择还要看内存余量、写入强度及是否存在其他持久化任务。

易错点

不要承诺“RDB 完全不影响延迟”。大数据集 fork 与写时复制会带来暂停和内存压力；快照恢复能恢复某一时刻，不能重现其后未保存的写入。

延伸阅读 / 原始参考资料

Redis 官方 • 后台快照

Redis 官方 • 持久化概览

08. AOF、重写与混合持久化有什么关系？

持久化与内存 / 进阶

AOF 记录用于重建数据的写入，刷盘策略决定性能与丢失窗口；重写压缩冗余历史并生成等价恢复内容。Redis 7 起使用多部件 AOF，基础文件可采用 RDB 前缀，恢复时再应用增量日志。

回答要点

1. appendfsync always 强化提交落盘但增加同步成本，everysec 常在性能与丢失窗口间折中，no 依赖操作系统同步。everysec 通常讨论约一秒未刷盘风险，不应把它当任何故障下绝对最多一秒的契约。
2. 重写根据当前状态生成较精简的恢复表示，不是机械复制原 AOF。多部件 AOF 使用基础文件、增量文件与 manifest 管理切换，后台写入期间仍需维护新的变化，关注内存、磁盘与失败重试。
3. 同时启用 RDB 与 AOF 时，恢复策略通常优先使用更完整的 AOF。混合持久化把基础快照的装载优势与后续增量结合，但仍需正确管理相关文件，备份不能漏掉 manifest 所引用的组成部分。

常见追问

追问：AOF 重写是不是越频繁越好？不是。它减少恢复文件体积和重放工作，但也带来 fork、额外磁盘及写时复制压力。应根据日志增长比例、重写耗时、恢复目标和内存余量设置阈值，并监控失败与持续增长。

易错点

不要把 Redis 7 的 AOF 仍讲成永远只有一个文件，也不要将写入页缓存当已经 fsync。日志追加、系统同步及备份保留是不同环节。

延伸阅读 / 原始参考资料

Redis 官方 • AOF 重写

Redis 官方 • 持久同步等待

09. 过期 key 如何删除，TTL 有哪些细节？

持久化与内存 / 基础

Redis 结合访问时的被动过期和周期性的主动抽样清理。TTL 到点表示键在语义上失效，不保证对应内存立即按毫秒释放；修改键是否保留过期时间取决于具体命令及选项。

回答要点

1. 访问已超时键时触发被动过期；主动清理会抽样检查带过期时间的键，以处理再也不被访问的键。主动任务要在回收和 CPU 之间平衡，大量键同刻到期可能形成清理与缓存回源压力。
2. SET 替换值通常会移除原 TTL，可使用 KEEPTTL 或显式 EX/PX 定义行为；HSET 等修改内容的命令一般会自动移除整个键的 TTL。应看命令语义，不能笼统说写入都会续期。
3. TTL 返回剩余秒数，PTTL 返回毫秒；无过期时间与键不存在使用不同的特殊返回值。Redis 以绝对时间处理过期信息，系统时钟异常、恢复与复制场景也需要结合实际配置理解。

常见追问

追问：能用过期通知做可靠定时任务吗？不应直接依赖。实际清理时间可能晚于 TTL，通知基于 Pub/Sub 且订阅断开会丢消息。可靠调度需要持久任务状态、轮询或队列、幂等执行与补偿机制，通知只能作为加速提示。

易错点

过期与内存淘汰不同：前者依据时间，后者依据容量和策略。也不要将键已不可读等同于其所有内存已经立即释放。

延伸阅读 / 原始参考资料

Redis 官方 • EXPIRE 与过期机制

Redis 官方 • 键空间通知

10. 内存淘汰策略如何选择？

持久化与内存 / 基础

达到 maxmemory 后 Redis 根据策略选择淘汰或拒绝部分写入。allkeys 面向全部键，volatile 仅面向带过期的键；LRU 关注近期使用，LFU 关注访问频率，具体实现采用近似方法而非完美排序。

回答要点

1. allkeys-lru 或 allkeys-lfu 常用于可重新构建的缓存；volatile 策略限定候选为设置过期的键，无合适候选时可能无法释放空间。不要把重要持久数据与可淘汰缓存混放后期待策略自动分辨。
2. noeviction 不主动淘汰，但达到限制后可能拒绝需要增加内存的写操作，读取仍可工作。调用方必须处理错误，否则缓存异常可能传播为业务失败；无限增大 maxmemory 会压缩系统与持久化余量。

3. maxmemory 的计算与系统实际内存占用不是完全同一个数字，复制及 AOF 缓冲等有特殊处理，fork 写时复制还需要额外空间。选择容量要给操作系统、碎片和后台任务保留余地，并监控淘汰率。

常见追问

追问：LRU 和 LFU 谁更好？近期热度变化快时 LRU 可能更适合，稳定的长期热点可能受益于 LFU。两者需用真实访问分布、命中率与尾延迟验证，不能只根据名称判断，LFU 也有计数近似和衰减行为。

易错点

不能认为设置 TTL 后键就只能因过期删除，淘汰策略可能提前删除它。反过来，持久化开启也不会阻止键按策略被淘汰并写入持久状态。

延伸阅读 / 原始参考资料

Redis 官方 • 内存淘汰

11. 主从复制、全量与增量同步如何工作？

复制与集群 / 进阶

Redis 默认异步复制，副本按主节点复制流更新。首次或无法补齐缺口时需全量同步，短暂断线在复制标识和 backlog 范围允许时可部分重同步。副本能扩读，但读取可能落后。

回答要点

1. 全量同步把当前数据基线传给副本，并在相应流程中追加期间变化；基线装载会消耗网络、磁盘或内存资源。无盘复制等配置改变传输路径，但不消除恢复数据集与传送增量的成本。
2. 部分同步依据复制标识和偏移，请求缺失的历史复制流。backlog 是有限缓冲，断线过久或写入速率太高可能超出保留范围，就需要全量同步，所以应结合峰值写入和允许断线时间设置容量。
3. 主库向客户端确认时，副本未必已经收到并应用写入。故障提升落后副本可能丢失已确认的修改；读副本可出现旧值，要求写后读一致的操作应选择主节点或明确等待与回退策略。

常见追问

追问：WAIT 可以保证强一致吗？它等待当前连接之前的写入达到指定副本确认数量，必须检查实际返回数量。它提高特定故障下的保留概率，但不是共识协议，也不保证所有切换情形都不丢写，不能将 Redis 变成强一致数据库。

易错点

不要把 replication backlog 与客户端输出缓冲混为一物。复制标识、偏移和保留窗口共同决定能否部分同步，连接重建成功不等于数据已经追平。

延伸阅读 / 原始参考资料

Redis 官方 • 复制

Redis 官方 • 副本确认等待

12. Sentinel 解决什么问题，quorum 是什么？

复制与集群 / 进阶

Sentinel 监控非分片主从部署、判断故障、协调提升副本并通知客户端。quorum 用于客观故障判断，真正执行故障切换还需要多数 Sentinel 授权；它提供高可用管理，不消除异步复制的丢写窗口。

回答要点

1. 单个 Sentinel 的超时判断属于主观下线，多个监控者达到配置 quorum 才能形成客观下线。故障切换还涉及领导者选择和多数授权，因此把 quorum 简单设置为一并不能绕过多数机制。
2. 通常至少部署三个 Sentinel 并分散故障域，客户端通过 Sentinel 感知新主并重连。只在同一机器放多个进程，机器故障时仍一起消失，数量不能代替独立的网络、电源与主机容错。
3. 网络分区可能使旧主一侧继续接收写入，恢复后这些写入可能丢失。min-replicas-to-write 等配置能限制一部分风险，但会降低副本不足时的可用性，不能作为所有分区情形的完整保证。

常见追问

追问：Sentinel 会转发业务请求吗？通常不会，它负责监控与发现，客户端仍直接连接当前 Redis 主节点。客户端必须支持发现、重连、超时与未知执行结果处理，否则服务器切换完成也可能无法恢复应用访问。

易错点

Sentinel 不是数据分片方案，也不是持久化替代物。故障恢复服务可用与保证每一笔已确认写入永久存在，是不同目标。

延伸阅读 / 原始参考资料

Redis 官方 • Sentinel

13. Cluster 的 16384 个槽与 hash tag 怎么用？

复制与集群 / 进阶

Redis Cluster 把键空间划为 16384 个 hash slot，再把槽分配给主节点。CRC16 与 hash tag 决定键所在槽，客户端负责识别重定向；多键原子操作通常要求同槽，分片也不会自动拆开一个热点键。

回答要点

1. 通常按 CRC16(key) 对 16384 取模计算槽。符合规则的非空花括号内容可作为 hash tag，例如 order:{42} 与 lock:{42} 会落到相同槽，有助协调相关键，但大范围共用标签会集中负载。
2. MOVED 表示客户端应更新该槽的目标节点认知；迁移期间 ASK 要按临时重定向流程处理，不能简单永久改写全部映射。使用支持 Cluster 的客户端，可处理拓扑变化并减少不必要重试。

3. 每个主节点负责一部分槽并可带副本，高可用仍受多数、槽覆盖和配置影响。Cluster 复制默认异步，故障切换有写入丢失窗口；跨槽操作不具备自动的全局事务，需要业务重新设计。

常见追问

追问：增加节点为什么不能解决单 key 热点？一个键仍由它所在槽的主节点处理，迁移槽只是转移瓶颈。可考虑本地缓存、读取副本或可合并的业务拆分，但拆键会改变原子聚合和一致性，需要按操作特征设计。

易错点

16384 是槽数，不是必须有 16384 台机器，也不是一致性哈希环的虚拟节点数。hash tag 能保证同槽，不能保证键分散或跨节点原子提交。

延伸阅读 / 原始参考资料

Redis 官方 • Cluster 规范

14. 缓存穿透、击穿与雪崩如何区分？

缓存与业务 / 基础

穿透是不存在的数据反复回源，击穿是热点失效引发并发回源，雪崩是大量失效或缓存服务故障导致整体压力突增。这些是工程场景术语，解决方案要匹配流量形态，并保护后端容量。

回答要点

1. 穿透可通过输入校验、短 TTL 的空值缓存及布隆过滤器减少无效回源。布隆过滤器可能误判存在，必须允许继续查权威数据；更新遗漏会影响新数据可见性，负缓存也应在创建后失效。
2. 击穿可用请求合并、短租约互斥或提前刷新降低同一热点的并发加载。若允许短暂旧数据，可使用逻辑过期后台更新，但必须设定允许陈旧时间、失败回退和重建者崩溃后的处理。
3. 雪崩可通过 TTL 抖动、热点预热、分批刷新及缓存高可用降低发生概率。缓存故障时不能把所有请求无限制压给数据库，应设置限流、超时、隔离与有意义的降级，并实测后端承载。

常见追问

追问：缓存空值有什么副作用？刚创建的数据可能被旧空结果遮蔽，恶意随机键还能制造大量负缓存占内存。应控制 TTL、键数量与输入范围，创建成功后主动失效相关负缓存，并根据访问模式决定是否布隆过滤。

易错点

术语不是严格协议定义，面试先说清触发条件。不要认为加一个分布式锁就解决所有缓存故障，它只协调特定键的重建，不能替代整体容量控制。

延伸阅读 / 原始参考资料

Redis 官方 • 旁路缓存

Redis 官方 • 布隆过滤器

15. MySQL 与 Redis 缓存如何保持一致？

缓存与业务 / 进阶

旁路缓存通常先更新数据库，再删除缓存，让后续读取重新加载。数据库与 Redis 没有天然原子提交，这个顺序仍可能存在竞态或失效失败；可用幂等重试、事务 outbox 或 CDC 收敛，并明确允许陈旧窗口。

回答要点

1. 读请求先查缓存，未命中再读数据库并回填；写请求提交数据库后使缓存失效。删除缓存失败需要可重试的失效任务，若直接同时覆盖数据库与缓存，乱序写入更容易让旧值覆盖新值。
2. 并发读可能在写提交前拿到旧值，却在写请求删缓存之后回填旧结果。TTL 限制一般旧值寿命，但长请求、不断刷新或异步失效延迟也会影响窗口，因此不能仅凭顺序宣称绝对一致。
3. 可把失效事件与业务修改写入同一数据库事务，再由可靠消费者重试删除；或通过 CDC 传播变化。消费者需幂等并处理乱序，关键读取可直接读权威数据库，版本化缓存也可防止旧值覆盖新值。

常见追问

追问：延迟双删一定安全吗？不是。第二次删除只能覆盖某个估计的并发窗口，读耗时、任务失败和调度延迟变化会突破固定等待。它可降低竞态概率，但严格要求需依赖可靠事件、版本控制或统一的权威读写边界。

易错点

不要承诺“先写数据库再删缓存就强一致”。该模式是实用的收敛方案，业务需要说明能容忍多久旧值，并给失效失败、读写竞态和恢复流程明确处理。

延伸阅读 / 原始参考资料

Redis 官方 • 旁路缓存实现与失效边界

16. Redis 分布式锁怎么写，有哪些不能保证的事？

缓存与业务 / 进阶

常见单实例锁用 `SET key token NX PX lease` 原子设置占有与租约，释放时原子比较令牌再删除。租约过期、进程停顿与异步故障切换可能让多个执行者重叠，锁本身不能保证外部资源永远只有一个写入者。

回答要点

1. 令牌需为每次获取唯一的不可混淆值，释放时不能直接 `DEL`，否则旧持有者恢复后可能删除新持有者的锁。比较与删除必须在一个短脚本或等价原子操作中完成，获取和过期也必须合并。
2. 租约比业务耗时短会过期，续租仍可能受停顿、网络分区或调度影响。客户端应检查是否继续拥有锁并限制任务时间，但已经开始的外部写入不会被 Redis 自动撤销，也不能靠锁状态终止一切旧任务。

3. 对关键外部资源，可使用单调 fencing token 让资源拒绝旧持有者写入，或由数据库约束和幂等实现最终正确性。随机锁令牌用于确认所有权，不能自动代替单调 fencing；多节点方案也有时钟和故障假设。

```
SET lock:job:42 unique-request-token NX PX 10000
```

-- 释放使用比较令牌后 DEL 的原子脚本

常见追问

追问：主从切换为何可能破坏互斥？主节点确认锁后，副本尚未收到该写入就发生故障，提升后的节点会认为锁不存在，于是另一个客户端可获取。WAIT 可降低某些风险但不能无条件消除；关键业务需更强的资源端约束或协调协议。

易错点

不要把 Redis 锁当资金、库存或任务唯一执行的唯一证明。其保证依赖部署、租约与故障模型，必须说明目标是减少重复工作还是保证外部状态严格互斥。

延伸阅读 / 原始参考资料

Redis 官方 • 分布式锁

17. Pipeline 为什么提速，和事务有何区别？

消息与运维 / 基础

Pipeline 连续发送一批命令再集中接收响应，主要减少网络往返和系统调用开销。它不提供事务式连续执行或回滚，也不能让有数据依赖的客户端计算自动在批内完成，应控制批量大小。

回答要点

1. 逐条请求需要每条等待往返，高延迟网络下即使命令很快总吞吐也受 RTT 限制。批量发送可摊薄网络等待，但服务端实际命令计算仍要执行，复杂度不会因 Pipeline 而从线性变为常数。
2. 一批请求会占用客户端和服务端的回复缓冲，过大批次可能增加内存、排队与尾延迟。按字节数和命令数控制批次，并考虑 Cluster 中路由到多个节点后需分别发送和聚合结果。
3. 连接断开时一部分命令可能已执行但响应丢失，批量重试必须考虑幂等。Pipeline 只是传输优化，若需检查后修改用 Lua 或合适事务，若需要拿到第一条结果才能决定第二条则要重新设计。

常见追问

追问：Pipeline 与 MGET 怎么选？多个独立 GET 可用 Pipeline 降低往返，同槽且适合的多个键可用 MGET 一条命令返回。Cluster 跨槽受命令与客户端能力限制，且大量返回数据仍占网络和内存，应根据拓扑与大小决定。

易错点

不要把 Pipeline 当作自动 MULTI/EXEC。不同客户端有的批量接口默认包装事务，必须确认具体库配置，不能从一个方法名推断实际原子语义。

延伸阅读 / 原始参考资料

Redis 官方 • Pipeline

18. Stream 消费者组如何实现确认与重试？

消息与运维 / 进阶

Stream 是可保留消息的日志结构，消费者组记录未确认消息。XREADGROUP 读取、XACK 确认，故障后可检查 pending 并认领重试；通常需按至少一次处理设计幂等，不能声称天然端到端恰好一次。

回答要点

1. XADD 写入带 ID 的条目，组内消费者通过 XREADGROUP 分配新消息。交付后条目进入 pending 状态，XACK 只是确认该组已处理，不等于立即从整个 Stream 删除，也不替业务事务提交。
2. 消费者崩溃后，消息不会自动在瞬间交给所有其他消费者。需要检查 XPENDING，并根据合理的空闲阈值用 XAUTOCLAIM 等认领；阈值过短会让慢但存活的消费者与认领者重复处理。
3. 先完成业务提交再确认可减少丢处理风险，但业务提交成功、确认前崩溃会导致再次交付，所以要用唯一消息标识去重。修剪策略、持久化与复制也决定消息保留，不合理 trim 可能删除仍需处理的内容。

```
XGROUP CREATE events workers 0 MKSTREAM
XREADGROUP GROUP workers consumer-1 COUNT 10 STREAMS events >
-- 业务成功并保证幂等后
XACK events workers <message-id>
```

常见追问

追问：Pub/Sub 与 Stream 的区别？Pub/Sub 给在线订阅者推送，通常没有历史保留和消费确认，离线期间消息会丢。Stream 可存储条目与组消费状态，适合重放和重试，但可靠性仍取决于持久化、保留配置与业务幂等。

易错点

XACK 不等于删除消息，也不等于业务只执行一次。消费确认与外部数据库提交是两个系统动作，必须处理两者之间的崩溃窗口。

延伸阅读 / 原始参考资料

Redis 官方 • 消费者组读取

Redis 官方 • 认领 pending 消息

Redis 官方 • 确认消息

19. 大 key 与热 key 如何发现和治理？

消息与运维 / 进阶

大 key 关注单键字节数或集合元素多，热 key 关注访问频率高；两者可能重叠但处理不同。大 key 放大遍历、网络与删除成本，热 key 集中节点负载，应结合采样、命令统计和业务指标定位。

回答要点

1. 用 MEMORY USAGE 等了解单键内存，结合 SCAN 或工具抽样发现异常集合；全量 KEYS 会扫描并阻塞，不适合高负载线上排查。SCAN 分批更温和，但不提供静态一致快照且可能返回重复。
2. 大集合操作尽量限量与分批，避免 HGETALL、全集合运算或一次返回巨量成员。删除可用 UNLINK 将内存回收工作移到后台，但对象分离和异步释放仍有成本，不意味着内存立刻下降。
3. 热读可通过本地缓存、请求合并或容忍旧值的副本读缓解；热写可能需业务可合并的分片计数。单键在 Cluster 仍归一个槽主节点，增加节点只能扩大总容量，不会天然均匀摊开该键操作。

常见追问

追问：什么大小才算大 key？没有统一阈值，要结合命令复杂度、网络带宽、最大延迟和节点负载。一个几 MB 的 String 与几十万成员的 Hash 风险不同，应制定按类型、元素数、总字节和单次返回量区分的预算。

易错点

不要为了拆 key 破坏原子操作或引入更多小对象内存开销。治理前先确认瓶颈是字节量、成员遍历还是访问频率，并明确拆分后的聚合和一致性方式。

延伸阅读 / 原始参考资料

Redis 官方 • 内存用量

Redis 官方 • 渐进扫描

Redis 官方 • 异步释放

20. 如何评估 Redis 的可靠性与尾延迟？

消息与运维 / 进阶

可靠性由持久化、复制、切换与业务重试共同决定，尾延迟则受慢命令、fork、内存和网络影响。应明确允许的数据丢失与恢复时间，监控实际指标，并通过故障演练验证，而不是只看平均延迟与服务存活。

回答要点

1. 区分客户端确认、写入内存、日志追加、fsync 和副本确认几个阶段。WAIT 等待复制确认，WAITAOF 在支持版本中可等待相关 AOF 同步，但都应检查返回结果与超时，不能替代完整恢复验证。
2. SLOWLOG 关注命令执行耗时，通常不包含与客户端通信的全部时间；应用观察的延迟还包括排队、网络和解析。结合延迟监控、命令统计、fork 耗时、内存碎片、持久化状态与复制延迟找具体原因。
3. 故障演练覆盖进程退出、主机故障、网络分区、磁盘满和重启装载，记录恢复耗时与丢写结果。备份要跨故障域保存并实际恢复，业务超时重试要幂等，否则恢复后仍可能出现重复扣减或任务执行。

常见追问

追问：开了 AOF 加一个副本就一定不丢吗？不一定。刷盘策略、复制异步、共同故障域、自动切换选择与设备行为都影响结果。先定义 RPO/RTO，再选择配置、确认方式、备份和业务容错，并用实测说明保证范围。

易错点

不要把服务自动恢复称为数据绝不丢失，也不要将平均 1ms 视为每次 1ms。高峰、持久化任务和异常负载下的 P99 往往决定用户体验。

延伸阅读 / 原始参考资料

Redis 官方 • 慢命令日志

Redis 官方 • AOF 同步等待

Redis 官方 • 运行指标